Google 288
Algorithm Primer

Read this in three days, before you learn the code solutions

---

43 patterns · 228 questions · 738 follow-ups

Every algorithm used by the easy, medium and hard Google study path, explained once, with a short template and the traps to avoid.

How to use it. Each pattern page says how to spot it, the idea in plain words, a short template with clear names, the cost, the traps, the follow-ups to expect, and the list of study-path questions that use it, with one line telling you what the algorithm does there. Some of them are starred, 150 in the whole book, chosen by how recently each was reported to be asked: learn those first, in the star order. Read a pattern, close the book, and write the template from memory. Then the long solutions in the app will read as “this is pattern 24 with a twist” instead of a wall of code.

★ A green star means every starred question of that pattern has a solution on PracHub or HackTheRounds, so you can check your answer against it. learn N is how many questions of that pattern to learn (150 in all); they carry big orange stars in the book.

Contents

Seven day plan for the starred questions 4

Priority order: what Google asked most recently 4

Python toolbox and complexity 5

Follow-up families that repeat everywhere 6

Pattern finder: from the words of the question to the section 6

Patterns, most questions to learn first

☐ 15. Dict of per-key state (update, expire, rank) learn 7 16

☐ 16. Sweep over events in time order ★ learn 7 17

☐ 32. Trie: nested dicts, one character per level ★ learn 7 27

☐ 1. Union-find: merge the pairs, then ask who is together ★ learn 6 7

☐ 3. A heap of what is available: pop the best, put back the changed one ★ learn 6 9

☐ 4. Backtracking: choose, recurse, undo ★ learn 6 10

☐ 12. Recurrence on a position or a range (memo or table) ★ learn 6 15

☐ 5. Best-first search with a heap: Dijkstra and its minimax twin ★ learn 5 10

☐ 19. Greedy: scan once, carry the best so far, act only when forced ★ learn 5 19

☐ 21. Prefix sums, tier 1: precompute once, query in O(1) learn 5 20

☐ 27. Group by a canonical key learn 5 24

☐ 29. Nested structures: a stack of contexts, or recursion with a cache ★ learn 5 25

☐ 2. Frontier search (BFS or DFS), tier 1: spread, count, measure distance ★ learn 4 8

☐ 17. Binary search on a yes/no condition ★ learn 4 18

☐ 18. Sorted intervals: merge while they touch, then read gaps or search ★ learn 4 18

☐ 25. Sliding window: grow on the right, shrink on the left learn 4 22

☐ 30. Tree recursion, tier 1: visit a node, ask children, report upward ★ learn 4 25

☐ 40. Parent pointers: store who is above each node and climb ★ learn 4 31

☐ 8. Top-K by frequency: Counter plus nsmallest ★ learn 3 12

☐ 11. Rolling state: the next state comes only from the previous one learn 3 14

☐ 13. Grid DP: each cell from its neighbours above and to the left ★ learn 3 15

☐ 14. Counting or reaching an amount: the knapsack loop learn 3 16

☐ 20. Count pairs by looking up each partner learn 3 20

☐ 23. bisect on a sorted list: position, range count, keep sorted ★ learn 3 21

☐ 24. Sets and counters across two collections learn 3 22

☐ 26. Two pointers moving toward each other (or along two lists) learn 3 23

☐ 33. A dict for lookup plus an ordered container for who is next ★ learn 3 27

☐ 36. Simulate a machine: a class with state and one branch per event ★ learn 3 29

☐ 37. Digits and characters: peel one at a time and carry ★ learn 3 30

☐ 38. Split into pieces, fix each piece, join (and word wrap) ★ learn 3 30

☐ 39. Walk two strings at once with two indices learn 3 31

☐ 6. Topological sort (Kahn) and cycle detection ★ learn 2 11

☐ 7. Frontier search, tier 2: free steps, early exit, neighbours worked out on the fly ★ learn 2 12

☐ 22. Runs and groups: extend while it fits, restart when it breaks learn 2 21

☐ 28. Prefix sums, tier 2: running total plus a dict of earlier totals ★ learn 2 24

☐ 31. Tree DP: build a parent order once, add up from the leaves learn 2 26
☐ 9. Games: win or lose with a cache, and guessing with feedback ★ learn 1 13
☐ 10. K-way merge: a heap of the next item from each source ★ learn 1 13
☐ 34. Build or transform a grid with index arithmetic ★ learn 1 28
☐ 35. Look at the input once and keep only a small summary ★ learn 1 28
☐ 41. Random pick without repeats: swap with the last and pop ★ learn 1 32
☐ 42. Tree recursion, tier 2: pass a rule down, return a node or yes/no up ★ learn 1 33
☐ 43. Build and serialize: write out a structure, read it back with the same walk ★ learn 1 33

Appendix: algorithms hiding inside single questions 35

Coverage by section and source 37

After the 150 starred questions: what to do next 41

Behavioural questions: sample answers from your resume 42

Your friend's practice problem set (the ColorStack reports) 44

Seven day plan for the starred questions

The 150 starred questions split into seven days of about 20 to 25 questions each, grouped by theme so one day uses related ideas. Day 7 is lighter on purpose: use the spare time to redo what you got wrong on days 1 to 6. Click a section name to jump to it.

Day	Sections (learn count)	Questions
1	**Graphs and search** 1. Union-find (6); 2. Frontier search (BFS or DFS), tier 1 (4); 5. Best-first search with a heap (5); 6. Topological sort (Kahn) and cycle detection (2); 7. Frontier search, tier 2 (2); 9. Games (1); 34. Build or transform a grid with index arithmetic (1)	21
2	**Heaps and ordered containers** 16. Sweep over events in time order (7); 3. A heap of what is available (6); 8. Top-K by frequency (3); 33. A dict for lookup plus an ordered container for who is next (3); 10. K-way merge (1); 35. Look at the input once and keep only a small summary (1)	21
3	**Hashing and prefix sums** 15. Dict of per-key state (update, expire, rank) (7); 21. Prefix sums, tier 1 (5); 27. Group by a canonical key (5); 20. Count pairs by looking up each partner (3); 24. Sets and counters across two collections (3); 28. Prefix sums, tier 2 (2)	25
4	**Dynamic programming and backtracking** 4. Backtracking (6); 12. Recurrence on a position or a range (memo or table) (6); 11. Rolling state (3); 13. Grid DP (3); 14. Counting or reaching an amount (3)	21
5	**Arrays, windows and intervals** 19. Greedy (5); 17. Binary search on a yes/no condition (4); 18. Sorted intervals (4); 25. Sliding window (4); 23. bisect on a sorted list (3); 26. Two pointers moving toward each other (or along two lists) (3); 22. Runs and groups (2)	25
6	**Trees and nesting** 32. Trie (7); 29. Nested structures (5); 30. Tree recursion, tier 1 (4); 40. Parent pointers (4); 31. Tree DP (2); 42. Tree recursion, tier 2 (1); 43. Build and serialize (1)	24
7	**Strings, simulation and catch-up** 36. Simulate a machine (3); 37. Digits and characters (3); 38. Split into pieces, fix each piece, join (and word wrap) (3); 39. Walk two strings at once with two indices (3); 41. Random pick without repeats (1)	13

Priority order: what Google asked most recently

The patterns ranked by how often Google was reported asking their questions recently. If time is short, study the patterns in the order of this table. Rank uses questions reported in the last 90 days first, then all dated questions, then the study-path score.

Rank	#	App	Pattern	Last 90 days	Dated	Questions
1	15	10	Dict of per-key state (update, expire, rank)	4	7	8
2	29	27	Nested structures	4	5	5
3	1	9	Union-find	3	7	7
4	13	26	Grid DP	3	4	5
5	16	2	Sweep over events in time order	2	8	9
6	2	1	Frontier search (BFS or DFS), tier 1	2	6	6
7	3	4	A heap of what is available	2	6	8
8	17	11	Binary search on a yes/no condition	2	5	7
9	30	3	Tree recursion, tier 1	2	5	8
10	4	7	Backtracking	2	5	7
11	18	16	Sorted intervals	2	4	6
12	19	12	Greedy	2	4	5
13	31	31	Tree DP	2	4	5
14	20	30	Count pairs by looking up each partner	2	2	3
15	32	17	Trie	1	8	8
16	12	5	Recurrence on a position or a range (memo or table)	1	5	8
17	5	6	Best-first search with a heap	1	5	8

Rank	#	App	Pattern	Last 90 days	Dated	Questions
18	21	15	Prefix sums, tier 1	1	4	6
19	33	18	A dict for lookup plus an ordered container for who is next	1	4	5
20	22	8	Runs and groups	1	4	4
21	23	14	bisect on a sorted list	1	3	6
22	6	29	Topological sort (Kahn) and cycle detection	1	3	4
23	7	28	Frontier search, tier 2	1	3	4
24	24	13	Sets and counters across two collections	1	3	4
25	25	40	Sliding window	1	2	4
26	34	42	Build or transform a grid with index arithmetic	1	2	3
27	35	25	Look at the input once and keep only a small summary	0	6	6
28	8	23	Top-K by frequency	0	5	5
29	9	39	Games	0	5	5
30	36	22	Simulate a machine	0	5	5
31	37	24	Digits and characters	0	4	4
32	38	43	Split into pieces, fix each piece, join (and word wrap)	0	4	5
33	39	21	Walk two strings at once with two indices	0	3	6
34	26	20	Two pointers moving toward each other (or along two lists)	0	2	6
35	10	35	K-way merge	0	2	4
36	40	41	Parent pointers	0	2	4
37	41	37	Random pick without repeats	0	2	2
38	11	19	Rolling state	0	1	5
39	27	34	Group by a canonical key	0	1	5
40	28	33	Prefix sums, tier 2	0	1	3
41	42	36	Tree recursion, tier 2	0	1	3
42	14	32	Counting or reaching an amount	0	0	4
43	43	38	Build and serialize	0	0	3

Python toolbox and complexity

Pick the target complexity from the input size. Interview inputs are chosen so one family of algorithms fits.

n up to	Target	Typical tools
20 to 25	$O(2^n)$, $O(n!)$	backtracking, bitmask DP
1,000	$O(n^2)$	table DP, all pairs, nested loops
100,000	$O(n \log n)$	sort, heap, binary search, balanced structure
1,000,000 and up	$O(n)$ or $O(\log n)$	one pass, hashing, two pointers, math

Costs of the tools you will use all day.

Structure	Fast	Slow (avoid in loops)
`list`	append, pop, index $O(1)$	`insert(0)`, `pop(0)`, `x in list`: $O(n)$
`dict`, `set`	get, set, delete, in: $O(1)$ average	iteration order is insertion order only
`deque`	append, popleft, appendleft $O(1)$	indexing in the middle $O(n)$
`heapq`	push, pop $O(\log n)$; peek `h[0]` $O(1)$	search or delete a middle item $O(n)$
sorted list + `bisect`	search $O(\log n)$	`insort` $O(n)$ (shifts items)
`str`	index, slice $O(k)$	`s += t` in a loop: build a list and join

Standard library you are expected to know.

- `Counter(items)`: counts; `most_common(k)`; subtract with `c[x] -= 1`. `defaultdict(list)` or `defaultdict(int)` removes the "key exists?" check.
- `deque`: BFS queue, sliding window, 0-1 BFS. `OrderedDict.move_to_end` and `popitem(last=False)` give an LRU in a few lines.
- `heapq`: `heappush`, `heappop`, `heapify`, `heapreplace`, `nsmallest(k, xs, key=...)`. It is a min-heap: push `-x` for a max-heap. Push tuples `(priority, tiebreak, item)` so items never get compared.
- `bisect_left(a, x)` first index with `a[i] >= x`; `bisect_right(a, x)` first index with `a[i] > x`; `insort`.
- `lru_cache(maxsize=None)` from functools memoises a function of hashable arguments (use tuples, not lists). From itertools: `combinations`, `permutations`, `accumulate` (prefix sums). `zip(*matrix)` transposes.
- `sorted(xs, key=lambda x: (-count[x], x))`: sort by several criteria; sort is stable. Bits: `x & (x-1)` clears the lowest set bit, `1 << i` is bit i, `x >> i & 1` reads it.

- Recursion depth is about 1000 in Python: for deep trees or chains use an explicit stack or a queue (BFS), or raise the recursion limit and say why it is safe.

The loop to run on every question. (1) Restate it and ask about duplicates, ties, empty input, negative numbers, the size of the input. (2) Say a brute force and its cost. (3) Name the pattern in this book and the state or data structure it needs. (4) State the complexity before coding. (5) Code with clear names. (6) Test on an empty case, a one item case, a tie, and a case that breaks your first idea. (7) Be ready for the follow-ups listed with each pattern.

Follow-up families that repeat everywhere

Across the questions in this book the same follow-ups return in different clothes. Learn one answer to each family and you can answer most of them.

The input is a stream or does not fit in memory. Keep a small summary instead of the data: a running max, a size-k heap, a count dict, a set of ids from the smaller file. For two huge inputs partition both by a hash of the key and process one partition at a time. Many machines: map (emit key and value), shuffle by key, reduce.

Make it thread safe. One lock around each method is correct and simple; finer locks (per key or per bucket) give more concurrency but need a lock order to avoid deadlock. Read-mostly data can use copy-on-write. Say what is atomic.

Items can also be deleted or updated. A plain sorted list or prefix sum breaks: move to a Fenwick tree or a segment tree, a sorted container with $O(\log n)$ updates, lazy deletion in a heap, or swap-and-pop in a list with a position dict. Union-find cannot split groups: process offline in reverse or rebuild.

Return the path or the items, not only the number. Store a parent or choice for each state when you first improve it, then walk back from the goal: `parent[v] = u` in BFS and Dijkstra, the chosen split in DP, the chosen index in a greedy scan.

Weights, costs or priorities are added. BFS becomes Dijkstra (non-negative weights, or 0-1 BFS for 0 and 1), greedy may stop being correct (re-check the exchange argument), plain counts become sums stored in the same tables.

Recursion is too deep or too slow. Convert to an explicit stack or a queue; add a cache to turn exponential into polynomial; compute subtree values by sweeping a parent-first order backwards.

Do it in place, or in $O(1)$ extra space. Reuse the input as the table (swap each value into its own slot, encode old and new state in different bits), or use two pointers instead of a set after sorting.

Online arrivals, many queries, repeated calls. Precompute once (prefix sums, adjacency, a trie, a sorted list) and make each query cheap; for arrivals that cannot be sorted ahead use a heap or a balanced structure.

The input is huge numerically. Use modular arithmetic, matrix exponentiation or fast doubling for recurrences, a closed form for counting, or strings for numbers beyond machine size.

Prove or justify. Greedy: name the exchange argument and a counter-example for the neighbouring problem. Binary search: show the predicate is monotone. DP: state the meaning of the state. Dijkstra: weights are non-negative so a popped node is final.

Pattern finder: from the words of the question to the section

Read the question, match the words, open the section. The first number is the pattern's place in this book; App is its section number in the study workspace.

#	App	Pattern	Words that give it away
1	9	Union-find	connected components, groups, are A and B related, add land one cell at a time, points within a radius.
2	1	Frontier search (BFS or DFS), tier 1	number of islands, flood fill, rotten oranges, shortest path in an unweighted graph, distance to nearest X.
3	4	A heap of what is available	minimum rooms, streaming median, reusable locker, schedule on identical CPUs, no two adjacent the same.
4	7	Backtracking	all permutations / combinations, generate parentheses, count paths of length k, can this hand win, assign people to tasks.
5	6	Best-first search with a heap	shortest path with weights, earliest arrival with timetables, minimise the maximum height on a path, swim in rising water.
6	29	Topological sort (Kahn) and cycle detection	course schedule, build order, are these race results consistent, validate dependencies.
7	28	Frontier search, tier 2	edge direction flips cost 1, reach with blocked nodes, shortest directed cycle through v, radios within a radius.

#	App	Pattern	Words that give it away
8	23	Top-K by frequency	k most frequent words, top chatters, task scheduler with cooldown, bottom k.
9	39	Games	who wins with optimal play, merge stacks game, guess the secret word, Wordle strategy.
10	35	K-way merge	merge k sorted lists, k smallest pair sums, sort a nearly sorted array, top-k subset sums.
11	19	Rolling state	climbing stairs, house robber, paint houses, fibonacci, dice with limits.
12	5	Recurrence on a position or a range (memo or table)	edit distance, longest palindromic subsequence, word break, split into expressions, count decompositions.
13	26	Grid DP	unique paths, largest square, longest run of one colour, paths with checkpoints.
14	32	Counting or reaching an amount	coin change, partition equal subset sum, target sum with plus and minus, unbounded items that fill a capacity.
15	10	Dict of per-key state (update, expire, rank)	remember the last time we saw X, top users, within the last 10 seconds, design a logger / counter.
16	2	Sweep over events in time order	minimum rooms, maximum overlap, skyline, who is online at time t, servers needed.
17	11	Binary search on a yes/no condition	minimum speed / capacity / time such that ..., largest length such that ..., first bad version, median of two sorted arrays.
18	16	Sorted intervals	merge intervals, do bookings conflict, free time, insert an interval, union of two lists.
19	12	Greedy	minimum number of ..., maximum profit, largest number you can form, cover all holes, jumps.
20	30	Count pairs by looking up each partner	pairs with difference k, two ways to write as a sum, rectangle / square from points.
21	15	Prefix sums, tier 1	sum of a range, balanced split, pivot index, equal halves, area up to a height.
22	8	Runs and groups	longest contiguous run, group consecutive items, streak, one replacement allowed.
23	14	bisect on a sorted list	count in a range, k-th largest as items stream in, nearest value, longest increasing subsequence.
24	13	Sets and counters across two collections	appears in both, not in the other list, duplicates, pair each x with 2x.
25	40	Sliding window	longest/shortest subarray or substring such that ..., at most k distinct, minimum window, budget of changes.
26	20	Two pointers moving toward each other (or along two lists)	palindrome, reverse, sorted array and a target pair, merge two sorted lists, triplets that sum to zero.
27	34	Group by a canonical key	anagrams, same shape, shifted words, group strings that are equivalent.
28	33	Prefix sums, tier 2	number of subarrays with sum k, multiple of k, maximum subarray, equal endpoints.
29	27	Nested structures	decode string 3[a2[c]], evaluate expression, nested list weight, expand placeholders.
30	3	Tree recursion, tier 1	height, diameter, right side view, level order, delete leaves, evaluate expression tree.
31	31	Tree DP	total size of a folder, sum of distances from every node, time to inform everyone, cheapest cut.
32	17	Trie	autocomplete, words with a prefix, longest match tokenizer, URL router, dictionary substrings.
33	18	A dict for lookup plus an ordered container for who is next	LRU cache, waitlist, round robin scheduler, remove the cheapest, undo / redo.
34	42	Build or transform a grid with index arithmetic	rotate a matrix, range sum in a 2D matrix, draw a diamond, fill in snake order, block minimum.
35	25	Look at the input once and keep only a small summary	file does not fit in memory, stream, minimise the number of oracle calls, two logs.
36	22	Simulate a machine	design tic-tac-toe, elevator system, iterator over lists, game of life, scorekeeper.
37	24	Digits and characters	add two big numbers, palindrome number, monotone digits, caesar cipher, k-th permutation.
38	43	Split into pieces, fix each piece, join (and word wrap)	snake_case to camelCase, format a number, justify text, wrap at width.
39	21	Walk two strings at once with two indices	is a subsequence, lexicographic order in a custom alphabet, can board A become board B, similar sentences.
40	41	Parent pointers	lowest common ancestor with parent links, validate a tree, employee hierarchy, is this a root.
41	37	Random pick without repeats	random set with O(1) delete, shuffle, unique random generator, no repeat within k songs.
42	36	Tree recursion, tier 2	valid BST, lowest common ancestor, in-order successor, mode in a tree.
43	38	Build and serialize	serialize / deserialize, construct tree from preorder and inorder, print a directory tree, encode a list of strings.

1. Union-find: merge the pairs, then ask who is together ★

Union-find and connectivity (app section 9) 7 questions: 0 easy, 6 medium, 1 hard

Spot it: "connected components", "groups", "are A and B related", "add land one cell at a time", "points within a radius".

Idea: Each element points to a parent; the root is the group name. find(x) follows parents (flattening the chain as it goes), union(a, b) links the two roots. Pairs may come from edges, from neighbouring land cells, or from points within a radius (bucket points into cells of size r and test only the nine nearby cells). Offline path problems: sort edges from best to worst and union until two nodes meet; the last edge is the threshold.

```
parent = list(range(n))                    # items are 0..n-1 (5 of the 8 questions)
# parent = {}                               # names, tuples, cells that appear later:
#                                           # add parent.setdefault(x, x) as the first line of find
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]      # path halving
        x = parent[x]
    return x
def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb: return False              # already together
    parent[ra] = rb;  return True

groups = n                                 # with the dict form: count only items you added
for a, b in pairs:
    if union(a, b): groups -= 1
```

Cost: Close to O(1) per operation in practice: the path halving in find keeps chains short, and no size bookkeeping is needed. **Traps:** Pick the style from the input: ids 0..n-1 use the list (start groups at n); names, tuples or items that appear later use the dict with setdefault (and `find` on an unseen item quietly creates it). Unfriending breaks plain union-find (use offline reverse processing or rebuild). Count groups by decrementing only on a real merge. Keep extra data (boss, size) at the root and carry it over when merging.

Follow-ups: Land can be removed. Many (source, target) queries (build once). Report the largest group size (track size at the root). Directed reachability is not union-find (use BFS or topological order).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Count Islands in a Grid and Track the Count as Land Cells Are Added**: each new land cell: groups += 1, then union it with every land neighbour (groups -= 1 per real merge); report groups after each addition *[PracHub (solution)]*

H ☐ ★**2 Friend Graph with Timeline (Friend / Unfriend Events)**: pairs = friend events sorted by time; the answer is the time of the union that makes groups == 1 *[1Point3Acres (notes), PracHub (solution) as "Earliest Time a Friendship Network Is Fully Connected with Friend and Unfriend Events"]*

M ☐ ★**3 2D Points Connected Components by Radius**: pairs = points within distance r, found by bucketing points into cells of size r and checking only the 9 nearby cells; answer = groups *[1Point3Acres (notes), PracHub (solution) as "Count Clusters of 2D Points Within a Radius"]*

M ☐ ★**4 Find A Threshold-Limited Path With Minimum Required Safety**: pairs = edges sorted from the highest safety down; union until find(s) == find(t); the answer is the safety of that last edge *[PracHub (solution)]*

M ☐ ★**5 Count Clusters of 2D Points Within a Radius**: identical to the line above: cell buckets give the candidate pairs, answer = groups *[PracHub (solution), 1Point3Acres (notes) as "2D Points Connected Components by Radius"]*

M ☐ ★**6 Determine Reporting Relationships**: pairs = peer links; keep the boss at the root and carry it over when two groups merge *[PracHub (solution)]*

M ☐ **Determine whether two nodes are related**: pairs = the given edges; the query is find(a) == find(b) *[PracHub (solution)]*

2. Frontier search (BFS or DFS), tier 1: spread, count, measure distance ★

Graph traversal: BFS and DFS (app section 1) 6 questions: 0 easy, 5 medium, 1 hard

Spot it: "number of islands", "flood fill", "rotten oranges", "shortest path in an unweighted graph", "distance to nearest X".

Idea: A frontier search has starts, a rule for which neighbours are usable, and a seen set. DFS or BFS both visit everything reachable. Use BFS when you need distances (layer by layer, each layer is one more step). Multi-source BFS puts every start in the queue at distance 0, so you get the distance to the nearest start for all cells in one pass. Counting components: loop over all cells and start a search from each unseen land cell.

```
def nearest_distance(grid, starts):
    rows, cols = len(grid), len(grid[0])
    dist = {s: 0 for s in starts}
    queue = deque(starts)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r+1, c), (r-1, c), (r, c+1), (r, c-1)):
            if 0 <= nr < rows and 0 <= nc < cols and (nr, nc) not in dist \
                    and grid[nr][nc] != '#':
                dist[(nr, nc)] = dist[(r, c)] + 1
                queue.append((nr, nc))
```

```
    return dist                                 # missing key = unreachable
```

Cost: O(V + E), for a grid O(rows * cols). **Traps:** Mark cells seen when you enqueue, not when you dequeue (otherwise duplicates). Unreached cells stay unreachable (-1). To rebuild a path store parent[v] = u at first discovery.

Follow-ups: Huge grid (stream rows, or union-find). Cells change over time (rebuild only the affected area). Return every shortest path or count them (add counts per layer). Weighted edges (Dijkstra).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Number of Islands**: loop over every cell; each unseen land cell starts a new frontier_search (share one seen set); islands += 1 per start *[1Point3Acres (notes), PracHub (solution) as "Solve Number of Islands"]*

M ☐ ★**2 Rotten Oranges / Multi-Source BFS (taxis)**: starts = every rotten orange; usable = fresh orange; answer = rounds taken, or -1 if a fresh orange stays unreached *[1Point3Acres (notes), PracHub (solution) as "Solve Rotting Oranges and Bus Walk Reachability", Aced as "Rotting Oranges"]*

H ☐ ★**3 Infection Spread on a Grid (Cellular Automaton)**: starts = every infected cell; usable = healthy cell; the answer is the largest distance reached (number of rounds) *[PracHub (solution)]*

M ☐ ★**4 Find shortest relationship path using BFS**: one start; also store parent[v] = u when you first reach v; stop at the target and walk parent back to rebuild the path *[PracHub (solution), Aced (solution) as "Degrees of Friendship"]*

M ☐ **Compute distance to nearest taxi in grid**: starts = every taxi cell; usable = not a wall; the dist table is the answer, cells never reached stay -1 *[PracHub (solution)]*

M ☐ **Solve Grid and Counting Problems**: the island count again (queue version); the second part of the question is a short counting formula, no search *[PracHub (solution)]*

3. A heap of what is available: pop the best, put back the changed one ★

Heaps and top-k (app section 4) 8 questions: 0 easy, 7 medium, 1 hard

Spot it: "minimum rooms", "streaming median", "reusable locker", "schedule on identical CPUs", "no two adjacent the same".

Idea: Keep a heap of the things you can pick right now, keyed by what makes one better. Pop the best, use it, and push it back with its new state (a room with a new end time, a task with a smaller count). Two heaps hold a stream median: a max-heap for the lower half, a min-heap for the upper half, rebalanced so sizes differ by at most 1. Python only has a min-heap, so negate values for a max-heap.

```
free_at = []                                # heap of end times of rooms in use
for start, end in sorted(meetings):
    if free_at and free_at[0] <= start:
        heapq.heapreplace(free_at, end)     # reuse the room that frees first
    else:
        heapq.heappush(free_at, end)        # open a new room
rooms = len(free_at)

low, high = [], []                          # max-heap (negated), min-heap
def add(x):
    heapq.heappush(low, -x)
    heapq.heappush(high, -heapq.heappop(low))
    if len(high) > len(low): heapq.heappush(low, -heapq.heappop(high))
def median(): return -low[0] if len(low) > len(high) else (-low[0] + high[0]) / 2
```

Cost: O(log n) per push and pop. **Traps:** Tuples compare left to right: add a tie-breaker (an id) so unorderable objects never compare. Lazy deletion for stale entries. State why greedy is right (the soonest-free resource is the best one to reuse).

Follow-ups: Return which room each meeting uses. Rank by busy time. Online arrivals. Sliding median (two heaps with lazy deletion). Songs by the same artist at least k apart (hold artists in a cooldown queue).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Trip Expense Settlement**: a heap of creditors and a heap of debtors; the biggest of each settles min(amount) and the remainder goes back *[HackTheRounds #42 (solution), PracHub (solution) as "Compute minimal transfers to settle group expenses", Aced]*

M ☐ ★**3 Implement a Reusable Locker Allocation System**: heap of freed locker numbers; take the smallest freed one before handing out a new number *[PracHub (solution)]*

M ☐ ★**4 Streaming Median (LC 295) with Follow-ups**: two heaps: a max-heap for the lower half and a min-heap for the upper half; keep them balanced *[1Point3Acres (notes), PracHub (solution) as "Track the Running Median of an Integer Stream"]*

H ☐ ★**5 Huffman Tree Construction**: pop the two smallest weights, push their sum as a new node; repeat until one node remains *[1Point3Acres (notes), PracHub (solution) as "Implement binary encode/decode with frequency tree"]*

M ☐ ★**6 Compute minimum number of rooms needed**: heap of end times; a meeting reuses the room that frees first if it ended, otherwise push a new room *[PracHub (solution), Aced as "Given a list of car rental requests with start and end times, assign cars to rentals such ", 1Point3Acres (notes) as "Meeting Rooms / Interval Scheduling Variants"]*

M ☐ **Simulate room assignments for scheduled meetings**: a heap of free room numbers and a heap of (end time, room); a delayed meeting takes the earliest-ending room and shifts later *[PracHub (solution)]*

M ☐ **Design server allocation for multi-type nodes**: a heap per type of (flexibility, server); skip servers already allocated (lazy deletion) *[PracHub (solution)]*

M ☐ ★2 ★ **Playlist Shuffle No Adjacent Artists**: pop the artist with the most songs left, hold it for one turn, then push it back with a smaller count *[HackTheRounds #43 (solution)]*

4. Backtracking: choose, recurse, undo ★

Recursion and backtracking (app section 7) 7 questions: 0 easy, 6 medium, 1 hard

Spot it: "all permutations / combinations", "generate parentheses", "count paths of length k", "can this hand win", "assign people to tasks".

Idea: Build the answer one choice at a time. At each step loop over the legal choices, apply one, recurse, then undo it so the next choice starts from the same state. Prune as early as possible: only add a close bracket when closed < opened, always try the smallest remaining tile first for Mahjong. If you only need the count, return numbers instead of storing every solution. Matching problems use an augmenting path: if the preferred slot is taken, recursively ask its owner to move.

```
def parentheses(n):
    out = []
    def build(current, opened, closed):
        if len(current) == 2 * n:
            out.append(current);  return
        if opened < n:  build(current + '(', opened + 1, closed)
        if closed < opened:  build(current + ')', opened, closed + 1)
    build('', 0, 0)
    return out

def permutations(items):
    out, chosen, used = [], [], [False] * len(items)
    def build():
        if len(chosen) == len(items): out.append(chosen[:]);  return
        for i, x in enumerate(items):
            if used[i]: continue
            used[i] = True;  chosen.append(x)
            build()
            chosen.pop();  used[i] = False        # undo
    build()
    return out
```

Cost: Exponential: n! permutations, 2^n subsets, Catalan(n) bracket strings. Pruning cuts the constant. **Traps:** Copy the list when you store it (`chosen[:]`). Undo exactly what you did. Skip equal neighbours (after sorting) to avoid duplicate results. Use a bitmask for visited sets when n is small.

Follow-ups: Count only. Iterative version with an explicit stack. Next lexicographic permutation. k-th permutation directly (factorial number system). Priorities or capacities in matching.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Mahjong Winning Hand Detection (14 Tiles)**: take the smallest tile; try it as a triple, then as the start of a run; undo the counts after each try *[1Point3Acres (notes), PracHub (solution) as "Determine if a 14-tile hand is winning"]*

M ☐ ★2 **Permutations**: the pattern template above *[Aced (solution), PracHub (solution) as "Generate all permutations of an array"]*

M ☐ ★3 **Count Prefix-Matching Simple Paths in a Labeled Graph**: extend the path along edges whose label matches the next query letter, with a bitmask of visited nodes; count per depth *[PracHub (solution)]*

H ☐ ★4 **Matrix Flower Placement with House Adjacency**: place row by row; reject a row early when the houses two rows above are still not next to a flower *[1Point3Acres (notes), PracHub (solution) as "Solve Flower Placement and Directory Deletion"]*

M ☐ ★5 **Generate Parentheses**: choices are '(' and ')'; allowed when opened < n, and ')' only when closed < opened; complete at length 2n *[Aced (solution), PracHub (solution) as "Generate All Valid Combinations of Balanced Parentheses"]*

M ☐ ★6 **Determine if a 14-tile hand is winning**: the same try-triple-then-run recursion over a count array, after choosing the pair *[PracHub (solution), 1Point3Acres (notes) as "Mahjong Winning Hand Detection (14 Tiles)"]*

M ☐ **Match people to questions using tags and priorities**: try each question of a person; if it is taken, recursively ask its owner to move (augmenting path); undo by not recording on failure *[PracHub (solution)]*

5. Best-first search with a heap: Dijkstra and its minimax twin ★

Dijkstra and shortest paths (app section 6) 8 questions: 0 easy, 4 medium, 4 hard

Spot it: "shortest path with weights", "earliest arrival with timetables", "minimise the maximum height on a path", "swim in rising water".

Idea: Always expand the unfinished node with the smallest cost so far, kept in a heap. When a node is popped its cost is final (weights non-negative). The cost does not have to be a sum: use max(cost, edge) for "minimise the worst step" (minimax), or the

arrival time at the next station for timetables (you may leave only at or after you arrive). For maximising a quantity, push negated values.

```
def dijkstra(graph, source, combine=lambda cost, w: cost + w):
    best = {source: 0};  heap = [(0, source)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best.get(u, float('inf')): continue    # stale entry
        for v, w in graph[u]:
            new_cost = combine(cost, w)
            if new_cost < best.get(v, float('inf')):
                best[v] = new_cost
                heapq.heappush(heap, (new_cost, v))
    return best

# minimax: combine=lambda cost, w: max(cost, w)
```

Cost: O((V + E) log V). **Traps:** Skip stale heap entries. Negative edges break it (use Bellman-Ford). Track parent[v] to rebuild the path. With unit weights plain BFS is enough; with only 0 and 1 use 0-1 BFS.

Follow-ups: Return the path. Fewest transfers. Several travellers (run from each, combine at meeting nodes). Weights change over time. Layover rules (shift the departure test).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Path Minimizing the Maximum Cell (Swim in Rising Water)**: the same minimax loop with a given start and end cell (Swim in Rising Water) *[1Point3Acres (notes), PracHub (solution) as "Compute earliest arrival in time-grid"]*

M ☐ ★**2 Find the shortest path in a graph.**: combine = cost + w; also store parent[v]; stop at the target and rebuild the path *[Aced, PracHub (solution) as "Compute shortest path in cyclic graph"]*

M ☐ ★**3 Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.**: cost = arrival time; neighbors only offers trains that leave at or after the current time; combine = the train's arrival time *[Aced, PracHub (solution) as "Reachability Between Stations in a Timed Train Schedule"]*

H ☐ ★**4 Find minimum threshold enabling grid path**: the same minimax loop: the smallest threshold that lets you walk from corner to corner *[PracHub (solution)]*

M ☐ ★**5 Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.**: maximise the minimum distance to the cat along the path: push negated values, combine = min(value, cat distance of the cell) *[Aced, PracHub (solution) as "Maximize the minimum value along a grid path"]*

H ☐ **Minimize maximum height along a grid path**: the same minimax loop on a height grid *[PracHub (solution), 1Point3Acres (notes) as "Path Minimizing the Maximum Cell (Swim in Rising Water)"]*

H ☐ **Minimize travel time with optional meeting point**: three runs: from a, from b, and from the destination on the reversed graph; answer = min over meeting points m of max(da, db) + dd *[PracHub (solution)]*

H ☐ **Compute minimax grid path and network delay**: combine = max(cost, height of the next cell); the answer is the cost at the bottom-right cell *[PracHub (solution)]*

6. Topological sort (Kahn) and cycle detection ★

Topological sort and cycles (app section 29) 4 questions: 0 easy, 4 medium, 0 hard

Spot it: "course schedule", "build order", "are these race results consistent", "validate dependencies".

Idea: Compute the in-degree of every node. Put all in-degree 0 nodes in a queue, repeatedly remove one, append it to the order, and decrease its neighbours' in-degrees (adding new zeros). If the order contains every node the graph has an order; otherwise the remaining nodes lie on or behind a cycle. Layering by rounds gives the minimum number of semesters.

```
def topo_order(nodes, edges):              # edge (a, b): a must come before b
    indegree = {v: 0 for v in nodes};  after = defaultdict(list)
    for a, b in edges:
        after[a].append(b);  indegree[b] += 1
    queue = deque(v for v in nodes if indegree[v] == 0)
    order = []
    while queue:
        v = queue.popleft();  order.append(v)
        for w in after[v]:
            indegree[w] -= 1
            if indegree[w] == 0: queue.append(w)
    return order if len(order) == len(indegree) else None
```

Cost: O(V + E). **Traps:** Nodes that only appear in edges still need an in-degree entry. Direction of the edge (prerequisite to course). Duplicate edges inflate the in-degree consistently, which is fine.

Follow-ups: Report one actual cycle (DFS with colours). All missing references. Semesters (process the queue layer by layer). Recursive DFS overflow on long chains (use Kahn).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front

of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Course Scheduling and Dependency Variants**: build the vertex set from the edges too; return the order or None *[1Point3Acres (notes), PracHub (solution) as "Produce a Deterministic Course Plan"]*

M ☐ ★2 **Course Schedule**: edge = prerequisite -> course; return the order, or an empty list when there is a cycle *[Aced (solution), PracHub (solution) as "Course Schedule Feasibility"]*

M ☐ **Solve tree, graph, sliding-window problems**: the course-schedule part is can-finish: len(order) == n; the zigzag and window parts are separate small loops *[PracHub (solution)]*

M ☐ **Validate course catalog dependencies**: first reject prerequisites naming courses that do not exist; then the same peel; report the missing names or the cycle *[PracHub (solution)]*

7. Frontier search, tier 2: free steps, early exit, neighbours worked out on the fly ★

Graph traversal: BFS and DFS (app section 28) 4 questions: 0 easy, 4 medium, 0 hard

Spot it: "edge direction flips cost 1", "reach with blocked nodes", "shortest directed cycle through v", "radios within a radius".

Idea: Same BFS, but the graph is not given as lists: neighbours(u) is computed when needed (all radios within a radius, using squared distances). A mix of free and paid moves is 0-1 BFS: a deque where free steps use appendleft and paid steps use append, so the deque stays sorted by distance. Early exit: stop when the goal is popped. Shortest cycle through v: start at v and return dist[u] + 1 the moment a neighbour of u equals v.

```
def zero_one_bfs(graph, source):
    dist = {source: 0};  queue = deque([source])
    while queue:
        u = queue.popleft()
        for v, weight in graph[u]:          # weight is 0 or 1
            if dist[u] + weight < dist.get(v, float('inf')):
                dist[v] = dist[u] + weight
                if weight == 0: queue.appendleft(v)
                else:           queue.append(v)
    return dist
```

Cost: O(V + E) for BFS and 0-1 BFS; O(V^2) when neighbours are computed from all other nodes. **Traps:** Compare squared distances to avoid square roots. A blocked source means everything is unreachable. A node can be improved after first discovery in 0-1 BFS, so check strictly smaller.

Follow-ups: Arbitrary non-negative weights (Dijkstra). Shortest cycle in the whole graph (BFS from every node). Nodes blocked over time. Queries on the same graph (precompute adjacency).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Shortest Path Nodes and Escaping a Pursuer**: run it from a, from t and from b; a node is on a shortest a-to-t path when da + dt == da[t]; compare with the pursuer's distance db *[1Point3Acres (notes), PracHub (solution) as "Shortest Path, Nodes on Any Shortest Path, and Whether Alice Can Evade Bob"]*

M ☐ ★2 **Minimum Direction Violations in a Directed Graph**: an edge in its own direction costs w = 0, the same edge walked backwards costs w = 1; free steps use appendleft *[1Point3Acres (notes), PracHub (solution) as "Minimize Direction Violations in a Directed Road Network"]*

M ☐ **Compute shortest paths with blocked nodes**: neighbors skip every blocked node; if the source itself is blocked the answer is all -1; unreachable nodes stay -1 *[PracHub (solution)]*

M ☐ **Shortest Directed Cycle Through a Node**: start = v; the goal is an edge that leads back to v, so return dist[u] + 1 the moment a neighbour equals v *[1Point3Acres (notes), PracHub (solution) as "Find minimum cycle cost per node"]*

8. Top-K by frequency: Counter plus nsmallest ★

Heaps and top-k (app section 23) 5 questions: 0 easy, 5 medium, 0 hard

Spot it: "k most frequent words", "top chatters", "task scheduler with cooldown", "bottom k".

Idea: Count with Counter, then take the k best with a size-k heap: heapq.nsmallest(k, items, key=lambda w: (-count[w], w)) gives most frequent first with ties by name. This costs O(d log k) for d distinct items instead of sorting everything. For a cooldown scheduler, only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + number of tasks tied for the biggest).

```
counts = Counter(word for line in lines for word in line.split()[:1])
top = heapq.nsmallest(k, counts, key=lambda w: (-counts[w], w))

def least_interval(tasks, n):
    counts = Counter(tasks).values()
    biggest = max(counts)
    tied = sum(1 for c in counts if c == biggest)
    return max(len(tasks), (biggest - 1) * (n + 1) + tied)
```

Cost: O(N + d log k) time, O(d) space. **Traps:** State the tie-break rule. A stream with too many distinct keys needs an approximate counter (Count-Min sketch, Misra-Gries). The log may not fit on one machine: count per machine and merge, or partition by hash.

Follow-ups: Unbounded stream and bounded memory. top_k must be O(k) per call (maintain buckets by count). Different cooldowns or priorities. Print an actual schedule (max-heap of counts plus cooldown queue).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Top-K Frequent in Large Logs**: count the first word of every line (the speaker); return the names of the k most frequent *[1Point3Acres (notes), PracHub (solution) as "Return the Top K Most Frequent Elements", Aced as "Top k frequent elements"]*

M ☐ ★**2 Rearrange Tasks With Cooldown**: only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + how many tasks share the biggest count) *[PracHub (solution), Aced (solution) as "Design a task scheduler in Python."]*

M ☐ ★**3 Implement frequency + distance top-K queries**: count the point tuples; the tie-break is distance from the origin, then x, then y *[PracHub (solution)]*

M ☐ **Find top/bottom-k words in list or stream**: the same Counter; run nsmallest once with -count and once with +count for the bottom-k *[PracHub (solution)]*

M ☐ **Solve chat, grid paths, and car rentals**: the top-K part is the pattern template above; the other parts are a rolling-column path count and a greedy car assignment *[PracHub (solution)]*

9. Games: win or lose with a cache, and guessing with feedback ★

Recursion and backtracking (app section 39) 5 questions: 0 easy, 1 medium, 4 hard

Spot it: "who wins with optimal play", "merge stacks game", "guess the secret word", "Wordle strategy".

Idea: Describe the whole game position as a hashable state (a sorted tuple of stacks). A position is winning if some move leads to a losing position for the opponent; cache the answer per state so each state is solved once. For guessing games, choose the guess that splits the remaining candidates into the most even feedback groups (minimise the size of the largest group), then keep only candidates consistent with the feedback received.

```
@lru_cache(maxsize=None)
def wins(state):                          # state: sorted tuple of stack sizes
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            if state[i] + state[j] <= cap:
                rest = list(state);  merged = rest[i] + rest[j]
                del rest[j];  del rest[i]
                rest.append(merged)
                if not wins(tuple(sorted(rest))): return True
    return False                          # no move, or every move loses

def best_guess(words, candidates):
    def worst_group(guess):
        groups = Counter(matches(guess, w) for w in candidates)
        return max(groups.values())
    return min(words, key=worst_group)
```

Cost: Number of distinct states times moves per state; guessing is O(words * candidates). **Traps:** Normalise the state (sort) so equal positions share one cache entry. Say who moves next and what "no move" means (a loss in normal play). Terminal states need a base case.

Follow-ups: Count winning first moves. Misere play. A sum of independent games (Sprague-Grundy values). Weighted answer likelihoods. How big is the real state space?

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

H ☐ ★**1 Find secret word with match-count feedback**: guess the word that minimises the size of the biggest feedback group; keep the words with the same match count *[PracHub (solution)]*

M ☐ **Choose optimal guesses for green-only Wordle**: bitmask per (position, letter) of the words that have it; the best guess splits the words into the most even groups *[PracHub (solution), 1Point3Acres (notes) as "Wordle-Style 5-Letter Minimum-Guess Strategy"]*

H ☐ **Determine winner in stack-merging game**: state = the sorted tuple of stack sizes; a move merges two whose sum stays within the cap *[PracHub (solution), 1Point3Acres (notes) as "Babylon Tile-Stack Merge Game"]*

H ☐ **Wordle-Style 5-Letter Minimum-Guess Strategy**: ask about the letter at a position that splits the remaining words most evenly (min of yes-count and no-count is largest) *[1Point3Acres, PracHub (solution) as "Implement a Word Guessing Game"]*

H ☐ **Babylon Tile-Stack Merge Game**: state = the sorted tuple of stacks; a move merges two stacks that share a height or a colour; you win if some merge leaves the opponent without a winning reply *[1Point3Acres (notes), PracHub (solution) as "Determine winner in stack-merging game"]*

10. K-way merge: a heap of the next item from each source ★

Heaps and top-k (app section 35) 4 questions: 1 easy, 1 medium, 2 hard

Spot it: "merge k sorted lists", "k smallest pair sums", "sort a nearly sorted array", "top-k subset sums".

Idea: The smallest unseen item overall must be the head of one of the sources. Keep the current head of each source in a heap, pop the smallest, and push its successor (the next item of the same source). For pair sums of two sorted arrays, seed with (a[i] + b[0], i, 0); the successor of (i, j) is (i, j+1). For a k-sorted array, keep a heap of k + 1 items and always emit the smallest.

```
def merge_k(lists):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
    heapq.heapify(heap)
    out = []
    while heap:
        value, which, pos = heapq.heappop(heap)
        out.append(value)
        if pos + 1 < len(lists[which]):
            heapq.heappush(heap, (lists[which][pos + 1], which, pos + 1))
    return out
```

Cost: O(N log k) for N items in total from k sources. **Traps:** Include the source id so equal values never compare lists or nodes. Seed from the largest end if you want the largest items. Do not merge lists one by one into a growing result (O(N k)).

Follow-ups: Lists too large for memory (streams or files, same algorithm). Not sorted (sort first or use a selection algorithm). Three arrays. Return the subsets themselves.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

H ☐ ★**1 Merge k sorted linked lists.**: seed with each list head (value, list number, node); the successor of a node is node.next *[Aced (solution), PracHub (solution) as "Merge K Individually Sorted Integer Arrays Into One Sorted Array"]*
E ☐ **Find k pairs with smallest sums**: seed with (a[i] + b[0], i, 0); the successor of pair (i, j) is (i, j + 1) *[PracHub (solution)]*
M ☐ **Sort a nearly sorted (or K sorted) array.**: keep a heap of the first k + 1 items; pop the smallest and push the next input item (heapreplace) *[Aced (solution), PracHub (solution) as "Sort a nearly sorted array"]*
H ☐ **Return top-k subset sums in descending order**: heap of (sum, index); each popped state pushes two successors: add the next item, or swap the last item for the next *[PracHub (solution)]*

11. Rolling state: the next state comes only from the previous one

Dynamic programming and memoization (app section 19) 5 questions: 2 easy, 3 medium, 0 hard

Spot it: "climbing stairs", "house robber", "paint houses", "fibonacci", "dice with limits".

Idea: If dp[i] depends only on dp[i-1] (and dp[i-2]), keep just those few variables instead of the full table. Climbing stairs: (a, b) = (b, a + b). House robber: keep (best if you take this house, best if you skip it). Paint house with k colours: keep a row of costs and, for each colour, add the smallest previous value that is not the same colour (store the two smallest).

```
def rob(money):
    take, skip = 0, 0
    for x in money:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)

def climb(n):
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b
```

Cost: O(n) time, O(1) space (O(k) for a row of colours). **Traps:** Initialise with the true base cases. Update both variables in one tuple assignment. Circular houses: run twice, once without the first house and once without the last.

Follow-ups: n up to 10^18 (matrix exponentiation, or fast doubling). Steps 1 to k (sliding window sum). Broken steps. Return which houses (keep parent choices). Closed form (Binet) and its floating point limits.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.**: state = (best if you take this house, best if you skip it): take, skip = skip + x, max(take, skip) *[Aced (solution), PracHub (solution) as "Maximize sum without choosing adjacent elements"]*
E ☐ **Fibonacci Numbers**: the same pair update starting from (0, 1) *[Aced (solution), PracHub (solution) as "Compute Fibonacci Numbers"]*
M ☐ **Maximize sum without choosing adjacent elements**: exactly the house robber update on the array *[PracHub (solution)]*
E ☐ ★**2** ★ **Climbing Stairs**: state = the last two counts: a, b = b, a + b, repeated n - 1 times *[Aced (solution)]*
M ☐ ★**3** ★ **Write SQL code to publish the Fibonacci series.**: WITH RECURSIVE keeps (n, a, b); every new row is (n + 1, b, a + b): the same rolling pair, written

as a query *[Aced (solution)]*

12. Recurrence on a position or a range (memo or table) ★

Dynamic programming and memoization (app section 5) 8 questions: 0 easy, 6 medium, 2 hard

Spot it: "edit distance", "longest palindromic subsequence", "word break", "split into expressions", "count decompositions".

Idea: Define best(i) for a suffix or best(i, j) for a range in words, write how it follows from smaller states, and either memoise the recursion or fill a table in an order where smaller states come first. Strings: dp[i][j] for prefixes of two strings (equal letters copy the diagonal, otherwise 1 + the cheapest neighbour). Ranges: best(l, r) tries every split point m. Digit DP walks the number from the top with a tight flag and a state summary.

```
def edit_distance(a, b):
    previous = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        current = [i] + [0] * len(b)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]: current[j] = previous[j - 1]
            else: current[j] = 1 + min(previous[j], current[j - 1], previous[j - 1])
        previous = current
    return previous[-1]

@lru_cache(maxsize=None)
def ways(i):                                # decompositions of text[i:]
    if i == len(text): return 1
    return sum(ways(i + len(w)) for w in dictionary if text.startswith(w, i))
```

Cost: States times transitions, for example O(n * m) for edit distance and O(n^2 * split) for range DP. **Traps:** Write the state and recurrence in plain words before coding. Base cases. For palindromic subsequences the table is indexed by (i, j) with i <= j. Memoised recursion can hit the recursion limit on long inputs.

Follow-ups: Return the actual solution (keep choices). Count modulo a prime. Count in a range [A, B] (count up to B minus up to A-1). Faster version (Fenwick tree, sparse table). Different costs per operation.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Target Expressions from Ordered Digits**: best(l, r) = every value reachable from digits l..r; split at m and combine the two sides with +, - and * *[1Point3Acres (notes), PracHub (solution) as "Insert Operators to Reach a Target"]*

M ☐ ★**2 Count Word Decompositions into Chemical Element Symbols**: bottom-up ways[i]: add ways[i+1] if the one-letter symbol exists and ways[i+2] if the two-letter symbol exists *[1Point3Acres (notes), PracHub (solution) as "Count Spellings Using Chemical Element Symbols"]*

H ☐ ★**3 Edit distance**: table over two strings, two rows: equal letters copy the diagonal, otherwise 1 + the smallest of the three neighbours *[Aced (solution), PracHub (solution) as "Compute Edit Distance"]*

H ☐ ★**4 Count Perfect Wake Numbers up to N**: digit DP: cache best(remaining digits, used-digit mask, last digit, falling?) and walk along N with a tight flag *[1Point3Acres (notes), PracHub (solution) as "Count Good Numbers up to a Limit"]*

M ☐ ★**5 Fence Painting Minimum Operations (Gemini Team)**: range (l, r): paint one stripe at the lowest height, then solve the pieces on both sides; a sparse table finds the lowest point fast *[1Point3Acres (notes), PracHub (solution) as "Minimize Fence Painting Operations"]*

M ☐ ★**6 Find the longest palindromic subsequence using dynamic programming.**: table over (i, j): equal ends add 2 to the inside; otherwise the better of dropping either end *[Aced (solution), PracHub (solution) as "Compute longest palindromic subsequence"]*

M ☐ **Maximize 1D deliveries within distance and minimize total distance**: table over (orders used, drivers used): skip an order, skip a driver, or match them when they are within D (count, then minus distance) *[PracHub (solution)]*

M ☐ ★ **Jump Game with Profit**: best[i] = max over every j before i that can reach i (j + arr[j] >= i) of best[j] + profit[i] *[1Point3Acres (notes)]*

13. Grid DP: each cell from its neighbours above and to the left ★

Dynamic programming and memoization (app section 26) 5 questions: 0 easy, 3 medium, 2 hard

Spot it: "unique paths", "largest square", "longest run of one colour", "paths with checkpoints".

Idea: Fill the table row by row; each cell depends on cells already filled. Number of paths: ways[r][c] = ways[r-1][c] + ways[r][c-1] (zero on blocked cells or missed checkpoints). Same-colour squares: if the cell and its three neighbours match, side[r][c] = 1 + min of the three neighbouring sides. Longest run in four directions: keep four run lengths per cell from the previous row and the left cell.

```
squares = 0
side = [[0] * cols for _ in range(rows)]
for r in range(rows):
    for c in range(cols):
        if r and c and grid[r][c] == grid[r-1][c] == grid[r][c-1] == grid[r-1][c-1]:
            side[r][c] = 1 + min(side[r-1][c], side[r][c-1], side[r-1][c-1])
        else:
            side[r][c] = 1
        squares += side[r][c]               # a square of side s counts s squares
```

Cost: O(rows * cols) time, O(cols) space if you keep only the previous row. **Traps:** Edges (first row and column). Diagonal moves

need three predecessors. A checkpoint resets rows that do not pass through it.

Follow-ups: Largest square only. Allow one different cell. Grid arrives row by row (keep one row). Return where the run is.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Grid Path Count with Diagonal Moves and Checkpoints**: advance column by column keeping the number of ways in each row; new[r] = old[r-1] + old[r] + old[r+1]; zero out rows that miss a checkpoint *[1Point3Acres (notes), PracHub (solution) as "Count Up-Right/Right/Down-Right Grid Paths Through Ordered Checkpoints"]*

M ☐ ★**2 Longest Same-Color Grid Sequence**: keep four run lengths per cell (horizontal, vertical, two diagonals) from the previous row and the left cell; the answer is the largest *[1Point3Acres (notes), PracHub (solution) as "Find the Longest Straight Run of One Color"]*

H ☐ ★**3 Find the Longest Straight Run of One Color**: the same four run lengths; extend a run when the neighbour in that direction has the same colour *[PracHub (solution), 1Point3Acres (notes) as "Longest Same-Color Grid Sequence"]*

M ☐ **Count Same-Color Squares in Matrix**: the pattern template above: a square ends here if the three neighbours match; its size is 1 + min of their sizes *[1Point3Acres (notes), PracHub (solution) as "Count same-color squares in a character grid"]*

H ☐ **Count same-color squares in a character grid**: exactly the same recurrence on characters *[PracHub (solution), 1Point3Acres (notes) as "Count Same-Color Squares in Matrix"]*

14. Counting or reaching an amount: the knapsack loop

Dynamic programming and memoization (app section 32) 4 questions: 0 easy, 4 medium, 0 hard

Spot it: "coin change", "partition equal subset sum", "target sum with plus and minus", "unbounded items that fill a capacity".

Idea: dp[t] is the answer for total t. For each item, update dp across totals. Use items unlimited times: loop t upward (dp[t] = min(dp[t], dp[t - c] + 1)). Use each item once: loop t downward so an item cannot be used twice in one pass. For plus/minus signs, rewrite as a subset sum of (total + target) / 2. A bitset (Python int) makes subset sum fast: reach |= reach << x.

```
def coin_change(coins, amount):
    best = [0] + [float('inf')] * amount
    for t in range(1, amount + 1):
        for c in coins:
            if c <= t: best[t] = min(best[t], best[t - c] + 1)
    return -1 if best[amount] == float('inf') else best[amount]

def can_partition(nums):
    total = sum(nums)
    if total % 2: return False
    reach = 1                              # bit t is set if total t is reachable
    for x in nums: reach |= reach << x
    return (reach >> (total // 2)) & 1 == 1
```

Cost: O(amount * number of coins), or O(n * sum) for subset sum. **Traps:** Count combinations (loop items outside) versus permutations (loop totals outside). Direction of the inner loop decides reuse. Negative numbers break the plain table.

Follow-ups: Count the ways (LeetCode 518). Return which coins. Minimise the difference of the two parts. Does greedy work (only for special coin systems)?

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**3 Coin Change**: fewest coins: start with infinity, best[t] = min(best[t], best[t - c] + 1); -1 when it stays infinity *[Aced (solution), PracHub (solution) as "Find the Minimum Number of Coins for an Amount"]*

M ☐ **Explain how to find a target sum in an array.**: turn +/- signs into a subset sum of (total + S) / 2, items used once (sweep downwards) *[Aced (solution), PracHub (solution) as "Count expressions reaching a target sum"]*

M ☐ ★**1** ★ **Reverse Coin Change**: given the ways table, find the coin values: the first amount whose count is wrong must be a coin; add it and rebuild *[HackTheRounds #633 (solution)]*

M ☐ ★**2** ★ **Partition an array into two sub-arrays with equal sum.**: subset sum to total / 2, written with a bitset: reach |= reach << x *[Aced (solution)]*

15. Dict of per-key state (update, expire, rank)

Hash maps and sets (app section 10) 8 questions: 1 easy, 7 medium, 0 hard

Spot it: "remember the last time we saw X", "top users", "within the last 10 seconds", "design a logger / counter".

Idea: Keep one dictionary whose key is the thing you track (message, user, word) and whose value is the small state about it: a timestamp, a count, a set, a deque. Every operation is a lookup, an update, or an expiry check on that one entry, so each call costs O(1) on average. To rank, keep a second structure (buckets by count, or sort at read time) instead of re-sorting on every update.

```
last_shown = {}
def should_print(now, message, window=10):
```

```
    if message in last_shown and now - last_shown[message] < window:
        return False
    last_shown[message] = now
    return True

hits = deque()                      # sliding window of hit times
def hit(now):  hits.append(now)
def count(now, window=300):
    while hits and hits[0] <= now - window:
        hits.popleft()              # expire from the old end
    return len(hits)
```

Cost: O(1) per call; expiry is amortised O(1) because each item is removed once. **Traps:** Decide first whether a rejected call refreshes the timestamp (usually it does not). Timestamps can repeat. Use < or <= consistently at the window edge.

Follow-ups: Out of order events (keep sorted, or a heap). Bounded memory (evict expired keys lazily or in a background sweep). Thread safety (one lock, or a lock per key). Millions of hits per second (bucket by second instead of storing every hit).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Active Communication Top Users**: a set of seen pairs, a Counter of partners per user, and buckets by count to read the top users in O(1) *[1Point3Acres (notes), PracHub (solution) as "Most Active Users in a Live Communication Stream"]*

E ☐ ★2 **Robot Status Message Deduplication Window**: the pattern template above: show a message only if it was not shown in the last 10 seconds *[1Point3Acres (notes), PracHub (solution) as "Filter Repeated Robot Status Messages"]*

M ☐ ★3 **Next-Word Predictor**: defaultdict(Counter): counts[a][b] += 1 for neighbouring words; remember the best next word per word *[1Point3Acres (notes), PracHub (solution) as "Build a Next-Word Predictor"]*

M ☐ ★5 **Hit Counter**: one deque of hit times; drop the ones older than 300 seconds; the answer is the length *[HackTheRounds #39 (solution), PracHub (solution) as "Design a rolling hit counter API"]*

M ☐ ★6 **Filter Repeated Robot Status Messages**: the same rule applied to a stream of (timestamp, message) pairs; keep what is accepted *[PracHub (solution), 1Point3Acres (notes) as "Robot Status Message Deduplication Window"]*

M ☐ ★7 **Website Activity Analytics: Unique Users + Session Time**: dict activity -> set of users for unique counts; dict user -> sorted events to add up login-to-logout session time *[1Point3Acres (notes), PracHub (solution) as "User Activity Analytics: Unique Users per Activity and Average Time on Site"]*

M ☐ **Design a logger that returns top-K messages**: dict message -> list of timestamps; top_k counts each list inside the time range with bisect, then takes the k largest *[PracHub (solution)]*

M ☐ ★4 ★ **Batch and Streaming Log Deduplication**: dict message -> first (or latest) timestamp depending on the policy; sort the result at the end *[1Point3Acres (notes)]*

16. Sweep over events in time order ★

Intervals and scheduling (app section 2) 9 questions: 0 easy, 6 medium, 3 hard

Spot it: "minimum rooms", "maximum overlap", "skyline", "who is online at time t", "servers needed".

Idea: Turn every interval into two events (+1 at start, -1 at end), sort all events by time, and sweep while keeping a running count; the maximum count is the answer. Break ties explicitly (ends before starts if a touching meeting may reuse the room). In two dimensions, put +1/-1 at the four corners of a rectangle in a difference array and take running sums along both axes. A skyline sweep keeps a max-heap of active heights and records a point when the top changes.

```
events = []
for start, end in meetings:
    events.append((start, 1))
    events.append((end, -1))              # -1 sorts before +1 at equal time
events.sort()
rooms = best = 0
for time, delta in events:
    rooms += delta
    best = max(best, rooms)

diff = [[0] * (w + 1) for _ in range(h + 1)]   # rectangles on a grid
for x1, y1, x2, y2 in rects:
    diff[y1][x1] += 1;  diff[y1][x2] -= 1
    diff[y2][x1] -= 1;  diff[y2][x2] += 1
```

Cost: O(n log n) for the sort, O(n) for the sweep. **Traps:** Tie-break at equal times. Meetings that cross midnight must be split in two. Merge one job's own intervals first if it should count once. Simulations (two queues at a gate) are sweeps with a rule per second.

Follow-ups: Return which room or server each interval gets (heap of free rooms). Coordinates up to 10^9 (coordinate compression or a sparse map). Online arrivals (cannot sort ahead; use a heap of end times).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered

ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Train / Flight Schedule Reachability**: sort legs by departure; a leg is usable if you have already reached its origin before it leaves; keep the earliest arrival per place *[1Point3Acres (notes), PracHub (solution) as "Reachability Between Stations in a Timed Train Schedule"]*

M ☐ ★**2 Meeting Rooms III Variant**: split meetings that cross midnight into two pieces first, then the same sweep *[HackTheRounds #630 (solution), PracHub (solution) as "Compute minimum rooms for time intervals"]*

M ☐ ★**3 Meeting Rooms / Interval Scheduling Variants**: the pattern template above: the largest running count is the number of rooms *[1Point3Acres (notes), PracHub (solution) as "Assign Meetings to the Minimum Number of Rooms"]*

H ☐ ★**4 Count Overlapping Rectangle Updates on a Grid**: the same idea in two dimensions: a difference array (+1, -1, -1, +1 at the corners) and two running-sum passes *[PracHub (solution)]*

M ☐ ★**5 Solve Rooms and Top-K Streams**: sort starts and ends separately and walk two pointers: a start before the earliest end needs a new room; the stream part is a size-K heap *[PracHub (solution)]*

M ☐ ★**6 Gate Entry / Exit Time Resolution with Tie-Break Rules**: the same two-queue simulation with the tie-break rules from the statement *[1Point3Acres (notes), PracHub (solution) as "Compute Turnstile Crossing Times"]*

H ☐ ★**7 Compute Turnstile Crossing Times**: sort people by arrival time; two queues (entering, exiting) with a rule for who goes first at each second *[PracHub (solution)]*

M ☐ **Compute servers needed for daily recurring jobs**: merge each job's own intervals first so one job counts once, then sweep all of them together *[PracHub (solution)]*

H ☐ **Compute city skyline outline**: events at each building's left and right edge; a max-heap of active heights; record a point when the top height changes *[PracHub (solution)]*

17. Binary search on a yes/no condition ★

Binary search (app section 11) 7 questions: 0 easy, 4 medium, 3 hard

Spot it: "minimum speed / capacity / time such that ...", "largest length such that ...", "first bad version", "median of two sorted arrays".

Idea: Find a quantity x and a test ok(x) that is monotone: false up to a threshold and true after it (or the reverse). Then binary search the threshold in O(log range) calls to ok. Write ok(x) as a simple linear scan or sum. To find the largest x that works, search the first x that fails and subtract one.

```
def first_true(low, high, ok):            # smallest x in [low, high] with ok(x)
    while low < high:
        mid = (low + high) // 2
        if ok(mid): high = mid
        else:       low = mid + 1
    return low

speed = first_true(1, max(piles),         # minimum eating speed
        lambda v: sum(-(-p // v) for p in piles) <= hours)
```

Cost: O(log(range) * cost of ok). **Traps:** Choose bounds that surely contain the answer (and say why). Use `mid = (low + high) // 2` with `high = mid` to avoid an infinite loop. Median of two arrays: binary search how many items come from the shorter array.

Follow-ups: Unknown length or huge file (double the index until the condition flips). Rate limited oracle (parallel search: more calls, fewer rounds). k sorted arrays or the k-th element (generalise the partition).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

H ☐ ★**1 Median of Two Sorted Arrays (LC 4) with a Follow-up Variant**: x = how many items come from the shorter array's left side; ok(x) = the left part's largest <= the right part's smallest *[1Point3Acres (notes), PracHub (solution) as "Median of All Values Across Several Sorted Integer Arrays"]*

M ☐ ★**2 Cutting Ribbons (LC 1891)**: x = the piece length; ok(x) = the ribbons give at least k pieces; you want the LARGEST ok, so search the first x that fails and subtract 1 *[1Point3Acres (notes), PracHub (solution) as "Find maximum cable length by cutting"]*

M ☐ ★**3 First Bad Version with Parallel Search Follow-up**: x = the version number; ok(x) = is_bad(x) *[1Point3Acres (notes), PracHub (solution) as "Find the First Bad Version in a Monotonic Release History"]*

H ☐ ★**4 Find the median of two sorted arrays.**: the same partition search on the shorter array *[Aced, PracHub (solution) as "Find median of two sorted arrays"]*

M ☐ **Solve Banana Speed and Interval Merge**: x = the eating speed; ok(x) = total hours (ceil of pile / x summed) <= h; first_true over 1..max pile *[PracHub (solution)]*

H ☐ **Find longest duplicated substring**: x = the substring length; ok(x) = some substring of that length appears twice (rolling hash); search the largest ok *[PracHub (solution)]*

M ☐ ★ **Given a list with response time for each server, calculate how long time it will take to process n requests.**: x = the elapsed time; ok(x) = sum(x // time_i) >= n requests *[Aced (solution)]*

18. Sorted intervals: merge while they touch, then read gaps or search ★

Intervals and scheduling (app section 16) 6 questions: 1 easy, 3 medium, 2 hard

Spot it: "merge intervals", "do bookings conflict", "free time", "insert an interval", "union of two lists".

Idea: Sort by start. Walk once: if the next interval starts at or before the current end, extend the end with max; otherwise close the

current one and start a new one. After merging, intervals are disjoint and sorted, so the gaps are the spaces between them and a point query is a bisect over the starts. A booking conflicts if the previous interval ends after the new start or the next one starts before the new end.

```
intervals.sort()
merged = [list(intervals[0])]
for start, end in intervals[1:]:
    if start <= merged[-1][1]:              # touches or overlaps
        merged[-1][1] = max(merged[-1][1], end)
    else:
        merged.append([start, end])

starts = [s for s, _ in merged]
i = bisect_right(starts, x) - 1             # the only candidate containing x
inside = i >= 0 and merged[i][1] >= x
```

Cost: O(n log n) to sort and merge, O(log n) per query. **Traps:** Decide whether touching intervals ([1,3] and [4,5], or [1,3] and [3,4]) merge. Take max of the ends, not the new end. Closed versus half-open intervals change every comparison.

Follow-ups: Insert one interval into a merged list. Remove a range. Several blocked windows (subtract each). Double booking allowed but never triple (counter dict of boundaries). Too many intervals for memory (stream and merge in chunks).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Union of Sorted Interval Lists**: walk both sorted lists with two indices, always taking the smaller start next, then the same merge test *[1Point3Acres (notes), PracHub (solution) as "Compute the Union of Two Sorted Interval Lists"]*

M ☐ ★**2 Merge Intervals**: the pattern template above *[Aced (solution), PracHub (solution) as "Merge overlapping intervals"]*

E ☐ ★**3 Check an Interval Against Existing Intervals for Conflicts**: bisect on the starts; a new booking conflicts if the previous one ends after its start or the next one starts before its end *[PracHub (solution)]*

M ☐ ★**4 Find Containing Range**: merge once, keep the starts in a list, and bisect to find the one interval that could contain x *[PracHub (solution)]*

H ☐ **Find safe travel intervals between planet influences**: turn each planet into (centre - radius, centre + radius), merge, and read the gaps (including the infinite ends) *[PracHub (solution)]*

H ☐ **Implement a function that processes a stream of interval data.**: the Range Module add: bisect, merge the block, splice in one interval; intervals() returns the pairs *[Aced, PracHub (solution) as "Insert and merge an interval"]*

19. Greedy: scan once, carry the best so far, act only when forced ★

Greedy (app section 12) 5 questions: 1 easy, 4 medium, 0 hard

Spot it: "minimum number of ...", "maximum profit", "largest number you can form", "cover all holes", "jumps".

Idea: Make the locally best choice and never revisit it. It is correct when you can argue an exchange: any optimal answer can be changed to use your choice without getting worse. Typical shapes: carry the farthest reach and jump only when you hit the end of the current jump; scan digits from the right fixing violations; sort then take as much of the best item as fits.

```
jumps = end = farthest = 0                  # minimum jumps
for i in range(len(a) - 1):
    farthest = max(farthest, i + a[i])
    if i == end:                            # forced to jump now
        jumps += 1
        end = farthest

holes.sort();  boards = 0;  covered_to = -1   # cover holes with boards
for h in holes:
    if h > covered_to:
        boards += 1
        covered_to = h + length - 1
```

Cost: O(n) or O(n log n) when a sort is needed. **Traps:** State why greedy is valid (exchange argument) and give a counter-example where it fails (general knapsack, coins like 1, 3, 4). Ties and empty input.

Follow-ups: Many trades or a fee (turn into DP). Return the chosen indexes. Weights not all 1 (greedy may stop working: ask what changes). Huge numbers given as strings.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Bounded Knapsack Variant**: sort items by value, highest first; take min(available count, remaining capacity) of each *[1Point3Acres (notes), PracHub (solution) as "Maximum Value from Unit-Weight Items with Limited Copies Under a Capacity"]*

M ☐ ★**2 Visible People in a Line (Last Person, Then Everyone)**: scan from the end carrying the tallest person seen between; count a person when no taller one blocks the view *[1Point3Acres (notes), PracHub (solution) as "Count visible people to the right"]*

M ☐ ★3 **Find the Largest Monotone Increasing Number**: scan the digits right to left; when a digit is smaller than the one before it, lower the earlier digit and fill 9s after it *[PracHub (solution)]*

E ☐ ★4 **Maximum Profit**: pick the cheapest and the dearest city with a fixed tie-break (min and max once) *[Aced (solution), PracHub (solution) as "Maximum Profit from One Stock Purchase and a Later Sale"]*

M ☐ ★5 **Min boards to cover roof holes**: the pattern template above *[PracHub (solution)]*

20. Count pairs by looking up each partner

Hash maps and sets (app section 30) 3 questions: 0 easy, 2 medium, 1 hard

Spot it: "pairs with difference k", "two ways to write as a sum", "rectangle / square from points".

Idea: For every item, the partner you need is determined by the rule (v + k, target - v, the opposite corner). Store what you have seen in a Counter or set, then look the partner up in O(1) instead of scanning all other items. When a pair would be found twice, either count only one direction or divide the total by 2.

```
counts = Counter(values)
pairs = 0
for v in counts:
    if k == 0:
        pairs += counts[v] >= 2          # the partner is itself
    elif v + k in counts:
        pairs += 1

points = set(map(tuple, grid_points))     # squares from two opposite corners
for (x1, y1), (x2, y2) in combinations(points, 2):
    if x1 != x2 and y1 != y2 and abs(x1 - x2) == abs(y1 - y2):
        total += (x1, y2) in points and (x2, y1) in points
total //= 2                               # each square is seen from both diagonals
```

Cost: O(n) for single partner lookups, O(n^2) for all pairs of points. **Traps:** The partner may equal the item itself (k = 0). Count distinct pairs, not index pairs. Bucket pairs by a signature (sum of coordinates, squared length) to find rectangles without checking all quadruples.

Follow-ups: Stage 2 with new rules (keep the code adaptable). Duplicates allowed (count with multiplicity). Beat O(n^2) pairs (bucket by signature).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Sum of Two Squares in Two Ways**: enumerate every pair (a, b) with a <= b once, add a*a + b*b to a Counter, answer = sums counted exactly twice *[HackTheRounds #632 (solution), PracHub (solution) as "Find numbers with exactly two sum-of-squares forms"]*

H ☐ ★2 **Maximum-Area Rectangle from Planar Points**: bucket every pair of points by (sum of coordinates, squared length); two pairs in one bucket are the diagonals of a rectangle *[1Point3Acres (notes), PracHub (solution) as "Largest-Area Rectangle From a Point Set When Sides Need Not Be Axis-Aligned"]*

M ☐ ★3 ★ **Progressive Array Partitioning**: Counter of the values; stage 1 asks whether all the counts are different: len(set(counts.values())) == len(counts) *[1Point3Acres (notes)]*

21. Prefix sums, tier 1: precompute once, query in O(1)

Arrays and prefix sums (app section 15) 6 questions: 1 easy, 5 medium, 0 hard

Spot it: "sum of a range", "balanced split", "pivot index", "equal halves", "area up to a height".

Idea: Let prefix[i] be the sum of the first i items. Then any range sum is prefix[r+1] - prefix[l], and a balance point is an index where the left total equals total minus left minus the item. The same trick works for counts of any property that adds up (equal-parity neighbours, ones in a string, area strips).

```
prefix = [0]
for x in a:
    prefix.append(prefix[-1] + x)
def range_sum(l, r): return prefix[r + 1] - prefix[l]

total = sum(a);  left = 0
for i, x in enumerate(a):                 # pivot index
    if left == total - left - x:
        return i
    left += x
```

Cost: O(n) to build, O(1) per query. **Traps:** Off by one between prefix[i] and prefix[i+1]. Use prefix[0] = 0 so a range starting at 0 works. For a removed element, look up the value that would balance both sides in a dict of first/last positions.

Follow-ups: Array changes between queries (Fenwick tree or segment tree). Circular array (double it). Remove one or k elements. Count the ways instead of finding one.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front

of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

E ☐ ★1 **Parity Alternating Subarray Query**: pre counts neighbouring pairs of equal parity; a range alternates when pre[r] - pre[l] == 0 *[HackTheRounds #35 (solution), PracHub (solution) as "Answer parity-alternation range queries"]*

M ☐ ★2 **Range Duplicate Check with Constant-Time Queries**: next[i] = next occurrence of the same value; reach[l] = smallest next over l..end; the range l..r is all distinct when reach[l] > r *[1Point3Acres (notes), PracHub (solution) as "O(1) Range Queries: Are All Elements of a Subarray Distinct?"]*

M ☐ ★3 **Find a Horizontal Cut That Bisects Rectangle Area**: events by height with a running width; accumulate area strip by strip until half the total, then solve inside that strip *[PracHub (solution)]*

M ☐ ★5 **Check pivot after removing one element**: prefix sums plus first/last position maps: look up the value whose removal would balance both sides *[PracHub (solution)]*

M ☐ ★4 ★ **Gold Chain Split into Two Equal Halves**: prefix sums with first/last position maps: find two cut points that leave equal halves around one removed link *[1Point3Acres (notes)]*

M ☐ ★ **Count the number of ways to split a string.**: positions of the ones; the answer multiplies the gap sizes around the k-th and 2k-th one (zero ones: choose two cut points) *[Aced (solution)]*

22. Runs and groups: extend while it fits, restart when it breaks

Arrays and prefix sums (app section 8) 4 questions: 0 easy, 4 medium, 0 hard

Spot it: "longest contiguous run", "group consecutive items", "streak", "one replacement allowed".

Idea: Walk the array once with a running length (or running sum). If item i fits with item i-1, extend the run; otherwise close it, record the best, and restart at 1. For "one change allowed", precompute for each index the run length ending there (left[i]) and starting there (right[i]); changing the element at i can join left[i-1] and right[i+1].

```
best = run = 1
for i in range(1, len(a)):
    run = run + 1 if a[i] >= a[i - 1] else 1
    best = max(best, run)

left = [1] * n;  right = [1] * n          # one change allowed
for i in range(1, n):
    if a[i] >= a[i - 1]: left[i] = left[i - 1] + 1
for i in range(n - 2, -1, -1):
    if a[i] <= a[i + 1]: right[i] = right[i + 1] + 1
best = max(left[i - 1] + 1 + right[i + 1] for i in range(1, n - 1)
           if a[i + 1] - a[i - 1] >= 0)
```

Cost: O(n) time, O(1) extra for one pass, O(n) for the left/right tables. **Traps:** Strict versus non-strict comparison. Remember to record the last run after the loop. A run of one element is valid. The joined run needs the two outer neighbours to be compatible.

Follow-ups: Return where the longest run is (store its start). Up to k modifications (then it is a sliding window). Many (l, r) queries (prefix arrays). Group by key per user.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Longest Non-Decreasing Subarray with One Replacement**: left[i] and right[i] run lengths; the best answer joins the run before and after one changed element *[1Point3Acres (notes), PracHub (solution) as "Longest Non-Decreasing Subarray After One Replacement"]*

M ☐ **Find longest increasing contiguous subarray**: fits = a[i] > a[i-1]; also keep a left run and a right run per index so one change can join two runs *[PracHub (solution)]*

M ☐ ★2 ★ **Piano Hand Movement Grouping**: start a new group when adding the note would make max - min exceed k *[1Point3Acres (notes)]*

M ☐ ★ **Sum of Good Subarrays (Adjacent Diff ±1)**: keep the running length and running sum of the current up-run and down-run; add both to the total at every index *[1Point3Acres (notes)]*

23. bisect on a sorted list: position, range count, keep sorted ★

Binary search (app section 14) 6 questions: 0 easy, 4 medium, 2 hard

Spot it: "count in a range", "k-th largest as items stream in", "nearest value", "longest increasing subsequence".

Idea: bisect_left(a, x) gives the first index with a[i] >= x and bisect_right gives the first index with a[i] > x. Their difference counts items equal to x; two calls count a range. insort keeps a list sorted while inserting. The tails list of the longest increasing subsequence is also maintained with bisect_left: replace the first tail that is at least x, or append.

```
def count_in_range(sorted_a, low, high):   # low <= a[i] <= high
    return bisect_right(sorted_a, high) - bisect_left(sorted_a, low)

tails = []                                 # longest increasing subsequence length
for x in values:
    i = bisect_left(tails, x)
    if i == len(tails): tails.append(x)
```

```
    else:               tails[i] = x
length = len(tails)

envelopes.sort(key=lambda e: (e[0], -e[1]))  # width up, height down
```

Cost: O(log n) per search, O(n) per insort (list shift), O(n log n) for LIS. **Traps:** Strict versus non-strict increase decides bisect_left versus bisect_right. For Russian dolls, sort heights descending on equal widths. bisect on prefixes: `prefix + chr(0x10FFFF)` is a safe upper bound.

Follow-ups: Deletions too (use a Fenwick tree or a sorted container). Return the actual chain (store parent links). Count smaller numbers after self (Fenwick tree or merge sort).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Sliding-Window Mean After Removing the Largest K Values**: keep the window in a sorted list; insort the new value, remove the old one with bisect_left, and track the sum of the top-K region *[PracHub (solution)]*

H ☐ ★**3 Find Maximum Envelope Nesting**: exactly the same sort and tails list *[PracHub (solution), 1Point3Acres (notes) as "Russian Doll Envelopes"]*

M ☐ **Streaming Insert + Find K-th Largest**: insort every new value; the k-th largest is sorted_list[-1 - k] *[1Point3Acres (notes), PracHub (solution) as "Design structure for insert and k-th largest"]*

M ☐ **Russian Doll Envelopes**: sort by width ascending and height descending, then the longest increasing run of heights using a tails list and bisect_left *[1Point3Acres (notes), PracHub (solution) as "Find Maximum Envelope Nesting"]*

H ☐ ★**2** ★ **Count of Smaller Numbers After Self**: go from the right, insort each number into a sorted list; the count is bisect_left of the number before inserting (a Fenwick tree is the fast version) *[HackTheRounds #36 (solution)]*

M ☐ ★ **Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.**: for each kid bisect into the sorted sellers and take the nearer of the two neighbours; the answer is the largest of those distances *[Aced]*

24. Sets and counters across two collections

Hash maps and sets (app section 13) 4 questions: 1 easy, 2 medium, 1 hard

Spot it: "appears in both", "not in the other list", "duplicates", "pair each x with 2x".

Idea: Put the first collection into a set (membership) or a Counter (multiplicity), then stream the second collection past it and test each item in O(1). A set answers "is it there"; a Counter answers "how many are left", and you decrement it as you use items up. When the smaller side fits in memory, always load the smaller side.

```
seen = set()
for value in first_list:
    if value in seen:
        return True                 # duplicate found
    seen.add(value)

remaining = Counter(values)         # pair each x with 2x
for x in sorted(remaining, key=abs):
    if remaining[x] > remaining[2 * x]:
        return []
    remaining[2 * x] -= remaining[x]
```

Cost: O(n + m) time, O(min(n, m)) extra space. **Traps:** Zero is its own double. Negative numbers need sorting by absolute value. Do not mutate the Counter while iterating its keys unless you iterate a sorted copy.

Follow-ups: Second input is a huge stream (load only the smaller set). No extra memory allowed (sort both and use two pointers, or use the array itself as the set by swapping). Three or more collections (intersect step by step).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

H ☐ ★**1 Smallest Missing Positive Integer in Linear Time and Constant Extra Space**: use the array itself as the set: swap each value x into slot x - 1, then the first slot with the wrong value is the answer *[PracHub (solution)]*

M ☐ ★**3 Min deletions to avoid overlap in first k**: Counter of the left part and of the right part; for each value in both, delete min(left count, right count) *[PracHub (solution)]*

M ☐ **Reconstruct original array from doubled shuffle**: Counter of the values; go from the smallest, pairing each x with 2x and using both counts up *[PracHub (solution)]*

E ☐ ★**2** ★ **Find the Duplicates**: values that appear in both lists, each reported once (set intersection, or two pointers on sorted lists) *[Aced (solution)]*

25. Sliding window: grow on the right, shrink on the left

Two pointers and sliding window (app section 40) 4 questions: 0 easy, 3 medium, 1 hard

Spot it: "longest/shortest subarray or substring such that ...", "at most k distinct", "minimum window", "budget of changes".

Idea: Keep a window [left, right] and a summary of what is inside (a count dict, a sum, a deque of maxima). Move right forward one item at a time and add it. While the window is too bad, remove items from the left. For a longest window, record the size after shrinking; for a shortest window, record while the window is still valid, then shrink.

```
count = Counter();  left = 0;  best = 0
for right, ch in enumerate(s):             # longest with at most k distinct
    count[ch] += 1
    while len(count) > k:
        count[s[left]] -= 1
        if count[s[left]] == 0: del count[s[left]]
        left += 1
    best = max(best, right - left + 1)

maxima = deque()                           # window maximum
for i, x in enumerate(a):
    while maxima and a[maxima[-1]] <= x: maxima.pop()
    maxima.append(i)
    if maxima[0] <= i - size: maxima.popleft()
```

Cost: O(n): each index enters and leaves the window once. **Traps:** Delete keys whose count reaches 0 or `len(count)` lies. Works only when removing items from the left can fix the window (non-negative values, monotone condition). "Exactly k" = at most k minus at most k-1.

Follow-ups: Window by elapsed time instead of count (pop while the oldest is too old). Minimum or median in the window. Count windows instead of finding the best. Many queries with different parameters.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Shortest Subarray with K Distinct Integers**: state = a count dict; while it holds at least k distinct values, record the window and shrink from the left *[HackTheRounds #34 (solution), PracHub (solution) as "Find shortest subarray with at least k distinct"]*

H ☐ ★**3 Necklace Cut into Two Halves with Equal D/R Counts**: a fixed-size window of half the length slides around the doubled string, keeping a count of D *[1Point3Acres (notes), PracHub (solution) as "Find fair split of a two-color necklace"]*

M ☐ ★**4 Find the minimum window substring.**: state = how many of each needed letter are still missing; while nothing is missing, record the window and shrink *[Aced, PracHub (solution) as "Minimum Window Substring with Repeated Required Characters"]*

M ☐ ★**2 ★ Temperature Monitoring Data Structure**: a deque of the last n readings with a running total; a second deque of decreasing values gives the maximum *[1Point3Acres (notes)]*

26. Two pointers moving toward each other (or along two lists)

Two pointers and sliding window (app section 20) 6 questions: 4 easy, 2 medium, 0 hard

Spot it: "palindrome", "reverse", "sorted array and a target pair", "merge two sorted lists", "triplets that sum to zero".

Idea: Put one pointer at each end (or at the start of each list) and move whichever pointer the rule says. On a sorted array, if the pair sum is too small you must move the left pointer right, and if too large the right pointer left; this discards a whole row of pairs per step. For triplets, fix the first number and run the pair routine on the rest.

```
i, j = 0, len(a) - 1
while i < j:                               # pair with a given sum
    s = a[i] + a[j]
    if s == target: return i, j
    if s < target: i += 1
    else:          j -= 1

previous, current = None, head             # reverse a linked list
while current:
    current.next, previous, current = previous, current, current.next
```

Cost: O(n) after sorting, O(n log n) with the sort, O(1) space. **Traps:** Sort first when the problem is about values, not positions. Skip duplicates for unique triplets. Update all three pointers in one assignment when reversing links.

Follow-ups: Do it recursively. Reverse only a range or in groups of k. Allow one deletion (palindrome). Runs stored as (value, count): advance by min of the two remaining counts.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

E ☐ ★**1 Reverse a linked list.**: three pointers prev, current, next: point current.next back at prev, then move all three one step *[Aced (solution), PracHub (solution) as "Reverse a linked list"]*

E ☐ ★**2 Sort a binary array in linear time.**: left skips 0s, right skips 1s; when left sits on a 1 and right on a 0, swap them *[Aced, PracHub (solution) as "Sort Three Categories In Place"]*

E ☐ **Reverse a Sentence**: swap from both ends to reverse the whole array, then reverse every word the same way *[Aced (solution), PracHub (solution) as "Reverse Words While Preserving Spaces"]*

M ☐ **Find median of merged RLE arrays**: walk both sorted run lists, always taking the smaller value, adding its count until you pass the middle position *[PracHub (solution)]*

M ☐ **Run-Length Vector Storage + Dot Product**: one pointer per run list; multiply the values and take min(remaining counts) of the two current runs *[1Point3Acres (notes), PracHub (solution) as "Implement sparse vector dot product", Aced as "Find the dot product of two sparse vectors."]*

E ☐ ★3 ★ **Is this a valid palindrome?**: compare the two ends; skip characters that are not letters or digits; stop when the pointers cross *[Aced (solution)]*

27. Group by a canonical key

Hash maps and sets (app section 34) 5 questions: 1 easy, 3 medium, 1 hard

Spot it: "anagrams", "same shape", "shifted words", "group strings that are equivalent".

Idea: Compute a key that is identical for every member of one group and different across groups, then put items in a dict of lists under that key. The art is choosing the key: sorted letters or a 26-slot count tuple for anagrams, the differences between neighbouring letters (mod 26) for shifted words, the keyboard row for typeable words.

```
groups = defaultdict(list)
for word in words:
    key = tuple(sorted(word))              # or a 26-count tuple
    groups[key].append(word)
return list(groups.values())

def shift_key(word):                       # shifted-word key
    return tuple((ord(b) - ord(a)) % 26 for a, b in zip(word, word[1:]))
```

Cost: O(total letters * log L) with sorted keys, O(total letters) with count tuples. **Traps:** Lists are not hashable: convert the key to a tuple or string. The count-tuple key avoids the log factor. Decide how to treat case, spaces and Unicode.

Follow-ups: Different keyboard or alphabet. Subsets of letters instead of full anagrams. Words spread over many machines (hash the key to pick the machine).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★2 **Group anagrams**: key = the sorted letters (or a tuple of 26 letter counts) *[Aced (solution), PracHub (solution) as "Group Anagrams"]*

M ☐ ★4 **Check if all substrings are anagrams of words**: key = the tuple of letter counts; keep the keys of the allowed words in a set and test every substring of length 3 or more against it *[PracHub (solution)]*

M ☐ ★5 **Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.**: key = the differences between neighbouring letters, modulo 26 (a shifted word has the same differences) *[Aced, PracHub (solution) as "Group Strings Equivalent under Uniform Alphabet Shifts"]*

E ☐ ★1 ★ **Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.**: key = the keyboard row of each letter; a word qualifies when its letters map to one row *[Aced]*

H ☐ ★3 ★ **Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?**: same letters used, and the sorted list of letter frequencies must match: compare set(s) == set(t) and the sorted Counter values *[Aced]*

28. Prefix sums, tier 2: running total plus a dict of earlier totals ★

Arrays and prefix sums (app section 33) 3 questions: 0 easy, 2 medium, 1 hard

Spot it: "number of subarrays with sum k", "multiple of k", "maximum subarray", "equal endpoints".

Idea: A subarray (j, i] has sum prefix[i] - prefix[j]. So for each i you do not scan all j: you ask the dict "have I seen prefix[i] - target before, and how many times?". For a maximum subarray the dict degenerates to one number, the smallest earlier prefix (this is Kadane). For multiples of k, store the first index of each remainder prefix % k.

```
seen = {0: 1};  total = ways = 0
for x in a:                                # count subarrays with sum k
    total += x
    ways += seen.get(total - k, 0)
    seen[total] = seen.get(total, 0) + 1

best, total, smallest = -inf, 0, 0         # maximum subarray sum
for x in a:
    total += x
    best = max(best, total - smallest)
    smallest = min(smallest, total)
```

Cost: O(n) time, O(n) space for the dict. **Traps:** Seed the dict with prefix 0 (the empty prefix). Update the dict after using it, otherwise an item pairs with itself. With a minimum length, update the "smallest earlier total" one step late.

Follow-ups: All positive numbers (then a sliding window works). Circular array. Return the subarray itself (store indices). Length limits (min or max length).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Find the maximum subarray sum.**: the dict is just one number: the smallest earlier total; the best subarray ends here with total - smallest (Kadane is the same idea) *[Aced (solution), PracHub (solution) as "Find the Maximum-Sum Contiguous Subarray"]*

M ☐ ★2 **Solve array-sum and city-community problems**: count subarrays summing to k with the pattern template above; the second task is union-find over building cells *[PracHub (solution)]*

H ☐ ★ **Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.**: key = total % k; store the first index; a hit at least 2 positions back means a subarray whose sum is a multiple of k *[Aced (solution)]*

29. Nested structures: a stack of contexts, or recursion with a cache ★

Stacks and queues (app section 27) 5 questions: 0 easy, 5 medium, 0 hard

Spot it: "decode string 3[a2[c]]", "evaluate expression", "nested list weight", "expand placeholders".

Idea: Each open bracket starts a new context, each close bracket finishes it and combines the result into the one below. Keep a stack of (partial result, pending multiplier) and push on open, pop on close. For recursive definitions (templates referring to keys), resolve with a cache and a "currently resolving" set so a cycle is detected instead of looping forever.

```
stack, current, number = [], '', 0
for ch in s:
    if ch.isdigit():      number = number * 10 + int(ch)
    elif ch == '[':       stack.append((current, number));  current, number = '', 0
    elif ch == ']':
        previous, times = stack.pop();  current = previous + current * times
    else:                 current += ch

def resolve(key, cache, visiting):
    if key in cache: return cache[key]
    if key in visiting: raise ValueError('cycle')
    visiting.add(key)
    cache[key] = expand(values[key], cache, visiting);  visiting.discard(key)
    return cache[key]
```

Cost: O(output size) for decoding, O(n) for parsing. **Traps:** Multi-digit numbers. The decoded string may be enormous (compute its length or the k-th character without building it). Unknown keys and unmatched brackets need a defined behaviour.

Follow-ups: Recursion depth (iterate with an explicit stack). Division and precedence in expressions. Variables and variadic functions. Inverse weighting (deeper counts less).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Template Variable Expansion with Follow-Ups**: resolve(name) with a cache and a 'being resolved' set; meeting a name that is being resolved is a cycle *[1Point3Acres (notes), PracHub (solution) as "Expand Recursive Key-Value Placeholders in a Template"]*

M ☐ ★2 **Nested List Weight Sum (LC 339) with Follow-Ups**: depth counter: '[' adds 1, ']' subtracts 1; each number adds value * depth (read digits and a minus sign as you go) *[1Point3Acres (notes), PracHub (solution) as "Compute a Depth-Weighted Sum of a Nested List"]*

M ☐ ★3 **Expand Recursive Key-Value Placeholders in a Template**: regex-find {key}, expand each key recursively with a cache and a visiting set to stop cycles *[PracHub (solution), 1Point3Acres (notes) as "Template Variable Expansion with Follow-Ups"]*

M ☐ ★4 **Decode String**: the pattern template above *[1Point3Acres (notes), PracHub (solution) as "Decode an encoded string"]*

M ☐ ★5 **Nested Function Expression Evaluator**: tokenise, then a recursive parse(i): a number returns itself; a name reads its arguments until ')' and applies the function *[1Point3Acres (notes), PracHub (solution) as "Evaluate a Lisp-Style Expression Language with let, add and mult Scoping"]*

30. Tree recursion, tier 1: visit a node, ask children, report upward ★

Binary trees and BSTs (app section 3) 8 questions: 3 easy, 4 medium, 1 hard

Spot it: "height", "diameter", "right side view", "level order", "delete leaves", "evaluate expression tree".

Idea: Write a function that takes a node and returns one number or object about the subtree (height, value, whether it is valid). Call it on the children first (post-order), combine, and update a global answer if the answer is not the return value. Diameter: return 1 + max(left, right) but update best with left + right. For per-level problems pass depth down and store the first or last node seen at each depth.

```
best = 0
def height(node):
    global best
    if node is None: return 0
    left, right = height(node.left), height(node.right)
    best = max(best, left + right)          # diameter in edges
    return 1 + max(left, right)
```

```
def right_view(node, depth, view):
    if node is None: return
    if depth == len(view): view.append(node.val)
    right_view(node.right, depth + 1, view)    # right first
    right_view(node.left, depth + 1, view)
```

Cost: O(n) time, O(height) stack space. **Traps:** Say whether lengths count nodes or edges. A chain of 100,000 nodes overflows Python recursion: convert to an explicit stack. Return and update are two different numbers.

Follow-ups: N-ary tree (loop over children). Is the tree balanced (return height or -1). Return the path itself (store the best endpoints). Delete subtrees that match a predicate.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Expression Tree Single-Leaf Mutation**: post-order evaluation (children first, then apply the operator); to test one leaf flip, re-evaluate with that leaf changed *[1Point3Acres (notes), PracHub (solution) as "Boolean Expression Tree with Leaf Flips"]*

H ☐ ★2 **Repeatedly Delete Leaves Matching a Target Value From a Binary Tree**: set node.left = dfs(node.left) and node.right likewise; return None when the node became a leaf with the target value *[PracHub (solution)]*

M ☐ ★3 **Given the root of a binary tree, return the length of the diameter of the tree.**: best = max(best, left + right); return 1 + max(left, right) (the diameter) *[Aced (solution), PracHub (solution) as "Compute binary tree diameter"]*

M ☐ ★4 **Longest Consecutive Path in Binary Tree**: pass the parent's value and the run length down; extend the run when value == parent + 1, else restart at 1 *[1Point3Acres (notes), PracHub (solution) as "Find the Longest Consecutive Path in a Binary Tree"]*

E ☐ **Find right-side view of binary tree**: pass depth down; the first node you reach at a new depth (visiting right before left) is the one you can see *[PracHub (solution)]*

E ☐ **Given two trees, determine if the pre-order traversals of two trees are same or not.**: walk both trees in preorder with an explicit stack (push right, then left) and compare value by value; stop at the first difference *[Aced, PracHub (solution) as "Compare Binary Trees at Matching Structural Positions"]*

M ☐ **Implement zigzag level-order traversal**: collect values by depth; reverse every odd level (or fill each level from the other end) *[PracHub (solution)]*

E ☐ ★ **Calculate the height of a binary tree.**: no update needed; return 1 + max(left, right) and read the value at the root *[Aced (solution)]*

31. Tree DP: build a parent order once, add up from the leaves

Binary trees and BSTs (app section 31) 5 questions: 0 easy, 3 medium, 2 hard

Spot it: "total size of a folder", "sum of distances from every node", "time to inform everyone", "cheapest cut".

Idea: Get an order where parents come before children (BFS or DFS from the root). Sweep it backwards to compute subtree values (size, sum, best) from children, then sweep it forwards to push information down. Distance sums for every root (rerooting): down[v] is the sum inside the subtree of v, and ans[child] = ans[parent] - size[child] + (n - size[child]).

```
order, parent = [root], {root: None}
for node in order:                          # BFS order, parents first
    for child in children[node]:
        parent[child] = node;  order.append(child)

size = {v: 1 for v in order};  down = {v: 0 for v in order}
for v in reversed(order):                   # leaves to root
    if parent[v] is not None:
        size[parent[v]] += size[v]
        down[parent[v]] += down[v] + size[v]

answer = {root: down[root]}
for v in order[1:]:                         # root to leaves (rerooting)
    answer[v] = answer[parent[v]] - size[v] + (n - size[v])
```

Cost: O(n), and no recursion so a long chain is safe. **Traps:** Prove the reroot formula with a two node tree. Hard links count a shared item once (seen set). Serve the child with the longest time first when informing in rounds.

Follow-ups: Weighted edges. Maximum distance instead of sum. Which edges to cut. The tree is queried often while files change (store subtree sizes and update upward).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★2 **Compute File System Total Size**: add each file's size while walking down from the root with a stack; a shared directory is counted once (seen set) *[1Point3Acres (notes), PracHub (solution) as "Compute Entity Size and Validate a Hierarchical File System"]*

M ☐ **Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.**: t[v] = max over children (rank among children sorted by time, longest first, + t[child]); read t at the root *[Aced, PracHub (solution) as "Validate a Hierarchy and Compute Propagation Time"]*

H ☐ **Compute distance-sum from every tree node**: down[v] = sum of distances to v's subtree; then a forward sweep: ans[child] = ans[parent] - size[child] + (n -

size[child]) *[PracHub (solution), 1Point3Acres (notes) as "Tree Distance Sum (Re-root DP)"]*

H ☐ **Tree Distance Sum (Re-root DP)**: exactly the same two sweeps (this is the re-rooting version of the question) *[1Point3Acres (notes), PracHub (solution) as "Compute distance-sum from every tree node"]*

M ☐ ★1 ★ **Minimum-Cost Tree Disconnection**: backwards sweep with two numbers per node: the best cost when the node is cut off, and the cost if it is free *[1Point3Acres (notes)]*

32. Trie: nested dicts, one character per level ★

Tries (app section 17) 8 questions: 0 easy, 7 medium, 1 hard

Spot it: "autocomplete", "words with a prefix", "longest match tokenizer", "URL router", "dictionary substrings".

Idea: Each node is a dict from character to child; a marker (END) says a word stops here. Insert walks and creates; search walks and checks END; prefix queries walk to the prefix node and collect everything below. For longest-match replacement, walk as far as the trie goes from each text position, remember the last END seen, and jump past it. A router uses one level per path segment with a literal child and one wildcard child.

```
END = '$'
def insert(root, word):
    node = root
    for ch in word:
        node = node.setdefault(ch, {})
    node[END] = True

def longest_match(root, text, start):
    node, last = root, -1
    for i in range(start, len(text)):
        if text[i] not in node: break
        node = node[text[i]]
        if END in node: last = i
    return last                              # -1 when nothing matches
```

Cost: O(length of word) per operation, O(total characters) space. **Traps:** END must be distinguishable from a real character. Deleting needs pruning of empty nodes. For prefix lookups on a static list, sorting plus bisect is lighter than a trie.

Follow-ups: Top 3 completions (store them per node). Wildcard "." search (recurse over all children). Fuzzy match with one error. Huge dictionaries (sharding, compressed trie).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **URL Request Handler Router**: one trie level per URL segment; a node has literal children and one wildcard child; try the literal first *[1Point3Acres (notes), PracHub (solution) as "Implement an HTTP Router That Resolves Handlers for Paths with Named Parameters"]*

M ☐ ★2 **Build Prefix Lookup with a Trie**: build_trie, then walk to the prefix node and collect every word below it *[PracHub (solution), 1Point3Acres (notes) as "Trie Prefix Search / Autocomplete"]*

M ☐ ★3 **Trie Prefix Search / Autocomplete**: walk to the prefix node; return the words under it (autocomplete) *[1Point3Acres (notes), PracHub (solution) as "Build Prefix Lookup with a Trie"]*

M ☐ ★4 **Greedy Longest-Match Dictionary Tokenizer**: the same longest-match walk, with a priority rule to choose between equal-length matches *[1Point3Acres (notes), PracHub (solution) as "Implement Longest-Match Text Replacement"]*

M ☐ ★5 **Check if all substrings are dictionary words**: for every start walk the trie; a substring of length 3 or more that leaves the trie or lacks END fails the check *[PracHub (solution)]*

M ☐ ★6 **Return dictionary words matching a prefix**: no trie needed: sort the words and bisect to the first match, then read forward while startswith(prefix) *[PracHub (solution)]*

M ☐ ★7 **Implement Trie Insert and Exact-Word Search**: insert is build_trie; search is walk and then END in node *[PracHub (solution)]*

H ☐ **Implement Longest-Match Text Replacement**: at each text position walk the trie as far as it goes and remember the last END; replace the longest match and jump past it *[PracHub (solution), 1Point3Acres (notes) as "Greedy Longest-Match Dictionary Tokenizer"]*

33. A dict for lookup plus an ordered container for who is next ★

Data structure and object design (app section 18) 5 questions: 1 easy, 3 medium, 1 hard

Spot it: "LRU cache", "waitlist", "round robin scheduler", "remove the cheapest", "undo / redo".

Idea: A dict gives O(1) access by id; a second structure gives order: a deque for first-in-first-out, a doubly linked list or OrderedDict for recency, a heap for cheapest. Deleting from the ordered side is expensive, so use lazy deletion: only remove the id from the dict, and when you pop from the order structure, skip entries that are no longer alive. Undo and redo are two stacks; a batch is one stack entry.

```
class LRU:
    def __init__(self, capacity):
        self.capacity, self.data = capacity, OrderedDict()
    def get(self, key):
        if key not in self.data: return -1
        self.data.move_to_end(key)          # most recent at the end
        return self.data[key]
```

```
    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)  # evict least recent

waiting = deque();  alive = {}             # lazy deletion waitlist
def next_party():
    while waiting and waiting[0] not in alive: waiting.popleft()
    return waiting.popleft() if waiting else None
```

Cost: O(1) per operation for LRU; O(log n) with a heap. **Traps:** Every access must refresh recency. Lazy deletion needs a unique ticket per insert (a name can rejoin). Say what the order is (arrival, price, deadline).

Follow-ups: Thread safety (a lock). TTL expiry. LFU instead of LRU (frequency buckets). Best fit instead of first fit (segment tree over sizes). Limit the undo history.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

H ☐ ★**1 Implement LRU Cache.**: dict key -> node plus a doubly linked list; every get/put moves the node to the front; evict from the back (or use OrderedDict.move_to_end) *[Aced (solution), PracHub (solution) as "Implement an LRU cache"]*

M ☐ ★**2 Implement Python's itertools.tee**: one deque per consumer; whoever runs out pulls the next value from the source and appends it to every queue *[1Point3Acres (notes), PracHub (solution) as "Implement a Lazy Tee Iterator in Python"]*

M ☐ ★**3 Design a restaurant waitlist system**: a segment tree of party sizes gives the first entry that fits in O(log n); the dict maps ids to slots *[PracHub (solution), 1Point3Acres (notes) as "Restaurant Waitlist: join / delete / find_first_match"]*

E ☐ **Implement Batched Undo/Redo Layer**: two stacks (undo, redo) of batches; a batch records (old value, new value) per property; undo applies the old values and moves the batch *[PracHub (solution)]*

M ☐ **Restaurant Waitlist: join / delete / find_first_match**: a deque per party size with tickets; delete only removes the name from the dict; find_first_match skips dead tickets *[1Point3Acres (notes), PracHub (solution) as "Design a restaurant waitlist system"]*

34. Build or transform a grid with index arithmetic ★

Grids and simulation (app section 42) 3 questions: 0 easy, 3 medium, 0 hard

Spot it: "rotate a matrix", "range sum in a 2D matrix", "draw a diamond", "fill in snake order", "block minimum".

Idea: Derive the formula for where each cell goes and write it directly. Rotating clockwise: new[r][c] = old[rows - 1 - c][r], or in Python zip(*matrix[::-1]). A 2D prefix table p[r+1][c+1] = cell + p[r][c+1] + p[r+1][c] - p[r][c] answers any rectangle with four lookups by inclusion and exclusion. Block grouping uses (r // k, c // k) as the key.

```
prefix = [[0] * (cols + 1) for _ in range(rows + 1)]
for r in range(rows):
    for c in range(cols):
        prefix[r + 1][c + 1] = (matrix[r][c] + prefix[r][c + 1]
                                + prefix[r + 1][c] - prefix[r][c])
def rect_sum(r1, c1, r2, c2):              # inclusive corners
    return (prefix[r2 + 1][c2 + 1] - prefix[r1][c2 + 1]
            - prefix[r2 + 1][c1] + prefix[r1][c1])

rotated = [list(row) for row in zip(*matrix[::-1])]
```

Cost: O(rows * cols) to build, O(1) per query. **Traps:** Do not rotate non-square matrices in place. Check the formula on a 2 by 3 example by hand. Blocks that do not tile the grid exactly need a decision (pad or drop).

Follow-ups: Rotate by k quarter turns (k % 4). Cell updates between queries (2D Fenwick tree). Detect impossible recipes by cycle (topological sort).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Connected Crop Arrangement on a Grid**: fill the cells in snake order (left to right, then right to left) so equal crops stay in one connected strip *[1Point3Acres (notes), PracHub (solution) as "Construct connected crop layout and safe paths"]*

M ☐ **Solve matrix groups and recipe inventory**: block (r // k, c // k) holds the minimum of every cell that maps to it; the recipe part is a topological sort *[PracHub (solution)]*

M ☐ **Rotate Any m×n 2D Matrix**: clockwise: new[r][c] = old[rows - 1 - c][r] on a cols-by-rows result (or zip(*matrix[::-1])) *[1Point3Acres (notes), PracHub (solution) as "Rotate a Matrix In Place", Aced as "Rotate a 2D matrix."]*

35. Look at the input once and keep only a small summary ★

Streaming, scale and concurrency (app section 25) 6 questions: 0 easy, 4 medium, 2 hard

Spot it: "file does not fit in memory", "stream", "minimise the number of oracle calls", "two logs".

Idea: Decide what the smallest summary is that still answers the question: a running maximum, a set of ids from the smaller file, a size-k heap, a count per key. Stream the large input past it. When a test oracle is expensive, use group testing: test a whole group in one call and split only the groups that fail, or binary search a prefix to find the first failure.

```
best = None                                  # maximum in a huge file
with open(path) as handle:
    for line in handle:
        value = int(line)
        if best is None or value > best: best = value

small_ids = {line.strip() for line in open(small_file)}
shared = [line for line in open(big_file) if line.strip() in small_ids]

def find_bad(group):                         # group testing
    if not run_test(group): return []
    if len(group) == 1: return group
    mid = len(group) // 2
    return find_bad(group[:mid]) + find_bad(group[mid:])
```

Cost: O(n) time, memory proportional to the summary, not the input. **Traps:** Never call `.read()` or `.readlines()` on a huge file. Say how you would handle both files being too big (partition by hash of the key, process partition by partition). Edge cases: empty file, blank lines, ties.

Follow-ups: Scale to many machines (map and reduce by key). Failures and retries. Top K instead of max (heap of size K). Is it I/O bound or CPU bound?

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Streaming Points: Emit K Within Distance**: keep the points in a sorted list; each new point is inserted with bisect and only windows around it are re-checked *[1Point3Acres (notes), PracHub (solution) as "Streaming Points: Remove Any Pair Within a Distance"]*

M ☐ **Two-Day Log Intersection under Memory Limit**: dict object -> set of event codes for each day; keep the objects that appear on both days with at least two distinct codes *[1Point3Acres (notes), PracHub (solution) as "Find shared objects across two log files"]*

M ☐ **Find shared objects across two log files**: load only the smaller file's object ids into a set, then stream the big file and keep ids found in the set *[PracHub (solution), 1Point3Acres (notes) as "Two-Day Log Intersection under Memory Limit"]*

M ☐ **Minimize calls to find all bad test pairs**: the same group testing: put a whole group in one call and split only the groups that fail *[PracHub (solution), 1Point3Acres (notes) as "Find All Bad Pairs Using runTest Oracle"]*

H ☐ **Find All Bad Pairs Using runTest Oracle**: for each test, binary search the prefix of earlier tests where runTest first fails; that test is a bad partner; repeat until none is left *[1Point3Acres, PracHub (solution) as "Minimize calls to find all bad test pairs"]*

H ☐ **Find maximum value in huge text file**: the pattern template above; the file is never loaded as a whole *[PracHub (solution)]*

36. Simulate a machine: a class with state and one branch per event ★

Grids and simulation (app section 22) 5 questions: 0 easy, 4 medium, 1 hard

Spot it: "design tic-tac-toe", "elevator system", "iterator over lists", "game of life", "scorekeeper".

Idea: Name the state (board, floor, queues, counters), write one method per kind of input, and make every method update the state in one place. For tic-tac-toe, keep counters per player for rows, columns and both diagonals so a win check is O(1). For game of life in place, encode old and new state in two bits of each cell so every cell is computed from the old state, then shift.

```
class TicTacToe:
    def __init__(self, n):
        self.n = n
        self.rows = defaultdict(int);  self.cols = defaultdict(int)
        self.diag = defaultdict(int);  self.anti = defaultdict(int)
    def move(self, r, c, player):
        for table, key in ((self.rows, (player, r)), (self.cols, (player, c))):
            table[key] += 1
            if table[key] == self.n: return player
        if r == c:                 self.diag[player] += 1
        if r + c == self.n - 1:  self.anti[player] += 1
        if self.n in (self.diag[player], self.anti[player]): return player
        return 0
```

Cost: O(1) per move for tic-tac-toe, O(cells) per step for life. **Traps:** Reject moves on a taken cell if the spec says so. Neighbour counts in life must read the old bit (value & 1). Elevator: define the cost of assigning a request to a car.

Follow-ups: Undo the last move (stack of moves). Infinite or sparse board (set of live cells). Wrap-around edges. Fairness, starvation and failures for schedulers.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how

recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Tic-Tac-Toe System with K Players and 3-in-a-Row Win**: store the board; after each move count the run through the new cell in four directions and compare with the target length *[1Point3Acres (notes), PracHub (solution) as "Implement Multi-Player Tic-Tac-Toe"]*

H ☐ ★2 **Simulate In-Place Cellular State Updates**: encode old and new state in two bits of each cell so every cell is computed from the old state; shift at the end *[PracHub (solution)]*

M ☐ ★3 **Design Tic-Tac-Toe With K Players**: per player Counters for rows, columns and both diagonals; a move wins when a counter reaches n *[PracHub (solution), 1Point3Acres (notes) as "Tic-Tac-Toe System with K Players and 3-in-a-Row Win"]*

M ☐ **Implement iterator merging lists with filtering**: keep a position per list and a turn counter; advance() looks for the next unblocked, unseen value and stores it for next()/has_next() *[PracHub (solution)]*

M ☐ **Design an elevator control system**: each car keeps floor, direction and stops; a request goes to the car with the lowest cost; step() moves every car one floor *[PracHub (solution)]*

37. Digits and characters: peel one at a time and carry ★

Math and bit tricks (app section 24) 4 questions: 0 easy, 3 medium, 1 hard

Spot it: "add two big numbers", "palindrome number", "monotone digits", "caesar cipher", "k-th permutation".

Idea: Work with one digit at a time using n % 10 (last digit) and n // 10 (drop it), or with ord/chr arithmetic for letters. Addition keeps a carry: total = a + b + carry, digit = total % 10, carry = total // 10. For the k-th permutation, divide k-1 by (n-1)!, (n-2)!, ... to choose each digit from the list of unused digits (factorial number system).

```
def add_digits(a, b):                       # lists, most significant first
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry + (a[i] if i >= 0 else 0) + (b[j] if j >= 0 else 0)
        out.append(total % 10);  carry = total // 10
        i -= 1;  j -= 1
    return out[::-1]

def shift(ch, k):                           # caesar
    return chr((ord(ch) - 97 + k) % 26 + 97)
```

Cost: O(number of digits). **Traps:** A final carry creates a new leading digit. Negative k works with % in Python. A number palindrome only needs the second half reversed.

Follow-ups: Numbers as linked lists (least significant first makes addition easy). Another base. Break a cipher given only ciphertext (frequency analysis). Position of a permutation (inverse of the factorial method).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Check Whether Digits Are Monotone Increasing**: peel digits from the right; each digit must not exceed the previous one you saw *[PracHub (solution)]*

H ☐ ★2 **Check Digits and Combine Ranges**: palindrome number: reverse only the second half with n % 10 and n // 10 and compare with the first half *[PracHub (solution)]*

M ☐ ★3 **Solve several coding interview problems**: the big-number addition on strings, with int(ch) per digit; the other parts are list-meeting and overlap sweeps *[PracHub (solution)]*

M ☐ **Add two big integers from digit lists**: the pattern template above *[PracHub (solution)]*

38. Split into pieces, fix each piece, join (and word wrap) ★

Strings and parsing (app section 43) 5 questions: 1 easy, 2 medium, 2 hard

Spot it: "snake_case to camelCase", "format a number", "justify text", "wrap at width".

Idea: Break the text at its separators, transform each piece with a tiny function, and join. For wrapping, add words greedily while current length + 1 + word length fits in the width, otherwise start a new line. For full justification, spread the spaces of a line over its gaps (the leftmost gaps get the remainder) and left-align the last line.

```
def to_camel(name):
    first, *rest = name.split('_')
    return first + ''.join(p.capitalize() for p in rest)

lines, current = [], []
for word in words:
    if current and len(' '.join(current + [word])) > width:
        lines.append(current);  current = []
    current.append(word)
lines.append(current)
```

Cost: O(total characters). **Traps:** Leading, trailing and repeated separators. A word longer than the width. Do not use += on strings in a long loop; collect pieces and join once.

Follow-ups: Convert back (camel to snake). Locale grouping (12,34,567). Words with pixel widths. Minimise raggedness instead of greedy packing (that is a DP over line breaks).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★**1 Snake Case to Camel Case with Repository-Wide Refactor**: the same function applied to every identifier found in the files *[1Point3Acres (notes), PracHub (solution) as "Convert Valid Snake Case to Lower Camel Case"]*

H ☐ ★**2 Convert Valid Snake Case to Lower Camel Case**: reformat(): split on '_', keep the first piece, capitalise the others *[PracHub (solution), Aced as "Define the scope and requirements for a snake_case to camelCase converter function.", 1Point3Acres (notes) as "Snake Case to Camel Case with Repository-Wide Refactor"]*

E ☐ ★**3 Implement string formatting and request tracking tasks**: split, fix each piece, join; the request-tracking part is a small dict *[PracHub (solution)]*

M ☐ **Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)**: count lines: go paragraph by paragraph, add a word if it fits (current + 1 + length <= width), else start a new line *[1Point3Acres (notes), PracHub (solution) as "Format words into wrapped/justified lines"]*

H ☐ **Implement a function to justify a list of words within a given width.**: pack_lines for the words, then spread the extra spaces over the gaps (the left gaps get the remainder); the last line is left-aligned *[Aced, PracHub (solution) as "Format words into wrapped/justified lines"]*

39. Walk two strings at once with two indices

Strings and parsing (app section 21) 6 questions: 2 easy, 3 medium, 1 hard

Spot it: "is a subsequence", "lexicographic order in a custom alphabet", "can board A become board B", "similar sentences".

Idea: Keep one index per string and advance them by a rule: advance both on a match, advance only the text pointer on a mismatch (subsequence), skip blank squares in both (board transforms). A custom alphabet order is just a rank dict: compare the ranks at the first differing position. Compress words to (letter, run length) pairs when stretched letters matter.

```
def is_subsequence(needle, text):
    i = 0
    for ch in text:
        if i < len(needle) and ch == needle[i]:
            i += 1
    return i == len(needle)

rank = {c: r for r, c in enumerate(alphabet)}
def in_order(a, b):                        # alien dictionary order
    for x, y in zip(a, b):
        if x != y: return rank[x] < rank[y]
    return len(a) <= len(b)                # a prefix comes first
```

Cost: O(n + m) per pair of strings. **Traps:** The prefix rule at the end of the comparison. Skipped squares must not shift the order of the remaining letters. Pieces that move in one direction only (L left, R right) cannot cross each other.

Follow-ups: Whole list of words in an alien order (compare neighbours). Case-insensitive or accented text. Minimum number of moves. Millions of words and many queries (compress once, index by shape).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

H ☐ ★**2 Find Dictionary Matches for a Jammed String**: compress every word to (letter, run length); a query matches when the letters agree and each run is equal or stretched to at least 3 *[PracHub (solution)]*

M ☐ ★**3 Can board states be transformed?**: skip the X squares in both strings; the remaining letters must match in order, and L may only move left, R only right *[PracHub (solution)]*

E ☐ **Determine if two words are sorted in lexicographic order.**: walk both words; at the first differing letter compare the letters' ranks in the given alphabet; if one word ends first, it must be the shorter *[Aced, PracHub (solution) as "Verify Ordering Under a Scrambled Alphabet"]*

M ☐ **Decide string transform with directional tokens and walls**: the same walk with dots as blanks and the movement rule from the statement (walls block) *[PracHub (solution)]*

E ☐ ★**1** ★ **Return all unique substrings of a given string.**: sort all suffixes; the unique substrings are all prefixes minus the shared prefixes of neighbouring suffixes *[Aced]*

M ☐ ★ **Determine if two sentences are similar.**: zip the two sentences; each pair of words must be equal or listed as similar (store pairs in both directions) *[Aced (solution)]*

40. Parent pointers: store who is above each node and climb ★

Data structure and object design (app section 41) 4 questions: 1 easy, 3 medium, 0 hard

Spot it: "lowest common ancestor with parent links", "validate a tree", "employee hierarchy", "is this a root".

Idea: Store parent[node] (and sometimes a set of children). Climbing is a loop following parents. Lowest common ancestor with parent links: advance two pointers, and when one falls off the top restart it at the other node; they meet at the ancestor after at most

len(a) + len(b) steps. Validating a parent array: climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle, and there must be exactly one root.

```
def lca(a, b):
    p, q = a, b
    while p is not q:
        p = p.parent if p else b
        q = q.parent if q else a
    return p

state = {}                                  # 0 unseen, 1 on this walk, 2 verified
def has_cycle(start):
    walk = []
    node = start
    while node is not None and state.get(node, 0) == 0:
        state[node] = 1;  walk.append(node);  node = parent[node]
    bad = node is not None and state.get(node) == 1
    for n in walk: state[n] = 2
    return bad
```

Cost: O(depth) per climb, O(n) for validating all nodes. **Traps:** The root has no parent (None). Check for exactly one root and for nodes that are not reachable. Deleting a manager means re-attaching the reports.

Follow-ups: Many LCA queries on a static tree (binary lifting or Euler tour). Move an employee and reject cycles. Nodes might not share a tree (return None).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Lowest Common Ancestor of a Binary Tree III**: two pointers climb; when one falls off the top it restarts at the other node, so both meet at the common ancestor *[HackTheRounds #56 (solution), PracHub (solution) as "Find LCA With Parent Pointers"]*

M ☐ ★2 **Employee Hierarchy Add / Delete**: manager[emp] and a set of reports per manager; delete re-attaches the reports to the deleted person's manager *[1Point3Acres (notes), PracHub (solution) as "Implement Employee Management and Expression Evaluation", Aced (solution) as "Employee Hierarchy"]*

E ☐ ★3 **Tree Node.**: the SQL version of the same idea: p_id is the parent pointer, NULL means root, being someone's parent means inner, else leaf *[Aced, PracHub (solution) as "Classify Tree Nodes in SQL"]*

M ☐ ★4 **Validate parent array forms a tree**: climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle; there must be exactly one root *[PracHub (solution)]*

41. Random pick without repeats: swap with the last and pop ★

Data structure and object design (app section 37) 2 questions: 0 easy, 1 medium, 1 hard

Spot it: "random set with O(1) delete", "shuffle", "unique random generator", "no repeat within k songs".

Idea: To delete from the middle of a list in O(1), swap the victim with the last element, update the position dict, and pop. Random pick is then a random index into the list. A shuffle is Fisher-Yates: for i from the end, swap a[i] with a random a[j] where j <= i. A huge pool 0..n-1 can be lazy: keep a dict of only the swapped slots.

```
class RandomSet:
    def __init__(self):  self.items, self.where = [], {}
    def add(self, x):
        if x in self.where: return False
        self.where[x] = len(self.items);  self.items.append(x);  return True
    def remove(self, x):
        if x not in self.where: return False
        i, last = self.where[x], self.items[-1]
        self.items[i] = last;  self.where[last] = i
        self.items.pop();  del self.where[x];  return True
    def pick(self):  return random.choice(self.items)

for i in range(len(a) - 1, 0, -1):          # Fisher-Yates
    j = random.randint(0, i);  a[i], a[j] = a[j], a[i]
```

Cost: O(1) per add, remove and pick. **Traps:** Update the position of the moved element before deleting. A shuffle that picks j from the whole array is biased. Raise or return a sentinel when the pool is empty.

Follow-ups: Duplicates allowed (map value to a set of positions). Reset in O(1) (keep a counter, do not rebuild). Only a coin flip API exists (rejection sampling). Thread safety.

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Design set with O(1) random access**: add appends and records the position; delete swaps with the last item and pops; get_random picks a random

index *[PracHub (solution)]*

H ☐ **Shuffle array using random integer API**: Fisher-Yates: for i from the end, j = rand_int(0, i), swap a[i] and a[j] *[PracHub (solution)]*

42. Tree recursion, tier 2: pass a rule down, return a node or yes/no up ★

Binary trees and BSTs (app section 36) 3 questions: 0 easy, 3 medium, 0 hard

Spot it: "valid BST", "lowest common ancestor", "in-order successor", "mode in a tree".

Idea: Pass information down as arguments (the allowed value window low and high for BST validation) and return a verdict or a node up. A BST in-order walk is sorted, so equal values are neighbours (running count gives the mode) and the successor of x is found by descending: go left after a hit, right otherwise. Lowest common ancestor: return the node if it is p or q; a node that receives an answer from both sides is the ancestor.

```
def is_bst(node, low=float('-inf'), high=float('inf')):
    if node is None: return True
    if not low < node.val < high: return False
    return is_bst(node.left, low, node.val) and is_bst(node.right, node.val, high)

def lca(node, p, q):
    if node is None or node is p or node is q: return node
    left, right = lca(node.left, p, q), lca(node.right, p, q)
    return node if left and right else left or right
```

Cost: O(n) for the walks, O(height) for a BST descent. **Traps:** Compare against the whole window, not only the parent. Decide how duplicates are placed. p or q may be missing from the tree (verify with a count).

Follow-ups: Not a BST (generic traversal). O(1) space (Morris traversal). Recover a BST with two swapped nodes. Deep trees (iterative).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Lowest Common Ancestor**: dfs returns the node if it is p or q; a node that gets an answer from both sides is the ancestor *[HackTheRounds #631 (solution), PracHub (solution) as "Lowest Common Ancestor in an Explicit Binary Tree", Aced (solution) as "Find the lowest common ancestor (LCA) of two nodes in a binary tree."]*

M ☐ **Find mode in a trinary search tree**: in-order walk (left, middle, right); equal values are neighbours, so one running count finds the mode(s) *[PracHub (solution)]*

M ☐ **Determine if a given binary tree is a binary search tree (BST).**: check(): pass lo and hi bounds down; fail when a value leaves its allowed window *[Aced (solution), PracHub (solution) as "Validate a Binary Search Tree"]*

43. Build and serialize: write out a structure, read it back with the same walk ★

Binary trees and BSTs (app section 38) 3 questions: 0 easy, 2 medium, 1 hard

Spot it: "serialize / deserialize", "construct tree from preorder and inorder", "print a directory tree", "encode a list of strings".

Idea: Pick a format that makes decoding unambiguous and decode with the same traversal order you used to encode. Strings: write length, a separator, then the text, so any character may appear inside. Preorder plus inorder: the first preorder item is the root and its position in the inorder list splits left from right. N-ary tree: write the value and the child count. Graph: write one line per node with the ids of neighbours, create all nodes first, then link.

```
def encode(strings):
    return ''.join(f'{len(s)}#{s}' for s in strings)
def decode(text):
    out, i = [], 0
    while i < len(text):
        j = text.index('#', i);  length = int(text[i:j])
        out.append(text[j + 1:j + 1 + length]);  i = j + 1 + length
    return out

def build(preorder, inorder):
    index = {v: i for i, v in enumerate(inorder)}
    it = iter(preorder)
    def make(low, high):
        if low > high: return None
        node = Node(next(it));  mid = index[node.val]
        node.left, node.right = make(low, mid - 1), make(mid + 1, high)
        return node
    return make(0, len(inorder) - 1)
```

Cost: O(n) time and space. **Traps:** Build left before right when consuming a preorder iterator. Values may repeat in preorder plus inorder construction (ambiguous). Graph: handle cycles with a seen map.

Follow-ups: Postorder plus inorder. Level order encoding. Binary encoding for compactness. Why not JSON (length prefix works for any bytes).

Questions in this section (E easy, M medium, H hard). Listed with PracHub solutions first, then 1Point3Acres, then the rest. Tick the small box in front of a question when you have done it (click it, then save the file). The numbered stars mark the 150 questions to learn in the whole book, chosen by how recently Google was reported asking them (every date found, from all sources): star 1 is the most recent in its pattern. Solve them first, in the order of the star number, because each carries an idea this pattern teaches and you can reuse it on a brand-new question. The big orange stars are those numbered ones. A small red star marks a question with no PracHub solution:

M ☐ ★1 **Construct a binary tree given its preorder and inorder traversals.**: the first preorder value is the root; its position in a value-to-index map of the inorder list splits left from right *[**Aced (solution)**, **PracHub (solution) as "Construct tree from inorder & postorder"**]*

M ☐ **Write functions to serialize and deserialize a list of strings.**: write len + '#' + text for each string; read the length, then take exactly that many characters *[**Aced (solution)**, **PracHub (solution) as "Implement encode/decode for list of strings"**]*

H ☐ **Implement a decode and encode function of a general N-ary tree.**: write value and child count for each node (preorder); read the count to know how many children to build *[**Aced**, **PracHub (solution) as "Encode an N-ary tree as a string and decode it back to the identical tree"**]*

Appendix: algorithms hiding inside single questions

These show up once or twice (often as a follow-up) but are worth knowing by name.

Fenwick tree (binary indexed tree)

Prefix sums with point updates in $O(\log n)$. Used for count of smaller numbers after self, dynamic range sums, and "array changes between queries" follow-ups. Index from 1; the lowest set bit of an index says how many items its cell covers.

```
class Fenwick:
    def __init__(self, n):  self.tree = [0] * (n + 1)
    def add(self, i, delta):                # i is 1-based
        while i < len(self.tree):
            self.tree[i] += delta;  i += i & -i
    def prefix(self, i):                    # sum of items 1..i
        total = 0
        while i > 0:
            total += self.tree[i];  i -= i & -i
        return total
# count of smaller numbers after self: go from the right, answer = prefix(rank - 1), then add(rank, 1)
```

Segment tree (first slot that fits, range minimum, range updates)

Store a summary of each range in a tree over positions. The waitlist question keeps the maximum party size per range, so "first group of at most s" descends left when the left half has a big enough slot. Cost $O(\log n)$ per update and query.

```
class MaxTree:
    def __init__(self, values):
        self.n = 1
        while self.n < len(values): self.n *= 2
        self.tree = [0] * (2 * self.n)
        for i, v in enumerate(values): self.tree[self.n + i] = v
        for i in range(self.n - 1, 0, -1):
            self.tree[i] = max(self.tree[2 * i], self.tree[2 * i + 1])
    def update(self, i, value):
        i += self.n;  self.tree[i] = value
        while i > 1:
            i //= 2;  self.tree[i] = max(self.tree[2 * i], self.tree[2 * i + 1])
    def first_at_least(self, need):         # leftmost index with value >= need
        if self.tree[1] < need: return -1
        i = 1
        while i < self.n:
            i = 2 * i if self.tree[2 * i] >= need else 2 * i + 1
        return i - self.n
```

Rolling hash (Rabin-Karp) for repeated substrings

Longest duplicated substring: binary search the length L (pattern 5), and for each L slide a hash over the string in $O(n)$, storing hashes in a set. Roll by removing the leading character and adding the next. Verify a hash hit to rule out collisions, or use two moduli.

```
def has_duplicate(s, length, base=131, mod=(1 << 61) - 1):
    if length == 0: return True
    power = pow(base, length - 1, mod);  h = 0;  seen = set()
    for i, ch in enumerate(s):
        if i >= length: h = (h - ord(s[i - length]) * power) % mod
        h = (h * base + ord(ch)) % mod
        if i >= length - 1:
            if h in seen: return True
            seen.add(h)
    return False
```

Sparse table (range minimum in $O(1)$)

Fence painting asks for the lowest stripe in a range again and again. Precompute `table[j][i]` = minimum of the 2^j items starting at i; a query takes two overlapping blocks.

```
def build(a):
    table = [a[:]];  j = 1
    while (1 << j) <= len(a):
        prev = table[-1]
        table.append([min(prev[i], prev[i + (1 << (j - 1))])
                      for i in range(len(a) - (1 << j) + 1)])
        j += 1
    return table
def range_min(table, l, r):                 # inclusive
    j = (r - l + 1).bit_length() - 1
```

```
    return min(table[j][l], table[j][r - (1 << j) + 1])
```

Huffman coding

Greedy with a heap (pattern 24): repeatedly pop the two lightest nodes and push a parent whose weight is their sum. Reading the final tree, going left is 0 and going right is 1. Canonical codes assign codes by length then symbol so only the lengths need storing.

```
def huffman_lengths(weights):               # weights: {symbol: weight}
    heap = [(w, i, {s: 0}) for i, (s, w) in enumerate(weights.items())]
    heapq.heapify(heap);  tiebreak = len(heap)
    while len(heap) > 1:
        w1, _, left = heapq.heappop(heap);  w2, _, right = heapq.heappop(heap)
        merged = {s: d + 1 for s, d in {**left, **right}.items()}
        heapq.heappush(heap, (w1 + w2, tiebreak, merged));  tiebreak += 1
    return heap[0][2]                       # symbol -> code length
```

Fast doubling and matrix power (huge n recurrences)

When n is 10^{18}, Fibonacci-like recurrences are computed in $O(\log n)$ by squaring. Fast doubling: $F(2k) = F(k)(2F(k+1)-F(k))$ and $F(2k+1) = F(k)^2 + F(k+1)^2$. Apply the modulus after every multiplication.

```
def fib(n, mod=10**9 + 7):                  # returns (F(n), F(n+1))
    if n == 0: return (0, 1)
    a, b = fib(n // 2, mod)
    c = a * ((2 * b - a) % mod) % mod
    d = (a * a + b * b) % mod
    return (d, (c + d) % mod) if n % 2 else (c, d)
```

Group testing, augmenting paths and other named ideas

- **Group testing** (find all bad pairs with few oracle calls): test a group in one call; if it passes, discard the whole group; if it fails, split it. Lower bound is about $k \log(n/k)$ calls for k bad items.
- **Augmenting path** (match people to questions): try each preferred question; if taken, recursively ask the owner to move to another; undo on failure. This is bipartite matching in $O(V \cdot E)$.
- **Fisher-Yates** and **reservoir sampling**: shuffle in place from the end; to pick one random item from a stream, replace the current pick with the i-th item with probability $1/i$.
- **Count-Min sketch** / **Misra-Gries**: approximate top-k over an unbounded stream in fixed memory.
- **Sprague-Grundy**: a sum of independent games is won by the player who makes the xor of the game values non-zero (the follow-up behind “games on several piles”).
- **Morris traversal**: in-order walk of a BST in $O(1)$ extra space by temporarily threading predecessor links.

Coverage by section and source

One table per source, in this order: PracHub, 1Point3Acres, HackTheRounds, Aced. In each table every pattern is listed, sorted by that source's own no column (no gaps first, then one gap, then two, and so on; ties keep the reading order), so the same pattern name appears in every table. A star marks the first pattern of each new no value (0, then 1, then 2, ...), with a thin line between the groups. Click a pattern name to jump to it.

PracHub: questions with a solution (the page has a Solution tab with code)

The pattern boxes here and in the contents are one linked box: tick it in either place and both tick. In Adobe Acrobat it also works with the question boxes in both directions: ticking the pattern box ticks every question of that pattern (unticking clears them), and ticking the last question ticks the pattern box. Preview does not run that automatic part, so there you tick each box yourself. Save the file (File, Save) to keep the ticks.

#	Pattern	Qs	PracHub yes	PracHub no
1	☐ ★ Union-find	7	7	0
2	☐ Frontier search (BFS or DFS), tier 1	6	6	0
4	☐ Backtracking	7	7	0
5	☐ Best-first search with a heap	8	8	0
6	☐ Topological sort (Kahn) and cycle detection	4	4	0
7	☐ Frontier search, tier 2	4	4	0
8	☐ Top-K by frequency	5	5	0
9	☐ Games	5	5	0
10	☐ K-way merge	4	4	0
13	☐ Grid DP	5	5	0
16	☐ Sweep over events in time order	9	9	0
18	☐ Sorted intervals	6	6	0
19	☐ Greedy	5	5	0
29	☐ Nested structures	5	5	0
32	☐ Trie	8	8	0
33	☐ A dict for lookup plus an ordered container for who is next	5	5	0
34	☐ Build or transform a grid with index arithmetic	3	3	0
35	☐ Look at the input once and keep only a small summary	6	6	0
36	☐ Simulate a machine	5	5	0
37	☐ Digits and characters	4	4	0
38	☐ Split into pieces, fix each piece, join (and word wrap)	5	5	0
40	☐ Parent pointers	4	4	0
41	☐ Random pick without repeats	2	2	0
42	☐ Tree recursion, tier 2	3	3	0
43	☐ Build and serialize	3	3	0
3	☐ ★ A heap of what is available	8	7	1
12	☐ Recurrence on a position or a range (memo or table)	8	7	1
15	☐ Dict of per-key state (update, expire, rank)	8	7	1
17	☐ Binary search on a yes/no condition	7	6	1
20	☐ Count pairs by looking up each partner	3	2	1
24	☐ Sets and counters across two collections	4	3	1
25	☐ Sliding window	4	3	1
26	☐ Two pointers moving toward each other (or along two lists)	6	5	1
28	☐ Prefix sums, tier 2	3	2	1
30	☐ Tree recursion, tier 1	8	7	1
31	☐ Tree DP	5	4	1
11	☐ ★ Rolling state	5	3	2
14	☐ Counting or reaching an amount	4	2	2
21	☐ Prefix sums, tier 1	6	4	2
22	☐ Runs and groups	4	2	2
23	☐ bisect on a sorted list	6	4	2
27	☐ Group by a canonical key	5	3	2
39	☐ Walk two strings at once with two indices	6	4	2
	All patterns	**228**	**203**	**25**

1Point3Acres: questions with notes (an approach write-up; this site has no solution code)

#	Pattern	Qs	1Point3Acres yes	1Point3Acres no
13	★ Grid DP	5	5	0
29	Nested structures	5	5	0
7	★ Frontier search, tier 2	4	3	1
20	Count pairs by looking up each partner	3	2	1
22	Runs and groups	4	3	1
31	Tree DP	5	4	1
34	Build or transform a grid with index arithmetic	3	2	1
9	★ Games	5	3	2
15	Dict of per-key state (update, expire, rank)	8	6	2
25	Sliding window	4	2	2
33	A dict for lookup plus an ordered container for who is next	5	3	2
35	Look at the input once and keep only a small summary	6	4	2
38	Split into pieces, fix each piece, join (and word wrap)	5	3	2
41	Random pick without repeats	2	0	2
6	★ Topological sort (Kahn) and cycle detection	4	1	3
12	Recurrence on a position or a range (memo or table)	8	5	3
19	Greedy	5	2	3
23	bisect on a sorted list	6	3	3
28	Prefix sums, tier 2	3	0	3
32	Trie	8	5	3
36	Simulate a machine	5	2	3
40	Parent pointers	4	1	3
42	Tree recursion, tier 2	3	0	3
43	Build and serialize	3	0	3
1	★ Union-find	7	3	4
2	Frontier search (BFS or DFS), tier 1	6	2	4
4	Backtracking	7	3	4
8	Top-K by frequency	5	1	4
10	K-way merge	4	0	4
14	Counting or reaching an amount	4	0	4
17	Binary search on a yes/no condition	7	3	4
21	Prefix sums, tier 1	6	2	4
24	Sets and counters across two collections	4	0	4
37	Digits and characters	4	0	4
3	★ A heap of what is available	8	3	5
11	Rolling state	5	0	5
18	Sorted intervals	6	1	5
26	Two pointers moving toward each other (or along two lists)	6	1	5
27	Group by a canonical key	5	0	5
5	★ Best-first search with a heap	8	2	6
16	Sweep over events in time order	9	3	6
30	Tree recursion, tier 1	8	2	6
39	Walk two strings at once with two indices	6	0	6
	All patterns	**228**	**90**	**138**

HackTheRounds: questions with a solution (the page shows solution code)

#	Pattern	Qs	HackTheRounds yes	HackTheRounds no
20	★ Count pairs by looking up each partner	3	1	2
41	Random pick without repeats	2	0	2
42	Tree recursion, tier 2	3	1	2
14	★ Counting or reaching an amount	4	1	3
25	Sliding window	4	1	3
28	Prefix sums, tier 2	3	0	3
34	Build or transform a grid with index arithmetic	3	0	3
40	Parent pointers	4	1	3
43	Build and serialize	3	0	3
6	★ Topological sort (Kahn) and cycle detection	4	0	4
7	Frontier search, tier 2	4	0	4
10	K-way merge	4	0	4
22	Runs and groups	4	0	4
24	Sets and counters across two collections	4	0	4
37	Digits and characters	4	0	4
8	★ Top-K by frequency	5	0	5
9	Games	5	0	5
11	Rolling state	5	0	5
13	Grid DP	5	0	5
19	Greedy	5	0	5
21	Prefix sums, tier 1	6	1	5
23	bisect on a sorted list	6	1	5
27	Group by a canonical key	5	0	5
29	Nested structures	5	0	5
31	Tree DP	5	0	5
33	A dict for lookup plus an ordered container for who is next	5	0	5
36	Simulate a machine	5	0	5
38	Split into pieces, fix each piece, join (and word wrap)	5	0	5
2	★ Frontier search (BFS or DFS), tier 1	6	0	6
3	A heap of what is available	8	2	6
18	Sorted intervals	6	0	6
26	Two pointers moving toward each other (or along two lists)	6	0	6
35	Look at the input once and keep only a small summary	6	0	6
39	Walk two strings at once with two indices	6	0	6
1	★ Union-find	7	0	7
4	Backtracking	7	0	7
15	Dict of per-key state (update, expire, rank)	8	1	7
17	Binary search on a yes/no condition	7	0	7
5	★ Best-first search with a heap	8	0	8
12	Recurrence on a position or a range (memo or table)	8	0	8
16	Sweep over events in time order	9	1	8
30	Tree recursion, tier 1	8	0	8
32	Trie	8	0	8
	All patterns	**228**	**11**	**217**

Aced: questions with a solution (the Solution tab shows code)

#	Pattern	Qs	Aced yes	Aced no
11	★ Rolling state	5	4	1
14	Counting or reaching an amount	4	3	1
28	Prefix sums, tier 2	3	2	1
42	Tree recursion, tier 2	3	2	1
43	Build and serialize	3	2	1
10	★ K-way merge	4	2	2
41	Random pick without repeats	2	0	2
6	★ Topological sort (Kahn) and cycle detection	4	1	3
20	Count pairs by looking up each partner	3	0	3
24	Sets and counters across two collections	4	1	3
26	Two pointers moving toward each other (or along two lists)	6	3	3
34	Build or transform a grid with index arithmetic	3	0	3
40	Parent pointers	4	1	3
7	★ Frontier search, tier 2	4	0	4
8	Top-K by frequency	5	1	4
19	Greedy	5	1	4
22	Runs and groups	4	0	4
25	Sliding window	4	0	4
27	Group by a canonical key	5	1	4
33	A dict for lookup plus an ordered container for who is next	5	1	4
37	Digits and characters	4	0	4
2	★ Frontier search (BFS or DFS), tier 1	6	1	5
4	Backtracking	7	2	5
9	Games	5	0	5
13	Grid DP	5	0	5
18	Sorted intervals	6	1	5
21	Prefix sums, tier 1	6	1	5
29	Nested structures	5	0	5
31	Tree DP	5	0	5
36	Simulate a machine	5	0	5
38	Split into pieces, fix each piece, join (and word wrap)	5	0	5
39	Walk two strings at once with two indices	6	1	5
12	★ Recurrence on a position or a range (memo or table)	8	2	6
17	Binary search on a yes/no condition	7	1	6
23	bisect on a sorted list	6	0	6
30	Tree recursion, tier 1	8	2	6
35	Look at the input once and keep only a small summary	6	0	6
1	★ Union-find	7	0	7
3	★ A heap of what is available	8	0	8
5	Best-first search with a heap	8	0	8
15	Dict of per-key state (update, expire, rank)	8	0	8
32	Trie	8	0	8
16	★ Sweep over events in time order	9	0	9
	All patterns	**228**	**36**	**192**

After the 150 starred questions: what to do next

Stop adding new problems once the starred questions are learned, and spend the remaining time on how you perform in the interview.

What to practise

1. **Mock interviews on the questions you have not seen.** Take a question from the unstarred ones. Within two minutes say the pattern, the data structure and the cost out loud, then code it in 25 to 30 minutes. This shows whether you can recognise a new question as a known pattern, which is what the real round tests.
2. **Practise talking while you code.** Ask clarifying questions, state a brute force first, then improve it, and say why you chose the structure. Practise with a listener or record yourself.
3. **Test every solution like you will in the interview.** Walk a small example by hand, then the edge cases: empty input, one item, duplicates, ties, and the case where there is no answer.
4. **Drill the follow-ups.** Google usually adds a twist after the first answer: a stream, a memory limit, many queries, an extra rule. Rehearse the follow-ups line of each pattern.
5. **Redo your misses.** Keep a short list of every problem you struggled with or peeked at, and redo each one from a blank file two days later. This beats starting new problems.
6. **Go through your friend's PDF** of reported questions at least 4 days before the interview, that is by Nov 2 for an interview on Nov 6. Those friend reports are no longer in the question lists, so read them in the set merged onto the end of this book: Your friend's practice problem set (page 44).
7. **Write the templates from memory** for your 10 weakest patterns the night before.
8. **Prepare the non-coding part.** Have two or three short stories ready for behavioural questions, know why Google, and sleep well before the interview. Sample answers written from your resume (page 42).

What to read in this book, in this order

1. Follow-up families that repeat everywhere (page 6): read it first and again the night before. The usual second questions Google asks.
2. Pattern finder (page 6): until you can pick the pattern from the words of a question without opening the book.
3. Python toolbox and complexity (page 5): skim for the tools you forget (heapq, deque, bisect, Counter) and the costs you must state.
4. The **Traps** and **Follow-ups** lines on every pattern page: a few minutes per pattern, they hold the mistakes and twists you are most likely to be caught on.
5. Priority order (page 4): if a high ranked pattern still is not comfortable, reread its page and do one more problem on it.
6. Appendix (page 35): skim last. Fenwick tree, segment tree and Huffman coding show up as follow-ups now and then, but you do not need to master them.

Leave for the very end: the questions with a red star, because you cannot check your answer against a PracHub solution.

Behavioural questions: sample answers from your resume

Every answer below uses only facts that are on your resume. Read each one aloud until you can say it in your own words, in about a minute and a half. Keep the numbers, because they are what the interviewer will remember.

Tell me about yourself.

I am a computer science student at Grambling State University with a 3.95 GPA, graduating in May 2028. What I enjoy most is building something from start to finish and then measuring whether it really works. I founded ZipMath, a mobile app that teaches mental math shortcuts, where I designed the lessons, the engagement system and a question engine that computes every answer in code. I have had three documentation pull requests merged into Apache Kafka, the Kubernetes Java client and PostgreSQL JDBC. In my GPU Robot Lab project I made a matrix multiply kernel 3.4 times faster by working out why my first version was slow. I also lead weekly data structures and algorithms workshops for more than 90 members as Academic Chair of ColorStack Grambling. I want to keep building at that depth next to strong engineers, which is why this internship at Google appeals to me.

Why Google?

Three parts of my own work lead me to Google. One of my open source contributions was to the Kubernetes Java client, which grew out of Google, and I wrote its release history from v15.0.0 to v25.0.0, including the breaking changes in v20.0.0, so I have read closely how a project like that explains change to its users. My GPU Robot Lab used JAX through MuJoCo MJX to run robot simulations up to 126 times faster than one CPU thread, so I already work with tools Google builds. And I like that Google engineers write code at a scale where a small inefficiency matters, which is the kind of problem I enjoyed chasing in my kernel work. I also teach algorithms to my peers, and I want to learn from people who are much better at it than I am.

Tell me about a hard technical problem you solved.

In my GPU Robot Lab I wrote Triton matrix multiply kernels and checked them against PyTorch on an NVIDIA T4. My first version was slow, and I did not guess at the cause. I traced it to register spilling and tuned the kernel from 1,096 to 3,727 GFLOPS, which is 3.4 times faster. I kept going and found that the compiled code uses no tensor cores, which is the likely reason for the gap that remains. I reported that as an open finding and did not claim the kernel was finished. To make sure my numbers could be trusted I ran 10 repeats of every version, because PyTorch itself varied by up to 44 percent between runs, and I changed one thing at a time to test whether the launch order mattered. The lesson I took is to measure before changing anything, and to say clearly what is still unexplained.

Tell me about a time you took initiative or dealt with an unclear problem.

With ZipMath nobody told me what to build, so I had to decide what mattered. I wanted learners to keep practising, so I designed an engagement system with XP, daily streaks, three hearts per practice round, and a one to three star rating where a round with no mistakes earns three stars. I also made one technical decision I care about: the question engine computes every answer in code, so a practice question can never have a wrong answer key. For an app that teaches math, one wrong answer would cost the learner's trust straight away. I shipped 26 levels across the Basic, Intermediate and Advanced tiers, with animated lessons and quizzes that save progress on the device, and there is a live demo.

Tell me about a time you led others.

As Academic Chair of ColorStack Grambling I lead weekly data structures and algorithms workshops for more than 90 members. The hard part is that people arrive at very different levels, so I teach each topic as a small set of patterns that they can recognise in new problems, instead of as a pile of separate questions. I am also starting as the AWS Student Builder Group Leader at Grambling in November 2026, and I plan to bring the same approach: make it easy to join, easy to learn from, and clear what to do next.

Tell me about a time you worked on a team.

In the AI4ALL Ignite project, our team of three built a model that predicts next day stock decreases from financial news headlines, using FinBERT sentiment features and more than 20 million stock price records. I took ownership of the evaluation. I wrote the script that runs 5-fold stratified cross validation on a sample of 50,000 articles and produces classification reports and confusion matrices for both models. That mattered because the Random Forest reached 54.4 percent accuracy against 50.8 percent for the Logistic Regression baseline on 64,599 held out headlines, which is a modest gap, and a careful evaluation was the only way to say honestly whether it was real. I learned that on a team, owning the part that keeps everyone honest is a useful role.

Tell me about a time you worked with feedback or a review.

My three documentation pull requests to Apache Kafka, the Kubernetes Java client and PostgreSQL JDBC each went through the project's own review process before they were merged, so I had to write for maintainers who know the code far better than I do. For Kafka I rewrote the Streams guide and the Javadoc of two core classes to explain that turning on topology optimizations takes two steps, with a code example and a warning that calling build() without Properties silently skips them. That came from spotting a mistake a reader could easily make. It taught me to write for the person who will misread the page, and to treat review as part of getting the change right.

Tell me about a time you had to learn something quickly.

Triton and MuJoCo MJX were both new to me when I started the GPU Robot Lab. I learned them by building: first the kernels, always checked against PyTorch so I knew at every step whether I was right, then the robot simulation, then a planner and a neural network controller trained with a curriculum until it could swing up and hold a pendulum. Having a reference to check against is how I learn quickly without fooling myself. I used the same habit when I wrote the Kubernetes client release history: I read each release in turn and checked every claim before writing it down.

Tell me about a time you made sure your work was high quality.

For InterviewPair, an app that pairs students for mock technical interviews, I did not stop at a working login. I wrote more than 30 automated tests, including cases for expired tokens, tampered tokens and tokens signed with the wrong key, because those are the cases where a login system fails in real life. I also set up continuous integration on GitHub Actions that builds the Docker image on every push, so a broken change shows up immediately and not days later. The GPU Robot Lab has the same habit, with more than 90 automated tests in continuous integration.

How do you manage your time with so many things going on?

I keep a 3.95 GPA while building ZipMath, running weekly workshops and contributing to open source, and the way I manage it is by working in small pieces with a clear next step, and by letting tests and automated builds remember what works so that I do not have to. That means I can come back to a project after a busy exam week and trust that it still runs. It also means I pick my projects deliberately: each one on my resume taught me something specific, instead of being one more thing to keep alive.

Tell me about a time you explained something complicated to someone.

Results like register spilling are easy to state and hard to feel, so for the GPU Robot Lab I built an interactive explainer site on GitHub Pages that lets a reader see the effect for themselves. I use the same approach in my algorithms workshops: start from one small example, show the pattern behind it, and then let the group apply it to a new problem on their own. When people can apply an idea to something they have not seen, I know they understood it.

What questions could you ask the interviewer?

I would ask what a strong intern on the team accomplishes in the first month, how interns get feedback on their code and how often, and what the hardest technical problem the team solved recently was and how they approached it. I would also ask what they would have wanted to know before they started at Google. These show that I care about doing good work and about learning, and each one is an opening for a conversation and not a yes or no answer.

Your friend's practice problem set

The next 139 pages are the Google SWE Intern practice problem set: 88 full, solvable Python problems rebuilt from the interview reports that ColorStack members posted in the company Google Slack channel, with statements, constraints, signatures, worked examples and follow-ups. They are the questions that were taken out of the Primer, so read them here.

Read them at least 4 days before the interview. Each problem says who reported it, when, and how closely it matches what they remembered. They are recollections for practice, not official Google material.

Google SWE Intern — Practice Problem Set

88 full, solvable Python problems reconstructed from real Google Software Engineering Intern interview reports posted by ColorStack members in the `#company-google` Slack channel (January 2025 – September 2026).

Each problem is self-contained: a complete statement with constraints, a Python function or class signature, worked examples with expected outputs, and follow-ups where the original interview had them. Team-matching rounds are excluded. Where a Slack post was only a fragment, the problem was completed or framed to faithfully match what the person described — each problem's **Fidelity** line says which.

Attribution names the ColorStack member who reported the question and the date. Treat all problems as candidate recollections reconstructed for practice, not official Google material.

Contents

1. Maximum Subarray Sum With a Minimum Length Constraint
2. Range Sum Query 2D (Immutable Matrix)
3. Round-Robin Job Scheduler
4. Decode String
5. Jump Game II (Minimum Jumps)
6. American Football Score Combinations
7. Contains Duplicate
8. K-diff Pairs in an Array
9. Coin Change (Fewest Coins)
10. Find the First Occurrence of a Substring
11. Find Median from a Data Stream
12. Guess the Secret Word
13. Sustained-Error Time Window
14. Insert Delete GetRandom in O(1)
15. Count Unique Triplets That Sum to Zero
16. Coins on the Table
17. Print a Directory Tree
18. Top-K Chatters by Word Count
19. Find the Pivot Index (Balanced Split) in O(n)
20. Footpath Beauty After Repainting
21. Shortest Distance From All Buildings
22. Minimum Makespan on Identical CPUs
23. Plant Crops in Connected Regions
24. Meeting Rooms III

25. Serialize and Deserialize a Graph
26. Longest Non-Decreasing Subarray From Two Arrays
27. Filled ASCII Diamond
28. Infinite Deck Card Duel
29. Decode String
30. Russian Doll Envelopes
31. Streaming Top-K Words
32. Cake Balance Line
33. Maximum-Sum Subarray With Equal Endpoints
34. Store With Cheapest-Item Removal
35. Delete an Entire Binary Tree
36. Next Upcoming Birthday in a Family Tree
37. Bigram Word Predictor
38. Number of Visible People in a Queue
39. Meeting Rooms II With Day Wrap-Around
40. Top-K Frequent IP Addresses From Logs
41. Validate Race Results (Cycle Detection)
42. Count Super Streaks
43. Meetings Minus a Do-Not-Schedule Block
44. Restaurant Waitlist
45. Group Timestamps Into Close Intervals
46. Unique Random Number Generator
47. Constrained Dice Roll Sequences
48. Minimum Number of Servers (Meeting Rooms II Variant)
49. Random Song Shuffler With a No-Repeat Window
50. Alarm for the Next Upcoming Birthday
51. Top K Most Frequent IP Addresses
52. Validate a Mahjong-Style Hand of Melds
53. Filter Common Elements Per Window (Two Lists)
54. Count Sorted Strings With a Given Prefix
55. Balance Loads by Removing One and Splitting Equally
56. Movie Recommendations via Transitive Similarity
57. Random Generator Without Repeats
58. Farthest City for a New Data Center
59. First Index Reaching a Threshold (Binary Search)
60. Mario-Style Score Tracker (OOP)
61. Minimum Conference Rooms (Meeting Rooms II)
62. Split D/R Sequence Into Balanced Halves (One Cut)

63. Aggregate FizzBuzz Offer Matches
64. Positional Matches Between Two Lists
65. Count Super Streaks
66. Maximum Nesting Depth of Boxes
67. Permutation Sequence
68. Paint House II
69. Exclude First-K-Unique From Another List
70. Can All Runners Be Scheduled (Course Schedule Variant)
71. Find First Occurrence of a Substring (strStr)
72. Nearest Favorite City by Travel Time
73. Maximize Shelf Space (Unbounded Knapsack)
74. Ways to Sum to N Using 1s and 2s
75. Minimum Hands to Play a Piano Melody
76. Count People in a Postal Code Range
77. K Largest Elements Across Sorted Lists
78. Count Axis-Aligned Squares Among Grid Points
79. Next Birthday (BST In-Order Successor with Wraparound)
80. Count Passcode Patterns on a Keypad
81. Root a Tree So It Becomes a Binary Tree
82. Group Shifted Strings
83. Range Module (Insert and Point Query)
84. First Out-of-Order Character
85. Radio Broadcast Reachability
86. Flood Fill
87. Word Break: All Sentences
88. Caesar Cipher: Recover Key, Encrypt, and Decrypt

Problem 1. Maximum Subarray Sum With a Minimum Length Constraint

Source: Tochi Onwuasoanya, 2025-08-19 — technical interview (Slack report) **Fidelity:** Completed from a partial description **Topics:** Array, Prefix Sum, Sliding Window, Kadane's Algorithm **Difficulty:** Medium

Problem

This is the classic "maximum subarray" problem with the extra constraint the report mentioned ("Max sub array question with a weird constraint").

Given an integer array `nums` and an integer `min_len`, find the contiguous subarray (containing **at least `min_len` elements**) that has the largest sum, and return that sum.

Constraints:

- `1 <= min_len <= len(nums) <= 10^5`
- `-10^4 <= nums[i] <= 10^4`
- The array is guaranteed to be non-empty and `min_len` never exceeds `len(nums)`, so a valid subarray always exists.
- Values may be negative; the answer can therefore be negative.

Function signature

```
def max_subarray_min_len(nums: list[int], min_len: int) -> int:
    """Return the maximum sum over all contiguous subarrays of length >= min_len."""
    ...
```

Examples

```
Input:  nums = [1, -2, 3, 4, -1], min_len = 2
Output: 7
Explanation: The subarray [3, 4] (length 2) has sum 7, the largest among all subarrays
of length >= 2.
```

```
Input:  nums = [-1, -2, -3], min_len = 2
Output: -3
Explanation: Every element is negative. The best length->=2 subarray is [-1, -2] with
sum -3.
```

```
Input:  nums = [5, 4, -1, 7, 8], min_len = 1
Output: 23
Explanation: With min_len = 1 this reduces to ordinary Kadane; the whole array sums to
23.
```

Follow-ups

- The report said each question had a follow-up where a constraint was changed. Now bound the subarray length to the range `[min_len, max_len]`. Hint: maintain a sliding window of prefix-sum minima with a monotonic deque.

Problem 2. Range Sum Query 2D (Immutable Matrix)

Source: Tochi Onwuasoanya, 2025-08-19 — technical interview (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** 2D Prefix Sum, Matrix, Design **Difficulty:** Medium

Problem

Given a 2D integer matrix, handle multiple queries of the following type: return the sum of the elements inside the rectangle defined by its upper-left corner `(row1, col1)` and lower-right corner `(row2, col2)`, inclusive.

Implement a class `NumMatrix` that is initialized once with the matrix and can then answer many `sum_region` queries efficiently (each query in O(1) after preprocessing).

Constraints:

- `1 <= rows, cols <= 200`
- `-10^5 <= matrix[i][j] <= 10^5`
- `0 <= row1 <= row2 < rows`, `0 <= col1 <= col2 < cols`
- There may be up to `10^4` calls to `sum_region`.

Function signature

```
class NumMatrix:
    def __init__(self, matrix: list[list[int]]) -> None:
        """Preprocess the matrix (e.g., build a 2D prefix-sum table)."""
        ...

    def sum_region(self, row1: int, col1: int, row2: int, col2: int) -> int:
        """Return the sum of the submatrix with corners (row1, col1) and (row2, col2),
inclusive."""
        ...
```

Examples

```
Input:
  matrix = [[3, 0, 1, 4, 2],
            [5, 6, 3, 2, 1],
            [1, 2, 0, 1, 5],
            [4, 1, 0, 1, 7],
            [1, 0, 3, 0, 5]]
  sum_region(2, 1, 4, 3)
Output: 8
Explanation: The 3x3 block rows 2..4, cols 1..3 is [[2,0,1],[1,0,1],[0,3,0]] which sums
to 8.
```

```
Input:  same matrix; sum_region(1, 1, 2, 2)
Output: 11
Explanation: Block [[6,3],[2,0]] sums to 11.
```

```
Input:  same matrix; sum_region(1, 2, 2, 4)
Output: 12
Explanation: Block [[3,2,1],[0,1,5]] sums to 12.
```

Follow-ups

- Follow-up constraint change from the report: what if the matrix is *mutable* (values can be updated between queries)? Hint: switch to a 2D Binary Indexed Tree (Fenwick) for O(log rows * log cols) updates and queries.

Problem 3. Round-Robin Job Scheduler

Source: Tochi Onwuasoanya, 2025-08-19 — technical interview (Slack report) **Fidelity:** Framed to match the described pattern **Topics:** Queue, Simulation, Scheduling **Difficulty:** Medium

Problem

The third question was "not really leetcode style" but was a job-scheduling problem solved with a round-robin approach. Reconstruct it as follows.

You are given a list of jobs, each with a unique string `id` and a positive integer `burst` (the number of time units of CPU it needs). All jobs are available at time 0 and are placed in a ready queue in the given order. The CPU runs jobs round-robin with a fixed time `quantum`: it takes the job at the front of the queue, runs it for `min(quantum, remaining)` time units, and then — if the job still has work left — puts it at the **back** of the queue; otherwise the job completes at that moment.

Return the list of `(id, completion_time)` pairs **in the order the jobs complete**.

Constraints:

- `1 <= len(jobs) <= 10^4`
- `1 <= burst <= 10^6`, `1 <= quantum <= 10^6`
- Job ids are unique, non-empty strings.

Function signature

```
def round_robin(jobs: list[tuple[str, int]], quantum: int) -> list[tuple[str, int]]:
    """Simulate round-robin scheduling; return (id, completion_time) pairs in completion order."""
    ...
```

Examples

```
Input:  jobs = [("A", 4), ("B", 3), ("C", 2)], quantum = 2
Output: [("C", 6), ("A", 8), ("B", 9)]
Explanation:
  t=0–2 A (rem 2), t=2–4 B (rem 1), t=4–6 C completes at 6,
  t=6–8 A completes at 8, t=8–9 B completes at 9.
```

```
Input:  jobs = [("X", 5)], quantum = 2
Output: [("X", 5)]
Explanation: One job, run in slices 2+2+1, finishing at time 5.
```

```
Input:  jobs = [("P", 1), ("Q", 1)], quantum = 3
Output: [("P", 1), ("Q", 2)]
Explanation: Each finishes within a single quantum: P at 1, Q at 2.
```

Follow-ups

- Report note ("a constraint was changed"): add per-job *arrival times* so a job only enters the queue once the clock reaches its arrival. Hint: keep a pointer over arrival-sorted jobs and admit them each time the clock advances.

Problem 4. Decode String

Source: Devan Quinn, 2025-08-19 — technical interview (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Stack, String Parsing, Recursion **Difficulty:** Medium

Problem

Given an encoded string, return its decoded form. The encoding rule is `k[encoded_string]`, where the `encoded_string` inside the brackets is repeated exactly `k` times. `k` is a positive integer. Encodings may be nested.

You may assume the input is always valid: no extra whitespace, brackets are always well-formed, and digits only ever appear as a repeat count `k` (never inside the plain text). All decoded lengths fit comfortably in memory.

Constraints:

- `1 <= len(s) <= 30`
- `s` consists of lowercase English letters, digits, and the characters `[` and `]`.
- `1 <= k <= 300`.

Function signature

```
def decode_string(s: str) -> str:
    """Decode a string encoded with the k[...] repetition rule."""
    ...
```

Examples

```
Input:  s = "3[a]2[bc]"
Output: "aaabcbc"
```

```
Input:  s = "3[a2[c]]"
Output: "accaccacc"
Explanation: Inner "2[c]" -> "cc", giving "acc", repeated 3 times.
```

```
Input:  s = "2[abc]3[cd]ef"
Output: "abcabccdcdcdef"
```

Problem 5. Jump Game II (Minimum Jumps)

Source: Devan Quinn, 2025-08-19 — technical interview (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Array, Greedy, Dynamic Programming **Difficulty:** Medium

Problem

You are given a 0-indexed array of non-negative integers `nums`. You start at index 0. From any index `i`, you can jump to any index in the range `[i + 1, i + nums[i]]` (i.e., `nums[i]` is the maximum forward jump length from `i`).

Return the **minimum number of jumps** required to reach the last index `len(nums) - 1`. The test cases guarantee that the last index is always reachable.

Constraints:

- `1 <= len(nums) <= 10^4`
- `0 <= nums[i] <= 1000`
- If the array has length 1 you are already at the last index, so the answer is 0.

Function signature

```
def jump(nums: list[int]) -> int:
    """Return the minimum number of jumps to reach the last index."""
    ...
```

Examples

```
Input:  nums = [2, 3, 1, 1, 4]
Output: 2
Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.
```

```
Input:  nums = [2, 3, 0, 1, 4]
Output: 2
Explanation: 0 -> 1 -> 4.
```

```
Input:  nums = [0]
Output: 0
Explanation: Already at the last index.
```

Follow-ups

- Solve it in O(n) time and O(1) extra space with a greedy "current reach vs. farthest reach" scan rather than O(n) DP.

Problem 6. American Football Score Combinations

Source: Bryan, 2025-08-25 — technical rounds (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Dynamic Programming, Backtracking, Combinations (unbounded knapsack) **Difficulty:** Medium

Problem

Bryan's DP question was: enumerate all the ways a football team can accumulate a given total number of points. In American football a single scoring play is worth one of: `2` (safety), `3` (field goal), `6` (touchdown), `7` (touchdown + extra point), or `8` (touchdown + 2-point conversion).

Given a target `total`, return **every distinct combination** of scoring plays whose values sum to `total`. Two combinations are the same if they use the same multiset of play values (order does not matter). Each combination should be returned as a list of play values sorted in non-decreasing order. The overall list of combinations may be in any order. A play value may be used any number of times.

Constraints:

- `0 <= total <= 60`
- Allowed play values are exactly `[2, 3, 6, 7, 8]`.
- If `total == 0`, the only combination is the empty one, so return `[[]]`.

Function signature

```
def football_combinations(total: int) -> list[list[int]]:
    """Return all distinct multisets of {2,3,6,7,8} that sum to total, each sorted
ascending."""
    ...
```

Examples

```
Input:  total = 6
Output: [[2, 2, 2], [3, 3], [6]]
Explanation: 2+2+2 = 3+3 = 6 = 6.
```

```
Input:  total = 7
Output: [[2, 2, 3], [7]]
Explanation: 2+2+3 = 7 and the single play 7.
```

```
Input:  total = 0
Output: [[]]
Explanation: The empty combination is the only way to score 0.
```

Follow-ups

- If you only need the *count* of combinations (not the lists), do it with an O(total) DP table (unbounded coin-change counting).

Problem 7. Contains Duplicate

Source: Boris Hernandez Jr., 2025-09-16 — interview/OA question bank (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Array, Hash Set **Difficulty:** Easy

Problem

Given an integer array `nums`, return `True` if any value appears **at least twice** in the array, and `False` if every element is distinct.

Constraints:

- `1 <= len(nums) <= 10^5`
- `-10^9 <= nums[i] <= 10^9`

Function signature

```
def contains_duplicate(nums: list[int]) -> bool:
    """Return True if any value appears more than once."""
    ...
```

Examples

```
Input:  nums = [1, 2, 3, 1]
Output: True
```

```
Input:  nums = [1, 2, 3, 4]
Output: False
```

```
Input:  nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2]
Output: True
```

Problem 8. K-diff Pairs in an Array

Source: Boris Hernandez Jr., 2025-09-16 — interview/OA question bank (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Array, Hash Map, Two Pointers **Difficulty:** Medium

Problem

Given an integer array `nums` and an integer `k`, return the number of **unique** k-diff pairs in the array. A k-diff pair is a pair of integers `(nums[i], nums[j])` where:

- `i != j`, and
- `abs(nums[i] - nums[j]) == k`.

Two k-diff pairs `(a, b)` and `(c, d)` are considered the same (counted once) if they consist of the same pair of *values*. Count each distinct value pair only once.

Constraints:

- `1 <= len(nums) <= 10^4`
- `-10^7 <= nums[i] <= 10^7`
- `0 <= k <= 10^7`

Function signature

```
def count_k_diff_pairs(nums: list[int], k: int) -> int:
    """Return the number of unique value-pairs with absolute difference k."""
    ...
```

Examples

```
Input:  nums = [3, 1, 4, 1, 5], k = 2
Output: 2
Explanation: The unique 2-diff value pairs are (1, 3) and (3, 5).
```

```
Input:  nums = [1, 2, 3, 4, 5], k = 1
Output: 4
Explanation: (1,2), (2,3), (3,4), (4,5).
```

```
Input:  nums = [1, 3, 1, 5, 4], k = 0
Output: 1
Explanation: With k = 0 count values that appear at least twice; only 1 does.
```

Problem 9. Coin Change (Fewest Coins)

Source: Boris Hernandez Jr., 2025-09-16 — interview/OA question bank (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Dynamic Programming, Unbounded Knapsack **Difficulty:** Medium

Problem

You are given an integer array `coins` representing coin denominations and an integer `amount` representing a target total. Return the **fewest number of coins** needed to make up `amount`. You have an unlimited supply of each denomination. If the amount cannot be formed by any combination of the coins, return `-1`.

Constraints:

- `1 <= len(coins) <= 12`
- `1 <= coins[i] <= 2^31 - 1`
- `0 <= amount <= 10^4`
- If `amount == 0`, the answer is `0`.

Function signature

```
def coin_change(coins: list[int], amount: int) -> int:
    """Return the minimum number of coins summing to amount, or -1 if impossible."""
    ...
```

Examples

```
Input:  coins = [1, 2, 5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1.
```

```
Input:  coins = [2], amount = 3
Output: -1
Explanation: 3 cannot be formed using only 2s.
```

```
Input:  coins = [1], amount = 0
Output: 0
```

Problem 10. Find the First Occurrence of a Substring

Source: Boris Hernandez Jr., 2025-09-16 — interview/OA question bank (Slack report) **Fidelity:** Framed to match the described pattern **Topics:** String, Two Pointers, String Matching **Difficulty:** Easy

Problem

Reconstructing the "find substring in array" item as substring search: given two strings `haystack` and `needle`, return the index of the **first** occurrence of `needle` in `haystack`, or `-1` if `needle` is not part of `haystack`.

Constraints:

- `1 <= len(haystack), len(needle) <= 10^4`
- `haystack` and `needle` consist of only lowercase English letters.
- If `needle` is the empty string it is considered to occur at index 0 (though the constraints above make it non-empty).

Function signature

```
def str_str(haystack: str, needle: str) -> int:
    """Return the index of the first occurrence of needle in haystack, or -1."""
    ...
```

Examples

```
Input:  haystack = "sadbutsad", needle = "sad"
Output: 0
Explanation: "sad" occurs at indices 0 and 6; the first is index 0.
```

```
Input:  haystack = "leetcode", needle = "leeto"
Output: -1
Explanation: "leeto" does not occur in "leetcode".
```

```
Input:  haystack = "abc", needle = "c"
Output: 2
```

Follow-ups

- Achieve O(n + m) worst-case time with the Knuth-Morris-Pratt (KMP) failure-function approach instead of the naive O(n * m) scan.

Problem 11. Find Median from a Data Stream

Source: Boris Hernandez Jr., 2025-09-16 — interview/OA question bank (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Heap, Two Heaps, Design **Difficulty:** Hard

Problem

The median of an ordered list is the middle value if the count is odd, or the average of the two middle values if the count is even. Design a data structure that supports adding integers from a stream and querying the median of all values added so far.

Implement the `MedianFinder` class:

- `add_num(num)` adds the integer `num` to the data structure.
- `find_median()` returns the median of all elements added so far, as a float. It is only called after at least one number has been added.

Aim for `add_num` in O(log n) and `find_median` in O(1).

Constraints:

- `-10^5 <= num <= 10^5`
- There will be at most `5 * 10^4` calls total.

Function signature

```
class MedianFinder:
    def __init__(self) -> None:
        """Initialize the data structure (e.g., a max-heap and a min-heap)."""
        ...

    def add_num(self, num: int) -> None:
        """Add num to the stream."""
        ...

    def find_median(self) -> float:
        """Return the median of all numbers added so far."""
        ...
```

Examples

```
Input:
  mf = MedianFinder()
  mf.add_num(1)
  mf.add_num(2)
  mf.find_median()   -> 1.5
  mf.add_num(3)
  mf.find_median()   -> 2.0
Explanation: After [1,2] the median is (1+2)/2 = 1.5; after [1,2,3] it is 2.0.
```

```
Input:
  mf = MedianFinder()
  mf.add_num(5)
  mf.find_median()   -> 5.0
Explanation: A single element is its own median.
```

Problem 12. Guess the Secret Word

Source: Boris Hernandez Jr., 2025-09-16 — interview/OA question bank (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Interactive, Minimax, Hashing, Greedy **Difficulty:** Hard

Problem

You are given a list `words` of distinct candidate words, **all of the same length 6**, consisting of lowercase letters. One of them is a secret word chosen by the judge.

You interact with a `Master` object that exposes one method, `guess(word)`, which returns the number of positions at which your guessed `word` matches the secret word (exact character-and-position matches). If you guess a word that is not in `words`, `guess` returns `-1`.

You may call `Master.guess` at most `allowed` times. You win if at some point `guess(secret) == 6` (i.e., you guessed the secret). Implement `find_secret_word(words, master, allowed)` that makes the guesses; it does not need to return anything, it just must guess the secret within the allowed number of calls.

Constraints:

- `1 <= len(words) <= 100`
- Every word has length exactly 6.
- `allowed` is typically 10; a correct strategy must reliably win within that budget.

A reference `Master` is provided so the problem is fully runnable:

```
class Master:
    def __init__(self, secret: str, word_list: list[str], allowed: int) -> None:
        self._secret = secret
        self._words = set(word_list)
        self._remaining = allowed
        self.won = False

    def guess(self, word: str) -> int:
        if self._remaining <= 0:
            raise RuntimeError("out of guesses")
        self._remaining -= 1
        if word not in self._words:
            return -1
        matches = sum(a == b for a, b in zip(word, self._secret))
        if matches == 6:
            self.won = True
        return matches
```

Function signature

```
def find_secret_word(words: list[str], master: "Master", allowed: int) -> None:
    """Discover the secret word by calling master.guess at most `allowed` times."""
    ...
```

Examples

```
Input:  words = ["acckzz","ccbazz","eiowzz","abcczz"], secret = "acckzz", allowed = 10
Output: master.won == True
Explanation: A guess of "acckzz" returns 6. A good strategy uses match counts from
earlier
             guesses to filter the candidate set, e.g. guessing "abcczz" (returns 3 vs
"acckzz")
             then keeping only candidates that also share 3 matches with "abcczz".
```

```
Input:  words = ["hamada","khaled"], secret = "khaled", allowed = 10
Output: master.won == True
Explanation: Guessing "hamada" returns the match count with the secret; keep candidates
consistent
             with that count, leaving "khaled", then guess it for 6.
```

Follow-ups

- Minimize the *worst-case* number of guesses by, at each step, choosing the guess that minimizes the size of the largest resulting candidate group (a minimax heuristic).

Problem 13. Sustained-Error Time Window

Source: Boris Hernandez Jr., 2025-09-16 — interview/OA question bank (Slack report) **Fidelity:** Completed from a partial description **Topics:** Array, Two Pointers, Window Expansion **Difficulty:** Medium

Problem

Reconstructing the "two arrays - Error and Time" question. You are given two parallel arrays: `times` (timestamps in seconds, strictly increasing) and `errors` (the error reading recorded at each timestamp), where `errors[i]` was recorded at `times[i]`. You are also given three integers: `second`, `interval`, and `baseline`.

Determine whether there exists a contiguous run of readings that:

1. Includes the reading taken at timestamp `second` (guaranteed to exist exactly once in `times`),
2. Has **every** reading in the run satisfy `error >= baseline`, and
3. Spans a duration of at least `interval` seconds — that is, `times[right] - times[left] >= interval`, where `left` and `right` are the endpoints of the run.

Return `True` if such a run exists, otherwise `False`. Note that if the reading at `second` is itself below `baseline`, the answer is immediately `False`.

Constraints:

- `1 <= len(times) == len(errors) <= 10^5`
- `times` is strictly increasing; `0 <= times[i] <= 10^9`
- `-10^9 <= errors[i] <= 10^9`, `baseline` in the same range
- `second` is one of the values in `times`; `0 <= interval <= 10^9`

Function signature

```
def has_sustained_error_window(
    times: list[int],
    errors: list[int],
    second: int,
    interval: int,
    baseline: int,
) -> bool:
    """Return True if a run of readings all >= baseline includes `second` and spans >= interval seconds."""
    ...
```

Examples

```
Input:  times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40],
        second = 1, interval = 1, baseline = 8
Output: True
Explanation: Starting at time 1 (err 10 >= 8) the run extends to time 2 (err 20 >= 8)
but stops
             at time 3 (err 5 < 8). The run covers times 1..2, a span of 1 >= interval
1.
```

```
Input:  times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40],
        second = 4, interval = 2, baseline = 8
Output: False
Explanation: Time 4 (err 30 >= 8) can only extend right to time 5 (err 40 >= 8); time 3
(err 5)
             blocks going left. The run covers times 4..5, span 1 < interval 2.
```

```
Input:  times = [1, 2, 3], errors = [10, 4, 10], second = 2, interval = 0, baseline = 8
Output: False
Explanation: The reading at second=2 is 4 < baseline 8, so no valid run includes it.
```

Problem 14. Insert Delete GetRandom in O(1)

Source: Boris Hernandez Jr., 2025-09-16 — interview/OA question bank (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Hash Map, Dynamic Array, Design **Difficulty:** Medium

Problem

Boris described "creating a class - hash map for one and then other was getting random element and popping from list (had to make into O(1) to pop)." Design a set-like structure supporting insert, remove, and random access, all in average O(1) time.

Implement the `RandomizedSet` class:

- `insert(val)` : inserts `val` if not already present. Returns `True` if the item was added, `False` if it was already there.
- `remove(val)` : removes `val` if present. Returns `True` if the item was removed, `False` if it was not present.
- `get_random()` : returns a random element currently in the set. Each element must have the **same probability** of being returned. It is only called when the set is non-empty.

Every operation must run in **average O(1)** time.

Constraints:

- `-2^31 <= val <= 2^31 - 1`
- At most `2 * 10^5` calls total across all methods.

Function signature

```
import random

class RandomizedSet:
    def __init__(self) -> None:
        """Initialize the structure (e.g., a list of values plus a value->index map)."""
        ...

    def insert(self, val: int) -> bool:
        """Insert val; return True if it was not already present."""
        ...

    def remove(self, val: int) -> bool:
        """Remove val; return True if it was present."""
        ...

    def get_random(self) -> int:
        """Return a uniformly random element currently in the set."""
        ...
```

Examples

```
Input:
  s = RandomizedSet()
  s.insert(1)      -> True
  s.remove(2)      -> False
  s.insert(2)      -> True
  s.get_random()   -> 1 or 2 (each with probability 1/2)
  s.remove(1)      -> True
  s.insert(2)      -> False
  s.get_random()   -> 2 (only element left)
Explanation: Matches the classic Insert/Delete/GetRandom sequence with O(1) operations.
```

```
Input:
  s = RandomizedSet()
  s.insert(5)      -> True
  s.insert(5)      -> False
  s.get_random()   -> 5
Explanation: Duplicate insert returns False; the set still contains one 5.
```

Follow-ups

- Allow duplicate values (a multiset): support `insert` / `remove` of duplicates while keeping `get_random` proportional to multiplicity. Hint: store a set of indices per value.

Problem 15. Count Unique Triplets That Sum to Zero

Source: Danielle Darfour, 2025-09-20 — OA (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Array, Two Pointers, Sorting, Hashing **Difficulty:** Medium

Problem

Given an integer array `nums`, count how many *unique* triplets `(nums[i], nums[j], nums[k])` with three distinct indices `i < j < k` satisfy `nums[i] + nums[j] + nums[k] == 0`.

Two triplets are considered the same if they consist of the same multiset of values, regardless of the indices they came from. Return the number of distinct value-triplets (not the number of index combinations).

Constraints:

- `0 <= len(nums) <= 3000`
- `-10^5 <= nums[i] <= 10^5`
- The array may contain duplicates.
- If `len(nums) < 3`, the answer is `0`.

Function signature

```
def count_zero_sum_triplets(nums: list[int]) -> int:
    ...
```

Examples

```
Input:  nums = [-1, 0, 1, 2, -1, -4]
Output: 2
Explanation: The distinct value-triplets are [-1, -1, 2] and [-1, 0, 1].
             Even though [-1, 0, 1] can be formed with different index sets,
             it is counted once.
```

```
Input:  nums = [0, 0, 0, 0]
Output: 1
Explanation: The only triplet is [0, 0, 0].
```

```
Input:  nums = [1, 2, 3]
Output: 0
Explanation: No three elements sum to 0.
```

Follow-ups

- Return the actual list of unique triplets instead of the count. Hint: sort the array, fix one element, and use two pointers while skipping duplicate values.

Problem 16. Coins on the Table

Source: Danielle Darfour, 2025-09-20 — OA (Slack report) **Fidelity:** Completed from a partial description **Topics:** String, Simulation **Difficulty:** Easy

Problem

There is a table that starts with `0` coins on it. You are given a string `events`, where each character is a single event applied left to right:

- `'+'` places one coin on the table.
- `'-'` removes one coin from the table.

The table can never hold a negative number of coins: a `'-'` event when the table is empty has no effect (it is ignored). After processing every event, return the number of coins on the table.

Constraints:

- `0 <= len(events) <= 10^5`
- Every character of `events` is either `'+'` or `'-'`.
- An empty string returns `0`.

Function signature

```
def coins_on_table(events: str) -> int:
    ...
```

Examples

```
Input:  events = "+++-"
Output: 2
Explanation: +1, +1, +1, then -1 -> 2 coins remain.
```

```
Input:  events = "----"
Output: 0
Explanation: Every removal happens on an empty table and is ignored.
```

```
Input:  events = "+-+-+"
Output: 1
Explanation: 1, 0, 1, 0, 1 -> 1 coin remains.
```

Follow-ups

- Generalize so each event is an integer delta (e.g. `"+3"`, `"-2"`), still clamping the running total at 0. Hint: parse tokens instead of single characters.

Problem 17. Print a Directory Tree

Source: Abbot, 2025-09-21 — technical rounds (Slack report) **Fidelity:** Completed from a partial description
Topics: String, Tree, Recursion **Difficulty:** Medium

Problem

You are given a list of file/directory `paths`, each a `'/'`-separated string (for example `"dir/subdir/file.txt"`). Reproduce what a Unix directory listing tool prints: build the directory hierarchy implied by the paths and return it as a single newline-separated string.

Rules:

- Each path component is a node. A component that appears under the same parent in several paths must be printed only once.
- Indent each node with `2 * depth` spaces, where the top-level components have depth `0`.
- Children are printed in the order they are first encountered across the input list (preserve first-seen order; do not sort).
- The output must not have a trailing newline.

Constraints:

- `0 <= len(paths) <= 10^4`
- Each path is non-empty, has no leading/trailing `'/'`, and no empty components.
- If `paths` is empty, return the empty string `""`.

Function signature

```
def print_directory_tree(paths: list[str]) -> str:
    ...
```

Examples

```
Input:  paths = ["a/b/c", "a/b/d", "a/e"]
Output: "a\n  b\n    c\n    d\n  e"
Explanation: Rendered:
a
  b
    c
    d
  e
```

```
Input:  paths = ["src/main.py", "src/util.py", "README.md"]
Output: "src\n  main.py\n  util.py\nREADME.md"
Explanation: "src" and its two files come first (first-seen order),
             then the top-level "README.md".
```

```
Input:  paths = []
Output: ""
```

Follow-ups

- Add box-drawing connectors (`├──` , `└──` , `│`) like the real `tree` command. Hint: track, per level, whether the current node is the last child of its parent.

Problem 18. Top-K Chatters by Word Count

Source: Evin, 2025-09-21 — technical rounds, interview 1 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Hash Map, Parsing, Heap, Top-K **Difficulty:** Medium

Problem

You are given a chat transcript as a list of message lines. Each line has the form `"Name: message text"`, where everything before the first `": "` is the author's name and everything after it is the message body. Count the number of words each person typed across the entire transcript (a word is a maximal run of non-whitespace characters in the body). A person may appear on many lines; sum their words.

Return the `k` people who typed the most words, as a list of `(name, word_count)` tuples sorted by `word_count` descending. Break ties by name in ascending (lexicographic) order.

Constraints:

- `0 <= len(lines) <= 10^5`
- Every line contains exactly one `": "` separating name from body; the name is non-empty and contains no `": "`.
- The body may be empty (0 words).
- `1 <= k`; if `k` exceeds the number of distinct people, return all of them.

Function signature

```
def top_k_chatters(lines: list[str], k: int) -> list[tuple[str, int]]:
    ...
```

Examples

```
Input:  lines = ["Alice: hello there friend", "Bob: hi", "Alice: how are you"], k = 1
Output: [("Alice", 6)]
Explanation: Alice typed 3 + 3 = 6 words, Bob typed 1. Top 1 is Alice.
```

```
Input:  lines = ["Ann: a b", "Bob: c d", "Cara: e"], k = 2
Output: [("Ann", 2), ("Bob", 2)]
Explanation: Ann and Bob both have 2 words; tie broken alphabetically,
             so Ann precedes Bob. Cara (1 word) is excluded.
```

```
Input:  lines = [], k = 3
Output: []
```

Follow-ups

- The interviewer's real ask was to implement the parsing inside a class alongside stubbed API methods. Wrap this in a `ChatAnalyzer` class that ingests lines incrementally via `add_line(line)` and answers

`top_k(k)` on demand.

Problem 19. Find the Pivot Index (Balanced Split) in O(n)

Source: Evin, 2025-09-21 — technical rounds, interview 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Array, Prefix Sum **Difficulty:** Easy

Problem

Given an integer array `nums`, return the leftmost index `i` such that the sum of all elements strictly to the left of `i` equals the sum of all elements strictly to the right of `i`. The element at `i` itself is not counted on either side. For the first index, the left sum is `0`; for the last index, the right sum is `0`.

If no such index exists, return `-1`. Your solution must run in `O(n)` time and `O(1)` extra space.

Constraints:

- `1 <= len(nums) <= 10^4`
- `-1000 <= nums[i] <= 1000`

Function signature

```
def pivot_index(nums: list[int]) -> int:
    ...
```

Examples

```
Input:  nums = [1, 7, 3, 6, 5, 6]
Output: 3
Explanation: Left of index 3: 1+7+3 = 11. Right of index 3: 5+6 = 11.
```

```
Input:  nums = [1, 2, 3]
Output: -1
Explanation: No index balances the two sides.
```

```
Input:  nums = [2, 1, -1]
Output: 0
Explanation: Left of index 0 is 0; right is 1 + (-1) = 0.
```

Follow-ups

- If several pivots exist, return them all. Hint: keep scanning after the first match, maintaining the running left sum.

Problem 20. Footpath Beauty After Repainting

Source: Moyo, 2025-09-22 — technical rounds, Q1 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Array, Sliding Window, Two Pointers **Difficulty:** Medium

Problem

A footpath is built from `n` blocks laid in a line, painted with one of `k` colors. `block_colors[i]` is the color of block `i` (colors are integers in `1..k`). The *beauty value* for a given favorite color `c` is the length of the longest run of consecutive blocks that are all color `c`.

You are allowed to repaint at most `m` blocks (each to any color you choose). Return the maximum possible beauty value for favorite color `c` after repainting at most `m` blocks.

Constraints:

- `1 <= n <= 10^5`, `block_colors` has length `n`
- `1 <= k`, `1 <= c <= k`, `1 <= block_colors[i] <= k`
- `0 <= m <= n`
- With `m = 0`, the answer is simply the longest existing run of color `c`.

Function signature

```
def max_beauty_after_repaint(block_colors: list[int], c: int, m: int) -> int:
    ...
```

Examples

```
Input:  block_colors = [1, 2, 1, 1, 2, 1], c = 1, m = 1
Output: 4
Explanation: Repaint one of the 2's. The best window has at most one
             non-c block, e.g. indices 2..5 [1,1,2,1] -> repaint the 2 -> 4.
```

```
Input:  block_colors = [3, 3, 3], c = 3, m = 0
Output: 3
Explanation: All blocks are already color 3; no repaint needed.
```

```
Input:  block_colors = [2, 2, 2], c = 1, m = 2
Output: 2
Explanation: Only 2 repaints allowed, so at most 2 consecutive c blocks.
```

Follow-ups

- (Reported) Answer `q` queries, each with a different favorite color, and return the beauty value (with `m` fixed) for each. Hint: a sliding window per query, or precompute runs.
- (Reported) The base version with `m = 0` was asked first; the repaint version is the twist.

Problem 21. Shortest Distance From All Buildings

Source: Moyo, 2025-09-22 — technical rounds, Q2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Grid, BFS, Multi-source Search **Difficulty:** Hard

Problem

You are given a 2D grid where each cell is:

- `0` = empty land (a school can be built here),
- `1` = a house,
- `2` = a tree (an obstacle that cannot be entered or crossed).

You want to build one school on an empty cell so that the total travel distance from every house to the school is minimized. Distance is measured by 4-directional steps (up/down/left/right); trees block movement. A valid school location must be reachable from *every* house.

Return the coordinates `(row, col)` of the empty cell with the minimum total distance to all houses. If several cells tie on total distance, return the one that comes first in row-major order (smallest row, then smallest column). If no empty cell can reach all houses, return `(-1, -1)`.

Constraints:

- `1 <= rows, cols <= 50`
- Each cell is `0`, `1`, or `2`.
- There is at least one house.

Function signature

```
def shortest_distance_from_all_buildings(grid: list[list[int]]) -> tuple[int, int]:
    ...
```

Examples

```
Input:  grid = [[1, 0, 2, 0, 1],
                [0, 0, 0, 0, 0],
                [0, 0, 1, 0, 0]]
Output: (1, 2)
Explanation: Cell (1,2) reaches all three houses with total distance
             3 + 3 + 1 = 7, the minimum over all reachable empty cells.
```

```
Input:  grid = [[1, 0, 1]]
Output: (0, 1)
Explanation: The only empty cell is distance 1 from each house; total 2.
```

```
Input:  grid = [[1, 2, 1]]
Output: (-1, -1)
Explanation: There is no empty cell, so no valid location exists.
```

Follow-ups

- (Reported) Minimize the *average* distance instead of the total. Since the number of houses is fixed, the minimizing cell is the same, but track the count to report the average.

Problem 22. Minimum Makespan on Identical CPUs

Source: Arboy Magomba, 2025-09-23 — technical rounds, interview 1 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Heap, Greedy, Scheduling, Sorting **Difficulty:** Medium

Problem

You have `C` identical CPUs and a list `release_times`, where `release_times[i]` is the earliest time job `i` may start. Every job runs non-preemptively for the same fixed duration `S`. When a job's release time arrives it may start on any free CPU; if all CPUs are busy it waits and starts on the first CPU that becomes free (never before its release time). Each CPU runs one job at a time.

Scheduling optimally, compute the minimum possible *makespan*: the time at which the last job finishes.

Constraints:

- `1 <= C <= 10^5`
- `0 <= len(release_times) <= 10^5`
- `0 <= release_times[i] <= 10^9`
- `1 <= S <= 10^9`
- If there are no jobs, the makespan is `0`.

Target complexity: `O(n log C)` time, `O(C)` space.

Function signature

```
def min_makespan(release_times: list[int], num_cpus: int, duration: int) -> int:
    ...
```

Examples

```
Input:  release_times = [0, 1, 2, 3], num_cpus = 2, duration = 3
Output: 7
Explanation: CPU-A: job@0 -> [0,3), job@2 starts at 3 -> [3,6).
             CPU-B: job@1 -> [1,4), job@3 starts at 4 -> [4,7).
             Last finish = 7.
```

```
Input:  release_times = [0, 0, 0], num_cpus = 1, duration = 2
Output: 6
Explanation: One CPU runs them back to back: [0,2), [2,4), [4,6).
```

```
Input:  release_times = [5], num_cpus = 3, duration = 4
Output: 9
Explanation: The single job starts at its release time 5, finishes at 9.
```

Follow-ups

- (Reported) The naive approach tracked an `O(n*C)` table of CPU availability; reduce it to a min-heap of the `C` next-free times, popping the earliest-free CPU per job after sorting jobs by release time.

Problem 23. Plant Crops in Connected Regions

Source: Arboy Magomba, 2025-09-23 — technical rounds, interview 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Grid, Constructive, Simulation **Difficulty:** Medium

Problem

You are given grid dimensions `n` (rows) and `m` (columns) and a list `counts`, where `counts[i]` is how many cells crop `i` must occupy. The sum of `counts` equals `n * m`, so every cell must be planted.

Fill the grid so that, for each crop `i`, all of its cells form exactly one 4-connected region (cells touching horizontally or vertically, not diagonally). Return any valid arrangement as an `n x m` grid of crop indices.

Constraints:

- `1 <= n, m <= 500`
- `1 <= len(counts)`
- `counts[i] >= 1` and `sum(counts) == n * m`

Any arrangement in which every crop's cells are contiguous (single 4-connected region) is accepted.

Function signature

```
def plant_crops(n: int, m: int, counts: list[int]) -> list[list[int]]:
    ...
```

Examples

```
Input:  n = 2, m = 3, counts = [2, 4]
Output: [[0, 0, 1],
         [1, 1, 1]]
Explanation: Snake-fill order (0,0),(0,1),(0,2),(1,2),(1,1),(1,0).
             Crop 0 takes the first 2 cells; crop 1 the next 4.
             Crop 1's cells (0,2)-(1,2)-(1,1)-(1,0) are 4-connected.
```

```
Input:  n = 1, m = 4, counts = [1, 1, 1, 1]
Output: [[0, 1, 2, 3]]
Explanation: Each crop occupies a single cell (trivially connected).
```

```
Input:  n = 3, m = 2, counts = [6]
Output: [[0, 0],
         [0, 0],
         [0, 0]]
Explanation: One crop fills the whole grid.
```

Follow-ups

- (Reported) Fill the grid within a single nested loop (assign as you traverse) rather than computing an order first and assigning afterward. Same time complexity, different code.
- (Reported) Consider the case where the number of crops equals the number of cells.

Problem 24. Meeting Rooms III

Source: Nathan Samson (peer report), 2025-09-24 — technical round (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Heap, Simulation, Greedy **Difficulty:** Hard

Problem

You have `n` meeting rooms numbered `0` to `n - 1`. You are given a list `meetings` where `meetings[i] = [start_i, end_i]` is a half-open interval `[start_i, end_i)`; all start times are distinct. Meetings are processed in order of increasing start time under these rules:

- When a meeting starts, if any room is free, hold it in the free room with the lowest number.
- If no room is free, the meeting is delayed until the room that will be free earliest becomes available. The delayed meeting keeps its original duration (`end_i - start_i`). If several rooms free up at the same time, pick the lowest-numbered one.

Return the number of the room that held the most meetings. If multiple rooms tie, return the one with the lowest number.

Constraints:

- `1 <= n <= 100`
- `1 <= len(meetings) <= 10^5`
- `0 <= start_i < end_i <= 5 * 10^5`
- All `start_i` are distinct.

Function signature

```
def most_booked_room(n: int, meetings: list[list[int]]) -> int:
    ...
```

Examples

```
Input:  n = 2, meetings = [[0, 10], [1, 5], [2, 7], [3, 4]]
Output: 0
Explanation: Room 0 holds [0,10] then [10,11] (the delayed [3,4]);
             room 1 holds [1,5] then [5,10]. Room 0 holds 2 meetings,
             room 1 holds 2, tie broken to the lower number 0.
```

```
Input:  n = 3, meetings = [[1, 20], [2, 10], [3, 5], [4, 9], [6, 8]]
Output: 1
Explanation: Room 1 ends up holding the most meetings (2).
```

```
Input:  n = 1, meetings = [[0, 2], [3, 4]]
Output: 0
Explanation: With one room, it holds every meeting.
```

Follow-ups

- Track total busy time per room instead of meeting counts. Hint: accumulate `end - start` per room during the same simulation.

Problem 25. Serialize and Deserialize a Graph

Source: Nathan Samson (peer report), 2025-09-24 — technical round (Slack report) **Fidelity:** Completed from a partial description **Topics:** Graph, BFS/DFS, Design, Serialization **Difficulty:** Hard

Problem

Design a codec that can serialize a connected, undirected graph to a string and deserialize that string back into an identical graph. Each node has a unique integer `val` and a list of `neighbors`. The graph may contain cycles. After a round trip, the reconstructed graph must have the same node values and the same adjacency relationships as the original.

You are given the reference node class below. Implement `serialize` (node -> string) and `deserialize` (string -> node). `serialize(None)` must round-trip to `None` via `deserialize`.

Constraints:

- `0 <= number of nodes <= 100`
- Node values are unique.
- The graph is connected when non-empty; you receive any one node as the entry point.

Function signature

```
class Node:
    def __init__(self, val: int = 0, neighbors: list["Node"] | None = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []

class Codec:
    def serialize(self, node: "Node | None") -> str:
        """Encode the whole graph reachable from `node` into a string."""
        ...

    def deserialize(self, data: str) -> "Node | None":
        """Rebuild the graph from the string produced by serialize."""
        ...
```

Examples

```
Input (adjacency list, 1-indexed):
    adjList = [[2, 4], [1, 3], [2, 4], [1, 3]]
    # node 1 <-> 2, 1 <-> 4, 2 <-> 3, 3 <-> 4  (a 4-cycle)
Round trip:
    node1 = build_graph(adjList)
    codec = Codec()
    restored = codec.deserialize(codec.serialize(node1))
Output: restored describes the same 4-cycle:
    node 1's neighbors -> {2, 4}, node 2's -> {1, 3}, etc.
```

```
Input:  adjList = [[]]        # a single node with no neighbors
Output: A single node with val 1 and an empty neighbor list.
```

```
Input:  node = None          # empty graph
Output: codec.deserialize(codec.serialize(None)) is None
```

Follow-ups

- Support directed graphs (asymmetric adjacency). Hint: record each directed edge (u -> v) rather than assuming both directions during deserialization.

Problem 26. Longest Non-Decreasing Subarray From Two Arrays

Source: Isaac, 2025-09-24 — technical round (Slack report) **Fidelity:** Framed to match the described pattern **Topics:** Array, Dynamic Programming **Difficulty:** Medium

Problem

You are given two integer arrays `nums1` and `nums2`, both of length `n`. You must construct a third array `nums3` of length `n` where, for each index `i`, `nums3[i]` is chosen to be either `nums1[i]` or `nums2[i]` (chosen independently per index).

Return the length of the longest *non-decreasing* subarray (a contiguous run) achievable in `nums3` over all valid choices.

Constraints:

- `1 <= n <= 10^5`
- `nums1` and `nums2` have the same length `n`.
- `1 <= nums1[i], nums2[i] <= 10^9`

Target complexity: `O(n)` time, `O(1)` extra space.

Function signature

```
def longest_non_decreasing_subarray(nums1: list[int], nums2: list[int]) -> int:
    ...
```

Examples

```
Input:  nums1 = [2, 3, 1], nums2 = [1, 2, 1]
Output: 2
Explanation: Choosing nums3 = [2, 3, ...] gives the run [2, 3] of length 2.
             No choice yields a non-decreasing run of length 3.
```

```
Input:  nums1 = [1, 3, 2, 1], nums2 = [2, 2, 3, 4]
Output: 4
Explanation: nums3 = [1, 2, 3, 4] (pick nums1[0], nums2[1], nums2[2], nums2[3])
             is non-decreasing over all 4 positions.
```

```
Input:  nums1 = [1, 1], nums2 = [2, 2]
Output: 2
Explanation: Any choice, e.g. [1, 1], is non-decreasing across both.
```

Follow-ups

- (Reported) The interviewer first asked for the longest contiguous non-decreasing subarray of a single array, then generalized to picking per index from two arrays; also asked for the time complexity and to

write test cases.

Problem 27. Filled ASCII Diamond

Source: Augustus, 2025-09-26 — Technical round(s) (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Simulation, Nested loops, Geometry/symmetry **Difficulty:** Easy

Problem

Given a non-negative integer radius `R`, render a filled diamond on a square grid of side `2*R + 1`. The grid is indexed by `(i, j)` with `0 <= i, j <= 2*R`, and its center is `(R, R)`. A cell belongs to the diamond (draw `'x'`) if its Manhattan distance from the center is at most `R`, i.e. `abs(i - R) + abs(j - R) <= R`. Every other cell is background (draw `'_'`).

Return the picture as a list of `2*R + 1` strings, one per row (top to bottom), each of length `2*R + 1`.

Constraints:

- `0 <= R <= 500`.
- For `R = 0` the result is the single cell `["x"]`.

Function signature

```
def render_diamond(R: int) -> list[str]:
    ...
```

Examples

```
Input:  R = 1
Output: ["_x_", "xxx", "_x_"]
Explanation: Center (1,1). Row 0 has distance 1 from center vertically, so only column 1
(distance 0) is inside.
```

```
Input:  R = 2
Output: ["__x__", "_xxx_", "xxxxx", "_xxx_", "__x__"]
Explanation: Each row i allows columns within (R - abs(i-R)) of the center.
```

```
Input:  R = 0
Output: ["x"]
Explanation: A 1x1 grid whose only cell is the center.
```

Follow-ups

- The naive fill is O(R^2). Can you produce each row directly from its width instead of testing every cell? (Row `i` is `'x'` repeated `2*(R - abs(i-R)) + 1` times, padded with `'_'`.)

Problem 28. Infinite Deck Card Duel

Source: Augustus, 2025-09-26 — Technical round(s) (Slack report) **Fidelity:** Completed from a partial description **Topics:** OOP design, Simulation, Randomness **Difficulty:** Medium

Problem

Design an `InfiniteDeck` that can be drawn from forever: each call to `draw()` returns a uniformly random integer card value in `[1, 13]` (cards are never exhausted). Make the deck seedable so games are reproducible.

Then simulate a duel between two players. In each **round**, each player draws `hand_size` cards. The round winner is the player holding the single highest card that round; if both players' highest cards are equal, the round is a **draw** (nobody scores). Play `num_rounds` rounds and tally round wins per player. Finally, declare a **champion**: the player is champion only if they lead the other in round wins **by at least 2**; otherwise there is no champion.

Implement:

- `InfiniteDeck.draw()` returning the next card.
- `round_winner(hand1, hand2)` -> `1`, `2`, or `0` (0 meaning draw), based on the highest card in each hand.
- `decide_champion(wins1, wins2)` -> `1`, `2`, or `0` (0 meaning no champion), applying the "lead by at least 2" rule.
- `play_game(...)` tying it together (uses the deck for randomness).

Constraints:

- `1 <= hand_size <= 20`, `1 <= num_rounds <= 100000`, card values in `[1, 13]`.

Function signature

```
import random

class InfiniteDeck:
    def __init__(self, seed: int | None = None) -> None:
        """Create a deck backed by a (optionally seeded) RNG."""
        ...

    def draw(self) -> int:
        """Return a uniformly random card value in [1, 13]."""
        ...

def round_winner(hand1: list[int], hand2: list[int]) -> int:
    """Return 1 if player 1's max card is higher, 2 if player 2's is higher, 0 on a
tie."""
    ...

def decide_champion(wins1: int, wins2: int) -> int:
    """Return the champion (1 or 2) only if they lead by >= 2 round wins, else 0."""
    ...

def play_game(deck: InfiniteDeck, hand_size: int = 5, num_rounds: int = 100) -> int:
    """Play the duel and return the champion via decide_champion, or 0 if none."""
    ...
```

Examples

```
Input:  round_winner([3, 7, 2], [7, 1, 9])
Output: 2
Explanation: Player 2's highest card (9) beats player 1's highest (7).
```

```
Input:  round_winner([5, 5], [5, 4])
Output: 0
Explanation: Both players' highest card is 5, so the round is a draw.
```

```
Input:  decide_champion(51, 49)  ->  1
        decide_champion(50, 49)  ->  0
Explanation: A lead of exactly 2 (51 vs 49) crowns player 1; a lead of 1 (50 vs 49)
crowns nobody.
```

Follow-ups

- Extend to `n` players: replace the two win counters with a dictionary keyed by player id and pick the highest card across all hands each round.
- Introduce multiple independent `InfiniteDeck` instances (or one shared deck with per-card metadata) to support probabilistic house rules.

Problem 29. Decode String

Source: Joshua Moinzadeh, 2025-09-26 — Technical rounds (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Stack, String parsing **Difficulty:** Medium

Problem

Given an encoded string, return its decoded form. The encoding rule is `k[encoded_string]`, meaning the `encoded_string` inside the brackets is repeated exactly `k` times. `k` is a positive integer and brackets may be nested. The input contains only lowercase letters, digits, and the characters `[` and `]`, and is always well-formed (every `[` has a matching `]`, and every `[` is preceded by a valid count). Digits appear only to denote a repeat count.

Return the fully expanded string.

Constraints:

- `1 <= len(s) <= 30000` for the input string.
- `1 <= k <= 300`.

Function signature

```
def decode_string(s: str) -> str:
    ...
```

Examples

```
Input:  s = "3[a]2[bc]"
Output: "aaabcbc"
Explanation: "a" repeated 3 times, then "bc" repeated twice.
```

```
Input:  s = "3[a2[c]]"
Output: "accaccacc"
Explanation: Inner "2[c]" -> "cc", so "a2[c]" -> "acc", repeated 3 times.
```

```
Input:  s = "2[abc]3[cd]ef"
Output: "abcabccdcdcdef"
Explanation: "abc" x2, then "cd" x3, then the literal "ef".
```

Follow-ups

- Solve it with a single stack in one pass rather than recursion.

Problem 30. Russian Doll Envelopes

Source: Joshua Moinzadeh, 2025-09-26 — Technical rounds (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Sorting, Dynamic programming, Longest increasing subsequence, Binary search **Difficulty:** Hard

Problem

You are given a list of envelopes where `envelopes[i] = [w, h]` is the width and height of envelope `i`. One envelope fits inside another only if BOTH its width and its height are strictly smaller than the other envelope's width and height. Rotation is not allowed. Return the maximum number of envelopes you can nest (Russian-doll style), one inside the next.

Constraints:

- `1 <= len(envelopes) <= 100000`.
- `1 <= w, h <= 100000`.
- Duplicate envelopes are possible and cannot nest into one another.

Function signature

```
def max_envelopes(envelopes: list[list[int]]) -> int:
    ...
```

Examples

```
Input:  envelopes = [[5,4],[6,4],[6,7],[2,3]]
Output: 3
Explanation: One valid chain is [2,3] -> [5,4] -> [6,7].
```

```
Input:  envelopes = [[1,1],[1,1],[1,1]]
Output: 1
Explanation: All envelopes are identical, so none fits inside another.
```

```
Input:  envelopes = [[4,5],[4,6],[6,7],[2,3],[1,1]]
Output: 4
Explanation: [1,1] -> [2,3] -> [4,5] -> [6,7] (or [4,6] -> [6,7]).
```

Follow-ups

- The O(n^2) DP is correct but too slow at the upper limit. Achieve O(n log n): sort by width ascending and, on ties in width, by height DESCENDING, then run patience-sorting / longest-increasing-subsequence on the heights.

Problem 31. Streaming Top-K Words

Source: Alexa Parchment, 2025-09-26 — Technical rounds, interview 1 (Slack report) **Fidelity:** Completed from a partial description **Topics:** Hash map, Heap, Top-K, OOP design **Difficulty:** Medium

Problem

Design a `WordCounter` initialized with an integer `k`. It maintains a map from word (string) to an integer count. Implement `record(word)` which increments that word's count by 1 (creating it at count 1 if new) and then returns the up-to-`k` words with the highest counts, in descending order of count. Break ties between equal counts by ascending lexicographic order of the word. The returned list has length `min(k, number of distinct words seen so far)`.

Constraints:

- `1 <= k <= 100000`.
- Words are non-empty lowercase strings; up to `10^6` calls to `record`.

Function signature

```
class WordCounter:
    def __init__(self, k: int) -> None:
        ...

    def record(self, word: str) -> list[str]:
        """Increment word's count, then return the current top-k words (count desc, then
word asc)."""
        ...
```

Examples

```
Input:  WordCounter(2); record("a"); record("b"); record("b")
Output: ["a"]  then  ["a","b"]  then  ["b","a"]
Explanation: After the first call only "a" exists. After "b", both have count 1 so they
tie and sort a<b. After the second "b", "b" has count 2 and leads "a".
```

```
Input:  WordCounter(1); record("x"); record("y"); record("y")
Output: ["x"]  then  ["x"]  then  ["y"]
Explanation: With counts x=1,y=1 the tie breaks to "x"; once y=2 it becomes the single
top word.
```

Follow-ups

- Discuss the run time of `record`. A naive re-sort is O(m log m) per call; can a heap or bucketed-by-count structure make repeated queries cheaper?

Problem 32. Cake Balance Line

Source: Alexa Parchment, 2025-09-26 — Technical rounds, interview 2 (Slack report) **Fidelity:** Framed to match the described pattern **Topics:** Prefix sums, Array **Difficulty:** Easy

Problem

A cake is described by a list `heights`, where `heights[i]` is the amount of cake sitting at position `i` (positions are left to right). You want to place a single vertical cut **exactly at some position `i`** so that the total amount of cake strictly to the left of the cut equals the total amount strictly to the right of the cut (the cake at position `i` itself is consumed by the cut and counts for neither side).

Return the smallest index `i` where `sum(heights[:i]) == sum(heights[i+1:])`, or `-1` if no such position exists.

Constraints:

- `1 <= len(heights) <= 10^5`.
- `-10^4 <= heights[i] <= 10^4` (heights may be negative in this abstracted version).

Function signature

```
def cake_balance_index(heights: list[int]) -> int:
    ...
```

Examples

```
Input:  heights = [1, 7, 3, 6, 5, 6]
Output: 3
Explanation: Left of index 3 is 1+7+3=11; right of index 3 is 5+6=11.
```

```
Input:  heights = [1, 2, 3]
Output: -1
Explanation: No cut position balances the two sides.
```

```
Input:  heights = [2, 1, -1]
Output: 0
Explanation: Left of index 0 is empty (0); right is 1+(-1)=0.
```

Follow-ups

- Do it in one pass and O(1) extra space by tracking a running left sum against `total - left - heights[i]`.

Problem 33. Maximum-Sum Subarray With Equal Endpoints

Source: Ruby Oliveira, 2025-09-29 — Technical rounds, follow-up (Slack report) **Fidelity:** Completed from a partial description **Topics:** Hash map, Prefix sums, Array **Difficulty:** Medium

Problem

Given an integer array `nums`, choose a (contiguous) subarray whose **first and last elements are equal** and whose sum is as large as possible. A single element is a valid subarray (its first and last elements coincide). Return the chosen subarray itself (as a list). If several subarrays tie for the maximum sum, return the one with the smallest starting index; if they still tie, return the shortest.

Constraints:

- `1 <= len(nums) <= 10^5`.
- `-10^4 <= nums[i] <= 10^4`.

Function signature

```
def max_sum_equal_endpoints(nums: list[int]) -> list[int]:
    ...
```

Examples

```
Input:  nums = [1, 3, -2, 4, 1]
Output: [1, 3, -2, 4, 1]
Explanation: The value 1 appears at indices 0 and 4; that whole window sums to 7,
beating any single element.
```

```
Input:  nums = [5, -10, 5]
Output: [5]
Explanation: The window from the first 5 to the last 5 sums to 0, which is worse than
taking a single 5 (sum 5).
```

```
Input:  nums = [-3, -1, -2]
Output: [-1]
Explanation: All values are distinct, so only single elements qualify; -1 is the
largest.
```

Follow-ups

- Achieve O(n): for each value, you only ever care about pairing the earliest occurrence (with the best prefix sum) against later occurrences.

Problem 34. Store With Cheapest-Item Removal

Source: Ruby Oliveira, 2025-09-29 — Technical rounds, question 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** OOP design, Heap, Bucket/counting **Difficulty:** Medium

Problem

Design a `Store` that holds items. Each item has a string `value`, an integer `price` in **cents**, and an integer `timestamp`. Support:

- `add(value, price, timestamp)`: insert an item.
- `remove_cheapest()`: remove and return the `value` of the item with the smallest price. If several items share the smallest price, remove the one with the smallest `timestamp`. Return `None` if the store is empty.

A heap keyed by `(price, timestamp)` gives O(log n) per operation.

Constraints:

- Prices are integers with `1 <= price <= 10000` (i.e. 1 cent up to $100.00).
- Up to `10^6` operations.

Function signature

```
class Store:
    def __init__(self) -> None:
        ...

    def add(self, value: str, price: int, timestamp: int) -> None:
        ...

    def remove_cheapest(self) -> str | None:
        """Remove and return the value of the lowest-priced item (ties broken by
earliest timestamp)."""
        ...
```

Examples

```
Input:  add("a",5,1); add("b",2,2); add("c",8,3); remove_cheapest()
Output: "b"
Explanation: "b" has the smallest price (2 cents).
```

```
Input:  add("x",7,10); add("y",7,4); remove_cheapest()
Output: "y"
Explanation: Prices tie at 7; "y" wins on the earlier timestamp (4 < 10).
```

```
Input:  remove_cheapest()   # on an empty store
Output: None
```

Follow-ups

- Improve `remove_cheapest` from O(log n) to O(1) amortized using the price bound: keep a length-10001 array indexed by price (each slot a queue/count of items) and track the smallest non-empty price index seen so far.

Problem 35. Delete an Entire Binary Tree

Source: kobbie, 2025-09-29 — Technical rounds, second interview (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Binary tree, DFS, Post-order traversal, O(1) space **Difficulty:** Hard

Problem

You are given the `root` of a binary tree (not a BST); each node has only `left` and `right` child pointers. You must "free" every node so that after the call the tree is empty. Because a node must not be freed before its children, the correct freeing order is a **post-order** traversal.

Implement `delete_tree(root)` that frees all nodes and returns the list of node values in the exact order they are freed (post-order: left subtree, right subtree, then node).

Constraints:

- `0 <= number of nodes <= 10^5`.
- Node values are integers.

Function signature

```
from typing import Optional

class TreeNode:
    def __init__(self, val: int = 0,
                 left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right

def delete_tree(root: Optional[TreeNode]) -> list[int]:
    """Free every node and return the values in the order freed (post-order)."""
    ...
```

Examples

```
Input:  root =    1
                 / \
                2   3
Output: [2, 3, 1]
Explanation: Post-order frees the left child, then the right child, then the root.
```

```
Input:  root = None
Output: []
Explanation: An empty tree frees nothing.
```

```
Input:  root =      1
                   /
                  2
                 /
                3
Output: [3, 2, 1]
Explanation: A left-leaning chain frees deepest-first.
```

Follow-ups

- Recursive DFS uses O(h) stack space. Achieve **O(1) extra space** by repeatedly rotating the tree (Morris-style): while the root has a left child, rotate it up; once the root has no left child, free the root and move to its right child.

Problem 36. Next Upcoming Birthday in a Family Tree

Source: Augustus, 2025-10-02 — Additional technical (tie-breaker) round (Slack report); same question reported by Mahesh Bachu, 2025-09-30 **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Tree traversal, Graph, Date arithmetic **Difficulty:** Medium

Problem

You are given a family tree rooted at "me". Each `Person` node has a `name`, a birthday given as `(month, day)`, and pointers to `parent1` and `parent2` (either may be `None`). Following parent pointers reaches the whole ancestry; treat the reachable set of `Person` nodes as the tree to search (each distinct person counted once, even if reachable by two paths).

Given today's date `(today_month, today_day)`, implement `alarm_next_birthday(me, today_month, today_day)` that scans every person and returns the `(month, day)` of the **soonest upcoming birthday**, where a birthday landing exactly on today counts as upcoming (distance 0) and the calendar wraps around the year. If the tree is empty (`me is None`), return the empty string `""`.

Use this ordering key to compare dates: `key(m, d) = (m − 1) ∗ 31 + d`, and cyclic distance `(key(person) − key(today)) mod 372`. Return the person with the smallest such distance; if several people share that soonest date, return that `(month, day)`.

Constraints:

- `1 <= month <= 12`, `1 <= day <= 31`.
- Up to `10^5` people; parent pointers form a DAG (no cycles).

Function signature

```
from typing import Optional, Union

class Person:
    def __init__(self, name: str, month: int, day: int,
                 parent1: "Optional[Person]" = None,
                 parent2: "Optional[Person]" = None) -> None:
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2

def alarm_next_birthday(me: Optional[Person],
                        today_month: int,
                        today_day: int) -> Union[tuple[int, int], str]:
    """Return (month, day) of the nearest upcoming birthday, or "" if there is
nobody."""
    ...
```

Examples

```
Input:  me = Me(12,1) with parent1 = Alice(9,25), parent2 = Bob(1,10); today = (9, 12)
Output: (9, 25)
Explanation: keys — today=260, Alice=273 (dist 13), Me=342 (dist 82), Bob=10 (dist (10−
260) mod 372 = 122). Alice is soonest.
```

```
Input:  me = None; today = (5, 5)
Output: ""
Explanation: No people to check.
```

```
Input:  me = Me(3,3) alone; today = (3, 3)
Output: (3, 3)
Explanation: The only birthday is today, distance 0.
```

Follow-ups

- Support repeated alarms: after reporting one birthday, the next call returns the following-soonest, and once all have been reported, return `""`.

Problem 37. Bigram Word Predictor

Source: Sackitey Joseph, 2025-10-02 — Technical round, second interview (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** OOP design, Hash map, NLP (bigrams) **Difficulty:** Medium

Problem

Design a `WordPredictor` trained on text and able to predict the most likely word to follow a given word. A **bigram** is a pair of consecutive words within a sentence; **bigram frequency** is how often one word immediately follows another. Training data is a list of sentences, each already tokenized into a list of words.

Implement:

- `train(sentences)` : ingest tokenized sentences, accumulating bigram counts. May be called more than once (counts accumulate).
- `predict(word)` : return the word that most frequently follows `word` . If two candidates tie on frequency, return the lexicographically smallest. If `word` was never seen followed by anything, return `None` .

Optimize `predict` for speed (it should be O(1) after training, not a scan).

Constraints:

- Total tokens across all training calls up to `10^6` .
- Words are non-empty strings.

Function signature

```
class WordPredictor:
    def __init__(self) -> None:
        ...

    def train(self, sentences: list[list[str]]) -> None:
        """Accumulate bigram counts from tokenized sentences."""
        ...

    def predict(self, word: str) -> str | None:
        """Return the most likely following word (ties -> lexicographically smallest),
or None."""
        ...
```

Examples

```
Input:  train([["i","am","happy"], ["i","am","sad"], ["i","like","cats"]]); predict("i")
Output: "am"
Explanation: "i" is followed by "am" twice and "like" once, so "am" wins.
```

```
Input:  (same training)  predict("am")
Output: "happy"
Explanation: "am" is followed by "happy" once and "sad" once; the tie breaks to the
lexicographically smaller "happy".
```

```
Input:  (same training)  predict("cats")
Output: None
Explanation: "cats" is never followed by any word in the training data.
```

Follow-ups

- Precompute and cache each word's best successor at the end of training so `predict` is O(1); if training can happen again later, invalidate/refresh the cache lazily.

Problem 38. Number of Visible People in a Queue

Source: kobbie, 2025-10-02 — Technical round, second interview (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Monotonic stack, Array **Difficulty:** Hard

Problem

`n` people stand in a line, each with a distinct height given in `heights` (person `i` at index `i`). Everyone faces to the **right**. Person `i` can see person `j` (with `j > i`) if and only if every person strictly between them is shorter than both of them, i.e. `min(heights[i], heights[j]) > max(heights[i+1 .. j-1])`.

Return an array `answer` where `answer[i]` is the number of people person `i` can see to their right.

Constraints:

- `1 <= n <= 10^5`.
- `1 <= heights[i] <= 10^5`, all heights distinct.

Function signature

```
def can_see_persons_count(heights: list[int]) -> list[int]:
    ...
```

Examples

```
Input:  heights = [10, 6, 8, 5, 11, 9]
Output: [3, 1, 2, 1, 1, 0]
Explanation: Person 0 (10) sees 6, 8, 11 (stops after the taller 11). Person 4 (11) sees
9. The last person sees nobody.
```

```
Input:  heights = [5, 1, 2, 3, 10]
Output: [4, 1, 1, 1, 0]
Explanation: Person 0 (5) sees 1, 2, 3, then 10 (which is taller, and blocks further
view).
```

```
Input:  heights = [7]
Output: [0]
Explanation: A single person sees nobody.
```

Follow-ups

- Solve in O(n) with a single monotonically decreasing stack processed from right to left.

Problem 39. Meeting Rooms II With Day Wrap-Around

Source: David Faleye, 2025-09-30 — Technical rounds, interview 1 (Slack report) **Fidelity:** Completed from a partial description **Topics:** Intervals, Sorting, Sweep line **Difficulty:** Medium

Problem

A day lasts `L` minutes, with minutes numbered `0 .. L-1`. You are given meetings as `(start, duration)` pairs where `0 <= start < L` and `duration > 0`. A meeting occupies the half-open interval `[start, start + duration)`. If a meeting runs past the end of the day it **wraps around**: a meeting with `start + duration > L` occupies `[start, L)` and `[0, start + duration - L)` (assume no meeting is long enough to wrap more than once, i.e. `duration <= L`).

Return the minimum number of rooms needed so that no two meetings sharing a room ever overlap in time.

Constraints:

- `1 <= L <= 10^9`.
- `1 <= len(meetings) <= 10^5`; `1 <= duration <= L`.

Function signature

```
def min_rooms_wrap(meetings: list[tuple[int, int]], L: int) -> int:
    ...
```

Examples

```
Input:  meetings = [(23, 2), (0, 1), (6, 3)], L = 24
Output: 2
Explanation: (23,2) wraps to [23,24) + [0,1); (0,1) is [0,1). They overlap on [0,1),
needing 2 rooms. (6,3) is separate.
```

```
Input:  meetings = [(1, 2), (2, 2)], L = 24
Output: 2
Explanation: [1,3) and [2,4) overlap on [2,3).
```

```
Input:  meetings = [(0, 5), (10, 5), (20, 4)], L = 24
Output: 1
Explanation: [0,5), [10,15), [20,24) never overlap; one room suffices.
```

Follow-ups

- The sort-based sweep is O(n log n). For integer times, discuss an O(n) approach via radix sort of the start/end events.

Problem 40. Top-K Frequent IP Addresses From Logs

Source: David Faleye, 2025-09-30 — Technical rounds, interview 2 (Slack report) **Fidelity:** Completed from a partial description **Topics:** String parsing, Hash map, Bucket sort, Top-K **Difficulty:** Medium

Problem

You are given a list of raw log lines. Each line contains exactly one IPv4 address (four dot-separated integers `0..255`) somewhere in the text, mixed with other tokens. Parse the IP out of each line, count how often each IP appears, and return the `k` most frequent IP addresses in descending order of count. Break ties between equal counts by ascending IP string (compared as plain strings).

Constraints:

- `1 <= len(logs) <= 10^6`.
- `1 <= k <= number of distinct IPs`.
- Every line contains exactly one valid IPv4 address.

Function signature

```
def top_k_ips(logs: list[str], k: int) -> list[str]:
    ...
```

Examples

```
Input:  logs = ["GET 1.1.1.1", "POST 2.2.2.2", "GET 1.1.1.1", "PUT 1.1.1.1", "GET
2.2.2.2"], k = 1
Output: ["1.1.1.1"]
Explanation: 1.1.1.1 appears 3 times, 2.2.2.2 twice.
```

```
Input:  (same logs), k = 2
Output: ["1.1.1.1", "2.2.2.2"]
Explanation: Both distinct IPs, ordered by descending count.
```

```
Input:  logs = ["a 9.9.9.9 x", "b 8.8.8.8 y", "c 9.9.9.9 z", "d 8.8.8.8 w"], k = 2
Output: ["8.8.8.8", "9.9.9.9"]
Explanation: Counts tie at 2, so the tie breaks by ascending IP string ("8..." <
"9...").
```

Follow-ups

- Two O(n log n) approaches exist (sort by count, or a size-k heap). Optimize to O(n) with bucket sort: index buckets by count and read them from high to low.

Problem 41. Validate Race Results (Cycle Detection)

Source: Praise Olaiya, 2025-10-01 — Technical round, first interview (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Graph, DFS, Cycle detection, Topological order **Difficulty:** Medium

Problem

You are given a list of pairwise race results as `(winner, loser)` string pairs, meaning `winner` beat `loser`. The results are **consistent** if they can all be true simultaneously under a strict ranking — equivalently, if the directed graph (edge `winner -> loser`) contains **no cycle**. Return `True` if the results are valid (acyclic) and `False` otherwise.

Constraints:

- `0 <= len(results) <= 10^5`.
- Player names are non-empty strings; duplicate result pairs may appear.
- An empty result list is valid (`True`).

Function signature

```
def results_are_valid(results: list[tuple[str, str]]) -> bool:
    ...
```

Examples

```
Input:  results = [("A","B"), ("B","C")]
Output: True
Explanation: A beat B beat C — a consistent ranking A > B > C, no cycle.
```

```
Input:  results = [("A","B"), ("B","C"), ("C","A")]
Output: False
Explanation: A > B > C > A is a contradiction (a cycle).
```

```
Input:  results = []
Output: True
Explanation: No results impose no contradiction.
```

Follow-ups

- Recursive DFS can hit Python's recursion limit on long chains; convert to an iterative DFS with an explicit stack, or use Kahn's topological-sort algorithm.

Problem 42. Count Super Streaks

Source: Joel Manu, 2025-10-03 — Technical round (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Sorting, Sliding window, Grouping **Difficulty:** Medium

Problem

A player produces a list of events, each `(event_id, event_type, event_time)`. Sort events by `event_time`. A **streak** is a maximal run of events, consecutive in time order, in which every event has the **same** `event_type` AND the gap between each event and the previous one in the run is at most `T` seconds. A streak ends when the next event has a different `event_type` or its gap from the previous event exceeds `T`.

A streak is a **Super Streak** if it contains at least `N` events AND spans at least `X` seconds (span = last event time minus first event time within the streak). Return the number of Super Streaks.

Constraints:

- `0 <= len(events) <= 10^5`.
- `N >= 1`, `T >= 0`, `X >= 0`; times are non-negative integers, possibly with duplicate timestamps.

Function signature

```
def count_super_streaks(events: list[tuple[int, str, int]],
                        N: int, T: int, X: int) -> int:
    ...
```

Examples

```
Input:  events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40)], N=4, T=10,
X=30
Output: 1
Explanation: One streak of 5 A-events, gaps all 10 (<=T), span 40 (>=30), size 5 (>=4).
Super.
```

```
Input:  events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40),
                  (6,"A",55),(7,"A",56),(8,"A",57)], N=4, T=10, X=30
Output: 1
Explanation: The jump 40->55 (gap 15 > T) breaks the run. The first streak [0..40] is
Super; the second [55,56,57] has only 3 events and span 2, so not Super.
```

```
Input:  events = [(1,"A",0),(2,"B",5),(3,"A",10)], N=2, T=10, X=1
Output: 0
Explanation: Type changes each step, so every streak has length 1 — none reaches N=2.
```

Follow-ups

- Follow-up 1: events are tagged with `user_id`; return a per-user count of Super Streaks.
- Follow-up 2: after processing, support `count_streaks(user_id, time_A, time_B)` returning how many of that user's Super Streaks completed entirely within `[time_A, time_B]`. Store each user's Super-Streak end-times sorted and answer with two binary searches in O(log n) per query.

Problem 43. Meetings Minus a Do-Not-Schedule Block

Source: Joel Manu, 2025-10-03 — Technical round, first round (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Intervals, Merge intervals, Sweep line **Difficulty:** Medium

Problem

You are given a list of `meetings`, each a `(start, end)` interval with `start < end`, and one "do-not-schedule" (DNS) block `(dns_start, dns_end)`. Consider the set of all time points covered by at least one meeting. Return the **smallest** list of non-overlapping, sorted intervals that covers exactly those points **except** any point lying inside the DNS block (treat the DNS block as removing the open coverage on `(dns_start, dns_end)`; equivalently split coverage at those bounds).

Meetings may overlap and are given in any order. Return the result sorted by start time.

Constraints:

- `1 <= len(meetings) <= 10^5`.
- Coordinates are integers; `0 <= start < end <= 10^9`.

Function signature

```
def subtract_dns(meetings: list[tuple[int, int]],
                 dns: tuple[int, int]) -> list[tuple[int, int]]:
    ...
```

Examples

```
Input:  meetings = [(1,7),(5,10),(12,30),(22,40),(50,65)], dns = (20,35)
Output: [(1,10),(12,20),(35,40),(50,65)]
Explanation: Merged coverage is [1,10],[12,40],[50,65]. Removing (20,35) splits [12,40]
into [12,20] and [35,40].
```

```
Input:  meetings = [(0,10)], dns = (0, 10)
Output: []
Explanation: The DNS block removes the entire covered range.
```

```
Input:  meetings = [(0,5),(6,9)], dns = (100, 200)
Output: [(0,5),(6,9)]
Explanation: The DNS block covers nothing that was scheduled.
```

Follow-ups

- Generalize to a list of non-overlapping DNS blocks: merge the meetings first, then walk both sorted lists together subtracting each block.

Problem 44. Restaurant Waitlist

Source: Santiago Palacios, 2025-10-03 — Technical round (intv3) (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** OOP design, Doubly linked list, Hash map **Difficulty:** Medium

Problem

Many groups wait to be seated at a restaurant. Each group has a unique integer `group_id` and a size `>= 1`. Groups are kept in arrival order. When a table of some size frees up, you seat the **earliest-arriving group that fits** (`group_size <= table_size`) and remove it from the waitlist. A group may also leave (be removed) at any time before being seated.

Design `Waitlist` supporting:

- `add_group(group_id, size)` : append a group at the back of the waitlist.
- `remove_group(group_id)` : remove that group (e.g. it left). No-op if absent.
- `match_table(table_size)` : find and remove the earliest-arriving group whose size `<= table_size`, returning its `group_id` ; return `None` if none fits.

Constraints:

- `group_id` values are unique; up to `10^6` operations.
- `add_group` and `remove_group` should be O(1).

Function signature

```
class Waitlist:
    def __init__(self) -> None:
        ...

    def add_group(self, group_id: int, size: int) -> None:
        ...

    def remove_group(self, group_id: int) -> None:
        ...

    def match_table(self, table_size: int) -> int | None:
        """Seat (remove and return) the earliest-arriving group that fits, or None."""
        ...
```

Examples

```
Input:  add_group(1,4); add_group(2,2); add_group(3,6); match_table(3)
Output: 2
Explanation: Group 1 (size 4) does not fit a table of 3; group 2 (size 2) does and
arrived before group 3.
```

```
Input:  (continuing) match_table(6)
Output: 1
Explanation: Now the earliest remaining group that fits size 6 is group 1.
```

```
Input:  add_group(5,10); match_table(4)
Output: None
Explanation: No remaining group has size <= 4.
```

Follow-ups

- Make `add_group` / `remove_group` O(1) with a doubly linked list plus a dict mapping `group_id -> node`. (`match_table` remains a front-to-back scan for the first fit.)

Problem 45. Group Timestamps Into Close Intervals

Source: Mj, 2025-10-03 — Technical round, question 1 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Array, Grouping, One-pass scan **Difficulty:** Easy

Problem

You are given an array of positive integer timestamps in **ascending** order and an integer `k`. Partition the timestamps into maximal runs such that within each run, the difference between every pair of adjacent values is strictly less than `k`. A run breaks wherever an adjacent difference is `>= k`. Return each run as an interval `[start, end]` (its first and last value), in order.

Constraints:

- `1 <= len(timestamps) <= 10^5`, values strictly increasing.
- `k >= 1`.

Function signature

```
def group_into_intervals(timestamps: list[int], k: int) -> list[list[int]]:
    ...
```

Examples

```
Input:  timestamps = [1,2,3,4,11,14,15,17,21,22,23], k = 2
Output: [[1,4],[11,11],[14,15],[17,17],[21,23]]
Explanation: Adjacent diffs of 1 stay in a run; a diff of >=2 (e.g. 4->11, 11->14, 15->17, 17->21) starts a new run.
```

```
Input:  timestamps = [5], k = 100
Output: [[5,5]]
Explanation: A single timestamp forms a run by itself.
```

```
Input:  timestamps = [1,2,3], k = 1
Output: [[1,1],[2,2],[3,3]]
Explanation: A diff of 1 is not strictly less than k=1, so every element stands alone.
```

Follow-ups

- One pass, O(n) time and O(1) extra space (besides the output).

Problem 46. Unique Random Number Generator

Source: Mj, 2025-10-03 — Technical round, question 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** OOP design, Randomness, Fisher-Yates **Difficulty:** Medium

Problem

Design a class `UniqueRandom` initialized with an integer `n`. Each call to `generate()` returns a number drawn uniformly at random from `[0, n-1]` that has **not been returned before**, so across successive calls no value ever repeats. After all `n` distinct values have been produced, further calls return `None`.

Every unused value must remain equally likely on each draw (no bias), and `generate()` must run in O(1) time per call.

Constraints:

- `1 <= n <= 10^7`.
- Support an optional `seed` for reproducible tests.

Function signature

```
import random

class UniqueRandom:
    def __init__(self, n: int, seed: int | None = None) -> None:
        ...

    def generate(self) -> int | None:
        """Return a not-yet-returned number in [0, n-1], or None once all n are
exhausted."""
        ...
```

Examples

```
Input:  g = UniqueRandom(3); [g.generate(), g.generate(), g.generate()]
Output: a permutation of [0, 1, 2]  (e.g. [2, 0, 1])
Explanation: The three calls return all of 0,1,2 in some random order with no repeats.
```

```
Input:  (continuing above)  g.generate()
Output: None
Explanation: All 3 values already returned, so the pool is exhausted.
```

```
Input:  g = UniqueRandom(1); [g.generate(), g.generate()]
Output: [0, None]
Explanation: The only value 0 is returned once, then the generator is exhausted.
```

Follow-ups

- Use a partial Fisher-Yates shuffle: keep a virtual array `0..n-1`, and on the `i`-th call swap a random unused slot into position `i` using a sparse dict of moved entries, giving O(1) time and O(number-of-draws) space.

Problem 47. Constrained Dice Roll Sequences

Source: Abbot, 2025-10-06 — Technical round / tie-breaker (Slack report) **Fidelity:** Completed from a partial description **Topics:** Dynamic programming, Memoization, Counting **Difficulty:** Hard

Problem

You roll a single 6-faced die exactly `n` times. You are given an array `roll_max` of length 6, where `roll_max[i]` is the maximum number of times the value `i + 1` may appear **consecutively**. That is, you may never roll the same value more than `roll_max[i]` times in a row.

Return the number of distinct sequences of `n` rolls that respect every consecutive-count limit. Because the answer can be very large, return it modulo `10**9 + 7`.

Constraints:

- `1 <= n <= 5000`
- `roll_max` has exactly 6 entries, each `1 <= roll_max[i] <= 15`.
- A naive brute force over all `6**n` sequences is far too slow; you must memoize over `(rolls_used, last_value, consecutive_count_of_last_value)`.

Function signature

```
def count_dice_sequences(n: int, roll_max: list[int]) -> int:
    ...
```

Examples

```
Input:  n = 2, roll_max = [1, 1, 2, 2, 2, 3]
Output: 34
Explanation: There are 6*6 = 36 sequences of length 2. Two of them are forbidden:
rolling value 1 twice, and rolling value 2 twice (both have roll_max = 1). 36 - 2 = 34.
```

```
Input:  n = 2, roll_max = [1, 1, 1, 1, 1, 1]
Output: 30
Explanation: No value may repeat consecutively, so the 6 sequences "value v then v" are
removed: 36 - 6 = 30.
```

```
Input:  n = 3, roll_max = [1, 1, 1, 2, 2, 3]
Output: 181
Explanation: Hand-verified against the memoized DP over (rolls_used, last_value,
run_length).
```

Follow-ups

- The interviewer's actual twist used **two dice** per turn (a pair rolled together, with the consecutive limit applied to each die independently). Extend the state to track the run lengths of both dice and count the joint sequences.

Problem 48. Minimum Number of Servers (Meeting Rooms II Variant)

Source: Isis Correa, 2025-10-06 — Technical round 1 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Sorting, Heap, Sweep line, Intervals **Difficulty:** Medium

Problem

A service receives a list of requests, each with a start time and an end time: `requests[i] = [start_i, end_i]`. A single server can handle at most one request at a time, and a request occupies its server for the half-open interval `[start_i, end_i)` (a request ending exactly when another starts can reuse the same server).

Return the **minimum number of servers** needed so that every request is handled.

Constraints:

- `0 <= len(requests) <= 10**5`
- `0 <= start_i < end_i <= 10**9`
- Return `0` for an empty list.

Function signature

```
def min_servers(requests: list[list[int]]) -> int:
    ...
```

Examples

```
Input:  requests = [[0, 30], [5, 10], [15, 20]]
Output: 2
Explanation: [5,10] and [15,20] can share one server; [0,30] overlaps both, so a second
server is needed.
```

```
Input:  requests = [[7, 10], [2, 4]]
Output: 1
Explanation: The two requests do not overlap, so one server suffices.
```

```
Input:  requests = []
Output: 0
Explanation: No requests, no servers.
```

Follow-ups

- Also return, for each request, the id of the server assigned to it.

Problem 49. Random Song Shuffler With a No-Repeat Window

Source: Isis Correa, 2025-10-06 — Technical round 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** OOP design, Hash map, Swap-and-pop, Randomization **Difficulty:** Medium

Problem

Design a shuffler that plays songs at random from a fixed playlist, subject to one rule: the **last `k` songs played may not be replayed**. In other words, a song becomes eligible again only after `k` other (distinct) plays have occurred since it was last played.

Implement the class so that each call to `play()` runs in **O(1)** time (no scanning of the playlist). The intended technique is a `valid_songs` list of currently eligible songs plus a `song_to_index` dictionary, using swap-and-pop to move a just-played song out of the eligible region and to re-admit the song that is leaving the window.

Constraints:

- `1 <= len(songs) <= 10**5`, song titles are distinct strings.
- `0 <= k < len(songs)` (so at least one song is always eligible).
- `play()` returns any eligible song chosen uniformly at random; every returned song must not be among the previous `k` returned songs.

Function signature

```
import random

class SongShuffler:
    def __init__(self, songs: list[str], k: int) -> None:
        """Initialize with the playlist and the no-repeat window size k."""
        ...

    def play(self) -> str:
        """Return a uniformly random eligible song in O(1); it must not be
        one of the last k songs returned. Update state so the window slides."""
        ...
```

Examples

```
Input:  SongShuffler(["A", "B", "C"], k=1); then play(), play(), play(), play()
Output: e.g. "B", "A", "C", "B"
Explanation: k=1 forbids repeating the single most recent song. "B" then "A" is fine
(A != B). Then "C" (C != A). Then "B" is allowed again because the only forbidden song
is the immediately previous one ("C"). Any sequence with no two adjacent equal songs is
valid.
```

```
Input:  SongShuffler(["X", "Y"], k=0); play(), play()
Output: e.g. "Y", "Y"
Explanation: With k=0 there is no restriction, so repeats are allowed.
```

Follow-ups

- The interviewer wanted the naive deque solution replaced by O(1). Show why swap-and-pop with an index map achieves constant time per `play()`.

Problem 50. Alarm for the Next Upcoming Birthday

Source: Zayan Ashraf, 2025-10-07 — Technical round (Slack report) **Fidelity:** Completed from a partial description **Topics:** Arrays, Sorting, Date arithmetic **Difficulty:** Easy

Problem

You maintain a list of people and their birthdays. Given today's date and a list of `(name, month, day)` records, return the name of the person whose birthday comes **soonest** starting from today. Birthdays recur every year, so if no birthday remains later this calendar year, the soonest one is the earliest birthday next year. A birthday that falls exactly on today counts as `0` days away (soonest).

Ignore leap years; treat every date simply as a `(month, day)` pair. If two people share the soonest birthday, return the one that is alphabetically first by name.

Constraints:

- `1 <= len(people) <= 10**4`
- `1 <= month <= 12`, `1 <= day <= 31` (assume valid calendar dates).
- `today` is a valid `(month, day)` pair.

Function signature

```
def next_birthday(today: tuple[int, int], people: list[tuple[str, int, int]]) -> str:
    ...
```

Examples

```
Input:  today = (10, 7), people = [("Alice", 10, 9), ("Bob", 1, 15), ("Carol", 12, 25)]
Output: "Alice"
Explanation: Alice's birthday is 2 days away; Carol's is later this year; Bob's is next
January. Alice is soonest.
```

```
Input:  today = (12, 31), people = [("Alice", 1, 1), ("Dan", 12, 31)]
Output: "Dan"
Explanation: Dan's birthday is today (0 days away), which beats Alice's tomorrow.
```

```
Input:  today = (6, 1), people = [("Eve", 5, 30), ("Finn", 5, 30)]
Output: "Eve"
Explanation: Both birthdays already passed this year, so both are ~next-year and tie;
alphabetical order picks "Eve".
```

Problem 51. Top K Most Frequent IP Addresses

Source: Zayan Ashraf, 2025-10-07 — Technical round (Slack report) **Fidelity:** Completed from a partial description **Topics:** Hash map, Counting, Heap / sorting, String parsing **Difficulty:** Medium

Problem

You are given a list of raw server log lines. Each line begins with an IPv4 address, followed by a space and then arbitrary text (method, path, status, etc.). Parse the IP from each line, count how many times each IP appears, and return the `k` most frequent IPs, ordered from most frequent to least frequent. Break ties between equally frequent IPs by ascending lexicographic order of the IP string.

Constraints:

- `0 <= len(logs) <= 10**6`
- Every non-empty line starts with a valid IPv4 address followed by a space.
- `1 <= k` ; if there are fewer than `k` distinct IPs, return all of them.

Function signature

```
def top_k_ips(logs: list[str], k: int = 10) -> list[str]:
    ...
```

Examples

```
Input:  logs = ["1.1.1.1 - GET /", "2.2.2.2 POST /a", "1.1.1.1 GET /b",
                "3.3.3.3 GET /", "2.2.2.2 GET /c", "1.1.1.1 POST /d"], k = 2
Output: ["1.1.1.1", "2.2.2.2"]
Explanation: Counts are 1.1.1.1 -> 3, 2.2.2.2 -> 2, 3.3.3.3 -> 1. The two most frequent
are returned.
```

```
Input:  logs = ["9.9.9.9 x", "8.8.8.8 y"], k = 10
Output: ["8.8.8.8", "9.9.9.9"]
Explanation: Both appear once; the tie is broken lexicographically, so 8.8.8.8 comes
first.
```

Follow-ups

- If the log stream is far larger than memory, how would you approximate the top-k (e.g. count-min sketch + a small heap)?

Problem 52. Validate a Mahjong-Style Hand of Melds

Source: Justin, 2025-10-08 — Technical round (Slack report) **Fidelity:** Completed from a partial description **Topics:** Backtracking, Recursion, Counting **Difficulty:** Medium

Problem

A hand consists of exactly 12 tiles, where each tile is an integer value in `1..9`. The hand is **valid** if the 12 tiles can be partitioned into exactly 4 melds of 3 tiles each, with no tile reused across melds. A meld is one of:

- a **run**: three tiles of consecutive values, e.g. `[3, 4, 5]`, or
- a **triplet**: three tiles of the same value, e.g. `[7, 7, 7]`.

Given the hand as a list of 12 integers, return `True` if it can be fully partitioned into 4 valid melds, otherwise `False`.

Constraints:

- `len(hand) == 12`, every value in `1..9`.
- Each tile must be used in exactly one meld.

Function signature

```
def is_valid_hand(hand: list[int]) -> bool:
    ...
```

Examples

```
Input:  hand = [1, 2, 3, 4, 5, 6, 7, 8, 9, 2, 2, 2]
Output: True
Explanation: Runs [1,2,3], [4,5,6], [7,8,9] and triplet [2,2,2].
```

```
Input:  hand = [1, 1, 2, 3, 4, 5, 6, 7, 8, 9, 9, 9]
Output: False
Explanation: Removing triplet [9,9,9] leaves 1,1,2,3,4,5,6,7,8 which cannot form three
runs or triplets (value 1 appears only twice, and no arrangement of runs covers both
1s).
```

```
Input:  hand = [1, 1, 1, 2, 2, 2, 3, 3, 3, 4, 4, 4]
Output: True
Explanation: Four triplets, or equivalently four runs [1,2,3],[1,2,3],[1,2,3] plus [4,?]
—
the triplet partition is valid.
```

Follow-ups

- The follow-up asked to detect a hand that is **one tile away** from valid: is there a single tile whose value could be changed to make the hand valid?

Problem 53. Filter Common Elements Per Window (Two Lists)

Source: Sahil, 2025-10-08 — Technical round 1 (Slack report) **Fidelity:** Completed from a partial description **Topics:** Hash set, Sliding window, Arrays **Difficulty:** Medium

Problem

You are given two lists `a` and `b` of equal length `n` and a window size `k` that divides `n`. Split both lists into consecutive, non-overlapping windows of size `k` (window 0 is indices `0..k-1`, window 1 is `k..2k-1`, and so on). For each window, remove from `b`'s window every value that also appears in the corresponding window of `a`. Concatenate the surviving elements of `b`, preserving their original order, and return that list.

Constraints:

- `0 <= n <= 10**5`, `n % k == 0`, `1 <= k <= n` (for `n > 0`).
- Values are integers.

Function signature

```
def filter_windows(a: list[int], b: list[int], k: int) -> list[int]:
    ...
```

Examples

```
Input:  a = [1, 2, 3, 4, 5, 6], b = [3, 7, 8, 6, 9, 5], k = 3
Output: [7, 8, 9]
Explanation: Window 0: a={1,2,3}, b=[3,7,8] -> remove 3 -> [7,8].
Window 1: a={4,5,6}, b=[6,9,5] -> remove 6 and 5 -> [9]. Concatenate -> [7,8,9].
```

```
Input:  a = [1, 1], b = [1, 2], k = 2
Output: [2]
Explanation: Single window: a={1}, b=[1,2] -> remove 1 -> [2].
```

Follow-ups

- The interviewer asked to do the removal **in place / O(1) extra space**. Naive in-place removal becomes O(n*k); discuss compaction with a write pointer to keep it O(n) time.

Problem 54. Count Sorted Strings With a Given Prefix

Source: Sahil, 2025-10-08 — Technical round 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Binary search, Strings **Difficulty:** Medium

Problem

Given a lexicographically sorted array of strings `words` and a query string `prefix`, return the number of strings in `words` that start with `prefix`. Because `words` is sorted, all matching strings occupy a contiguous range; find that range with binary search in `O(log n * L)` time (where `L` is the prefix length), not by scanning.

Constraints:

- `0 <= len(words) <= 10**6`, sorted in non-decreasing lexicographic order.
- `0 <= len(prefix) <= 100`. An empty prefix matches every string.

Function signature

```
def count_with_prefix(words: list[str], prefix: str) -> int:
    ...
```

Examples

```
Input:  words = ["app", "apple", "application", "banana", "bat"], prefix = "app"
Output: 3
Explanation: "app", "apple", "application" all start with "app".
```

```
Input:  words = ["app", "apple", "application", "banana", "bat"], prefix = "ba"
Output: 2
Explanation: "banana" and "bat".
```

```
Input:  words = ["app", "apple", "application", "banana", "bat"], prefix = "cat"
Output: 0
Explanation: No string starts with "cat".
```

Follow-ups

- Many repeated queries: cache counts in a hash map keyed by prefix.
- All queries unique: build a trie of all words storing subtree sizes, so each query is `O(L)`.

Problem 55. Balance Loads by Removing One and Splitting Equally

Source: Yori, 2025-10-09 — Technical round 1 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Arrays, Prefix sums, Enumeration **Difficulty:** Medium

Problem

You are given an array `loads` of positive integers. You must remove **exactly one** element, then decide whether the remaining elements (kept in their original order) can be partitioned into **two or more contiguous groups that all have the same sum**. Return `True` if some choice of removed element and some valid split exists, otherwise `False`.

Constraints:

- `1 <= len(loads) <= 2000`
- `1 <= loads[i] <= 10**6`
- The split must have at least 2 groups, and every group must be non-empty and contiguous.

Function signature

```
def can_balance_after_removal(loads: list[int]) -> bool:
    ...
```

Examples

```
Input:  loads = [5, 3, 1, 4]
Output: True
Explanation: Remove 3 -> remaining [5, 1, 4] splits into [5] and [1, 4], each summing to
5.
```

```
Input:  loads = [1, 1, 1, 1]
Output: True
Explanation: Remove one 1 -> [1, 1, 1] splits into [1], [1], [1], each summing to 1.
```

```
Input:  loads = [1, 2, 3]
Output: False
Explanation: Removing any single element leaves two elements that never form two equal-
sum contiguous groups.
```

Problem 56. Movie Recommendations via Transitive Similarity

Source: Yori, 2025-10-09 — Technical round 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Graph, DFS / Union-Find, Sorting **Difficulty:** Medium

Problem

You are given a target movie name, a dictionary `ratings` mapping each movie to its numeric rating, an integer `n`, and a list `similarities` of `[movieX, movieY]` pairs meaning the two movies are directly similar. Similarity is **transitive**: if A is similar to B and B is similar to C, then A, B, and C all belong to the same similarity group.

Return up to `n` recommended movies from the target's similarity group, **excluding the target itself**, ordered by rating descending (break ties alphabetically by name). If `n == 1`, this returns the single highest-rated similar movie. If the group has fewer than `n` other movies, return all of them.

Constraints:

- Every movie mentioned appears as a key in `ratings`.
- `1 <= n`; ratings are numbers, names are distinct strings.

Function signature

```
def recommend(target: str, ratings: dict[str, float], n: int,
              similarities: list[list[str]]) -> list[str]:
    ...
```

Examples

```
Input:  target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 1,
        similarities = [["A", "B"], ["B", "C"]]
Output: ["C"]
Explanation: A, B, C form one group. Excluding A, the highest rated is C (9).
```

```
Input:  target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 2,
        similarities = [["A", "B"], ["B", "C"]]
Output: ["C", "B"]
Explanation: Similar to A are {B (5), C (9)}; sorted by rating descending -> [C, B].
```

```
Input:  target = "D", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 3,
        similarities = [["A", "B"], ["B", "C"]]
Output: []
Explanation: D is in no similarity pair, so its group is just {D}; excluding D leaves
nothing.
```

Problem 57. Random Generator Without Repeats

Source: Kiitan, 2025-10-10 — Technical round (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** OOP design, Fisher-Yates, Randomization **Difficulty:** Medium

Problem

Design a class `RandomGenerator` initialized with a positive integer `n`. Each call to `generate()` returns one integer chosen uniformly at random from the values in `[0, n-1]` that have **not yet been returned**. After `n` calls every value in `[0, n-1]` has been returned exactly once; a further call must raise `IndexError` (the pool is empty).

Each `generate()` call must run in **O(1)** time. The intended approach maintains an array of the remaining values and, on each call, swaps a randomly chosen remaining value to the end of the active region and shrinks that region (partial Fisher-Yates).

Constraints:

- `1 <= n <= 10**6`.

Function signature

```
import random

class RandomGenerator:
    def __init__(self, n: int) -> None:
        """Prepare to yield a random permutation of [0, n-1], one value per
generate()."""
        ...

    def generate(self) -> int:
        """Return a uniformly random not-yet-returned value in O(1).
        Raise IndexError if all values have already been generated."""
        ...
```

Examples

```
Input:  g = RandomGenerator(3); [g.generate() for _ in range(3)]
Output: e.g. [2, 0, 1]
Explanation: The three calls return a permutation of {0, 1, 2} with no repeats; any of
the 6
permutations is a valid output. A 4th call would raise IndexError.
```

```
Input:  g = RandomGenerator(1); g.generate()
Output: 0
Explanation: Only one value exists.
```

Follow-ups

- Add a `reset()` that makes all `n` values available again in O(1) amortized.

Problem 58. Farthest City for a New Data Center

Source: Kiitan, 2025-10-10 — Technical round (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Graph, Multi-source BFS **Difficulty:** Medium

Problem

Cities are nodes `0..num_cities-1` of an undirected, unweighted graph; `edges[i] = [u, v]` is a direct connection. You are given a non-empty list `dcs` of cities that already host a data center. You want to build a new data center in the city that is **as far as possible** from the nearest existing data center — that is, the city whose shortest-path distance (in number of edges) to its closest data center is maximized.

Return that city's index. Break ties by choosing the smallest index. It is guaranteed every city can reach at least one data center.

Constraints:

- `1 <= num_cities <= 10**5`, `0 <= len(edges) <= 2*10**5`.
- `1 <= len(dcs) <= num_cities`, dcs are distinct valid city indices.

Function signature

```
def farthest_city(num_cities: int, edges: list[list[int]], dcs: list[int]) -> int:
    ...
```

Examples

```
Input:  num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0]
Output: 4
Explanation: Distances from city 0 are 0,1,2,3,4; city 4 is farthest.
```

```
Input:  num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0, 4]
Output: 2
Explanation: Nearest-DC distances are 0->0, 1->1, 2->2, 3->1, 4->0; city 2 is farthest
at distance 2.
```

Follow-ups

- Run a single BFS seeded from all data centers at once (distance 0) instead of a BFS per data center.

Problem 59. First Index Reaching a Threshold (Binary Search)

Source: Tatiane Wu Li, 2025-10-11 — Technical round 1 (Slack report) **Fidelity:** Completed from a partial description **Topics:** Binary search, Arrays **Difficulty:** Easy

Problem

Given a non-decreasing (sorted ascending, duplicates allowed) array `arr` and a target value `k`, return the smallest index `i` such that `arr[i] >= k` — the first position where the array "reaches" the value `k`. If no element is `>= k`, return `-1`. Solve in `O(log n)` time and `O(1)` extra space.

Constraints:

- `0 <= len(arr) <= 10**7`
- Values and `k` are integers.

Function signature

```
def first_index_at_least(arr: list[int], k: int) -> int:
    ...
```

Examples

```
Input:  arr = [1, 2, 2, 3, 5, 7], k = 2
Output: 1
Explanation: arr[1] = 2 is the first element >= 2.
```

```
Input:  arr = [1, 2, 2, 3, 5, 7], k = 4
Output: 4
Explanation: The first element >= 4 is 5 at index 4.
```

```
Input:  arr = [1, 2, 2, 3, 5, 7], k = 8
Output: -1
Explanation: No element is >= 8.
```

Follow-ups

- The interviewer probed space complexity; note the solution is already O(1) space. If `arr` is a huge sorted stream/file, describe reading via random access rather than loading it all.

Problem 60. Mario-Style Score Tracker (OOP)

Source: Tatiane Wu Li, 2025-10-11 — Technical round 2 (Slack report) **Fidelity:** Framed to match the described pattern **Topics:** OOP design, State machine, Simulation **Difficulty:** Easy

Problem

Design a class that tracks the state of a platformer character across a sequence of events. The character has a `score` (starts at 0), a `lives` count (starts at 3), and a `big` flag (starts `False`). Process events with these rules:

- `"coin"`: add `100` to the score.
- `"mushroom"`: add `1000` to the score and set `big = True`.
- `"enemy"`: add `200` to the score (the character stomps an enemy).
- `"damage"`: if `big` is `True`, set `big = False` (no life lost); otherwise decrease `lives` by 1. Score is unchanged.

The character is alive while `lives > 0`. Once `lives` reaches 0, the game is over and any further events are ignored. Implement the class and a method to run a batch of events.

Constraints:

- Event strings are always one of the four listed above.

Function signature

```
class MarioGame:
    def __init__(self) -> None:
        """score=0, lives=3, big=False."""
        ...

    def process(self, event: str) -> None:
        """Apply one event, ignoring it if the game is already over."""
        ...

    def run(self, events: list[str]) -> None:
        """Apply a sequence of events in order."""
        ...

    def get_score(self) -> int:
        ...

    def is_alive(self) -> bool:
        ...
```

Examples

```
Input:  g = MarioGame(); g.run(["coin", "mushroom", "enemy", "damage", "damage"]);
        (g.get_score(), g.is_alive())
Output: (1300, True)
Explanation: coin -> 100; mushroom -> 1100, big=True; enemy -> 1300; damage -> big=False
(no life lost);
damage -> small, lives 3->2. Score 1300, still alive.
```

```
Input:  g = MarioGame(); g.run(["damage", "damage", "damage", "coin"]); (g.get_score(),
g.is_alive())
Output: (0, False)
Explanation: Three small-state hits drop lives 3->0 (game over). The trailing "coin" is
ignored.
```

Problem 61. Minimum Conference Rooms (Meeting Rooms II)

Source: Ikram, 2025-10-13 — Technical round 1 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Sorting, Heap, Sweep line, Intervals **Difficulty:** Medium

Problem

Given an array of meeting time intervals `intervals[i] = [start_i, end_i]`, return the minimum number of conference rooms required so that no two meetings sharing a room overlap. A meeting occupies its room over `[start_i, end_i)`; a meeting that ends exactly when another begins can share the room.

Constraints:

- `0 <= len(intervals) <= 10**5`
- `0 <= start_i < end_i <= 10**9`
- Return `0` for an empty input.

Function signature

```
def min_meeting_rooms(intervals: list[list[int]]) -> int:
    ...
```

Examples

```
Input:  intervals = [[0, 30], [5, 10], [15, 20]]
Output: 2
Explanation: [0,30] overlaps both others; [5,10] and [15,20] can reuse one room -> 2
rooms.
```

```
Input:  intervals = [[7, 10], [2, 4]]
Output: 1
Explanation: The meetings do not overlap.
```

```
Input:  intervals = [[1, 5], [5, 9]]
Output: 1
Explanation: The second starts exactly when the first ends, so one room suffices.
```

Problem 62. Split D/R Sequence Into Balanced Halves (One Cut)

Source: Ahmed Mohammed, 2025-10-13 — Technical round 1 (Slack report) **Fidelity:** Completed from a partial description **Topics:** Prefix sums, Arrays, Sliding window **Difficulty:** Medium

Problem

You are given a string `s` of characters `'D'` and `'R'` whose length is even. A **single cut** at index `i` (with `1 <= i < len(s)`) splits `s` into `s[:i]` and `s[i:]`. The cut is **valid** if each of the two resulting pieces has an equal number of `'D'` and `'R'` characters (that is, within each piece the count of D equals the count of R). Return a sorted list of all valid cut indices; if none exist, return an empty list.

Constraints:

- `2 <= len(s) <= 10**5`, `len(s)` is even.
- `s` contains only `'D'` and `'R'`.

Function signature

```
def balanced_cuts(s: str) -> list[int]:
    ...
```

Examples

```
Input:  s = "DRDR"
Output: [2]
Explanation: Cut at 2 -> "DR" (1 D, 1 R) and "DR" (1 D, 1 R), both balanced. Cuts at 1
and 3 leave unbalanced pieces.
```

```
Input:  s = "DDRR"
Output: []
Explanation: No single cut leaves both halves with equal D and R.
```

```
Input:  s = "DRRD"
Output: [2]
Explanation: Cut at 2 -> "DR" and "RD", both balanced.
```

Follow-ups

- The full problem allowed **two cuts** (three pieces) distributed to two people, where one piece must have as many Ds and Rs as the other two pieces combined. Extend to find such a two-cut split; a prefix-sum + two-sum hash-table idea handles the two-cut case.

Problem 63. Aggregate FizzBuzz Offer Matches

Source: Ahmed Mohammed, 2025-10-13 — Technical round 2 (Slack report) **Fidelity:** Completed from a partial description **Topics:** Queues, Greedy matching, Hash map, Simulation **Difficulty:** Medium

Problem

You are given a list of offers in **priority order** (earliest = highest priority). Each offer is a dict with keys `type` (either `"Fizz"` or `"Buzz"`), `name` (a string), and `quantity` (a positive integer). Fizz offers match against Buzz offers on a first-come, first-served basis, and offers may be **partially** fulfilled.

Process the offers in order, maintaining a FIFO queue of unmatched Fizz offers and a FIFO queue of unmatched Buzz offers. When a new offer arrives, match it against the front of the opposite queue repeatedly, each time consuming `min(remaining quantities)` and recording a match `(fizz_name, buzz_name, matched_quantity)`; drop an offer from the queue once its quantity hits 0. If the arriving offer still has quantity left when the opposite queue empties, enqueue it in its own queue.

Aggregate matches so the same `(fizz_name, buzz_name)` pair is reported once with its summed quantity. Return the aggregated matches, in the order each pair was first produced, formatted as: `"FizzBuzz! <quantity> <FizzName>:<BuzzName>"`.

Constraints:

- `0 <= len(offers) <= 10**4`; quantities are positive integers.

Function signature

```
def fizzbuzz_matches(offers: list[dict]) -> list[str]:
    ...
```

Examples

```
Input:  offers = [
          {"type": "Fizz", "name": "A", "quantity": 5},
          {"type": "Buzz", "name": "X", "quantity": 3},
          {"type": "Buzz", "name": "Y", "quantity": 4},
        ]
Output: ["FizzBuzz! 3 A:X", "FizzBuzz! 2 A:Y"]
Explanation: A(5) waits. X(3) matches A -> (A,X,3), A now 2. Y(4) matches A's remaining
2 ->
(A,Y,2); Y's leftover 2 waits with no Fizz to match.
```

```
Input:  offers = [
          {"type": "Fizz", "name": "A", "quantity": 3},
          {"type": "Buzz", "name": "X", "quantity": 1},
          {"type": "Buzz", "name": "X", "quantity": 2},
        ]
Output: ["FizzBuzz! 3 A:X"]
Explanation: A matches the first X for 1 and the second X for 2. Both are the pair (A,
X),
so they aggregate to a single "FizzBuzz! 3 A:X".
```

Problem 64. Positional Matches Between Two Lists

Source: Nkwa Onwudiwe, 2025-10-15 — Technical round 1 (Slack report) **Fidelity:** Framed to match the described pattern **Topics:** Arrays, Iteration **Difficulty:** Easy

Problem

You are given two lists of equal length, `first` and `second`. Return, in order, the values at every position where the two lists agree — that is, all `first[i]` such that `first[i] == second[i]`.

Constraints:

- `len(first) == len(second)`, `0 <= length <= 10**6`.
- Values are integers.

Function signature

```
def positional_matches(first: list[int], second: list[int]) -> list[int]:
    ...
```

Examples

```
Input:  first = [3, 1, 4, 1, 5, 5, 7, 9], second = [1, 2, 3, 4, 5, 6, 7, 8]
Output: [5, 7]
Explanation: Positions 4 (5 == 5) and 6 (7 == 7) match. (The report described "matches"
between a
digits-of-pi list and an index list.)
```

```
Input:  first = [1, 2, 3], second = [4, 5, 6]
Output: []
Explanation: No position matches.
```

Problem 65. Count Super Streaks

Source: Nkwa Onwudiwe, 2025-10-15 — Technical round 2 (Slack report) **Fidelity:** Completed from a partial description **Topics:** Arrays, Counting, Single pass **Difficulty:** Easy

Problem

A player's match history is given as a list `results` of `1`s (a win) and `0`s (a loss). A **super streak** is a maximal run of consecutive wins whose length is at least `threshold`. Return the number of super streaks. "Maximal" means the run cannot be extended left or right (it is bounded by a loss or the array edge).

Constraints:

- `0 <= len(results) <= 10**6`, each value is `0` or `1`.
- `threshold >= 1` (default `3`).

Function signature

```
def count_super_streaks(results: list[int], threshold: int = 3) -> int:
    ...
```

Examples

```
Input:  results = [1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1], threshold = 3
Output: 2
Explanation: Win runs have lengths 3, 2, 4. Runs of length >= 3 count: the length-3 and
length-4 runs.
```

```
Input:  results = [1, 1], threshold = 3
Output: 0
Explanation: The only win run has length 2 < 3.
```

```
Input:  results = [1, 1, 1, 1, 1], threshold = 3
Output: 1
Explanation: A single run of length 5 counts once.
```

Problem 66. Maximum Nesting Depth of Boxes

Source: Omotayo Oludemi, 2025-10-17 — Technical round 2 (Slack report) **Fidelity:** Completed from a partial description **Topics:** Backtracking, Sorting, Longest chain (DP) **Difficulty:** Medium

Problem

You are given a list of boxes, each described by a `(width, height)` pair. Box `X` can be nested directly inside box `Y` if `X`'s width is **strictly** less than `Y`'s width **and** `X`'s height is strictly less than `Y`'s height (no rotation allowed). Boxes can be nested one inside another to form a chain. Return the **maximum nesting depth**, i.e. the largest number of boxes in any valid nesting chain.

Constraints:

- `0 <= len(boxes) <= 5000`.
- Dimensions are positive integers. Return `0` for an empty list.

Function signature

```
def max_nesting_depth(boxes: list[tuple[int, int]]) -> int:
    ...
```

Examples

```
Input:  boxes = [(1, 1), (2, 2), (3, 3), (2, 3)]
Output: 3
Explanation: (1,1) inside (2,2) inside (3,3) -> depth 3.
```

```
Input:  boxes = [(5, 4), (6, 4), (6, 7), (2, 3)]
Output: 3
Explanation: (2,3) inside (5,4) inside (6,7) -> depth 3. (6,4) cannot nest inside (6,7)
since 6 is not < 6.
```

```
Input:  boxes = [(3, 3), (3, 3)]
Output: 1
Explanation: Equal boxes cannot nest (need strictly smaller), so the best chain is a
single box.
```

Follow-ups

- The interviewer emphasized "combinations." Solve via backtracking over all chains, then optimize to an O(n log n) longest-increasing-subsequence approach (as in Russian Doll Envelopes).

Problem 67. Permutation Sequence

Source: majeed, 2025-10-17 — Technical round 1 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Math, Factorial number system, Combinatorics **Difficulty:** Hard

Problem

The set `[1, 2, ..., n]` has `n!` distinct permutations. Listing them in strictly increasing (lexicographic) order and numbering them from `1`, return the `k`-th permutation as a string. Aim for an `O(n)`-ish construction using the factorial number system rather than generating all permutations.

Constraints:

- `1 <= n <= 9`
- `1 <= k <= n!`

Function signature

```
def get_permutation(n: int, k: int) -> str:
    ...
```

Examples

```
Input:  n = 3, k = 3
Output: "213"
Explanation: Order is 123, 132, 213, 231, 312, 321; the 3rd is "213".
```

```
Input:  n = 4, k = 9
Output: "2314"
Explanation: The 9th permutation of 1234 in lexicographic order is "2314".
```

```
Input:  n = 3, k = 1
Output: "123"
Explanation: The first permutation is the sorted sequence.
```

Follow-ups

- A classmate asked whether a brute-force "generate all and index" is acceptable; discuss why the factorial-base method avoids `O(n!)`.

Problem 68. Paint House II

Source: majeed, 2025-10-17 — Technical round 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Dynamic programming **Difficulty:** Medium

Problem

There are `n` houses in a row, each to be painted one of `k` colors. `costs[i][j]` is the cost to paint house `i` with color `j`. **No two adjacent houses may share the same color.** Return the minimum total cost to paint all houses.

Constraints:

- `costs` is an `n x k` grid, `0 <= n`, `k >= 1` (and `k >= 2` whenever `n >= 2`, so a valid painting exists).
- Costs are non-negative integers. Return `0` for `n == 0`.
- Aim for `O(n*k)` time.

Function signature

```
def min_cost_paint_houses(costs: list[list[int]]) -> int:
    ...
```

Examples

```
Input:  costs = [[1, 5, 3], [2, 9, 4]]
Output: 5
Explanation: Paint house 0 color 0 (1) and house 1 color 2 (4) -> 5; or house 0 color 2
(3) and house 1 color 0 (2) -> 5.
```

```
Input:  costs = [[1, 3], [2, 4]]
Output: 5
Explanation: 1 (house0 color0) + 4 (house1 color1) = 5, the cheapest valid pairing.
```

```
Input:  costs = [[7]]
Output: 7
Explanation: A single house, one color.
```

Follow-ups

- Reduce the per-row transition from `O(k)` per cell to `O(1)` amortized by tracking the two smallest values of the previous row.

Problem 69. Exclude First-K-Unique From Another List

Source: Ahmed Osman, 2025-10-17 — Technical round 1 (Slack report) **Fidelity:** Framed to match the described pattern **Topics:** Hash set, Arrays, Ordered dedup **Difficulty:** Easy

Problem

You are given two lists `list1` and `list2` and an integer `k`. Scan `list1` left to right and collect the **first `k` distinct values** it contains (in order of first appearance; if `list1` has fewer than `k` distinct values, take all of them). Then return `list2` with every occurrence of any of those collected values removed, preserving the order of the remaining elements.

Constraints:

- `0 <= len(list1), len(list2) <= 10**5`, `1 <= k`.
- Values are integers.

Function signature

```
def exclude_first_k_unique(list1: list[int], list2: list[int], k: int) -> list[int]:
    ...
```

Examples

```
Input:  list1 = [3, 3, 1, 4, 1, 5], list2 = [1, 2, 3, 4, 3, 5], k = 2
Output: [2, 4, 5]
Explanation: First 2 distinct values of list1 are {3, 1}. Removing all 3s and 1s from
list2 leaves [2, 4, 5].
```

```
Input:  list1 = [7], list2 = [7, 8, 9], k = 3
Output: [8, 9]
Explanation: list1 has only one distinct value (7); remove all 7s from list2.
```

Problem 70. Can All Runners Be Scheduled (Course Schedule Variant)

Source: Ahmed Osman, 2025-10-17 — Technical round 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Graph, Topological sort, Cycle detection **Difficulty:** Medium

Problem

There are `num_runners` runners labeled `0..num_runners-1`. You are given a list `deps` where `deps[i] = [a, b]` means runner `a` must finish before runner `b` (so `b` depends on `a`). Determine whether there is an ordering in which all runners can run — i.e. whether the dependency graph has **no cycle**. Return `True` if such an ordering exists, otherwise `False`.

Constraints:

- `1 <= num_runners <= 10**5`, `0 <= len(deps) <= 2*10**5`.
- Dependency endpoints are valid runner labels.

Function signature

```
def can_schedule_runners(num_runners: int, deps: list[list[int]]) -> bool:
    ...
```

Examples

```
Input:  num_runners = 3, deps = [[0, 1], [1, 2]]
Output: True
Explanation: Order 0, 1, 2 satisfies all dependencies (no cycle).
```

```
Input:  num_runners = 2, deps = [[0, 1], [1, 0]]
Output: False
Explanation: 0 before 1 and 1 before 0 is a cycle; no valid ordering exists.
```

Follow-ups

- Return an actual valid order (topological sort, Kahn's algorithm) instead of just a boolean.

Problem 71. Find First Occurrence of a Substring (strStr)

Source: Elikem, 2025-10-17 — Technical round 1, question 1 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Strings, Two pointers, KMP **Difficulty:** Easy

Problem

Implement a function that returns the index of the **first occurrence** of `needle` in `haystack`, or `-1` if `needle` is not a substring of `haystack`. By convention, if `needle` is the empty string, return `0`.

Constraints:

- `0 <= len(haystack), len(needle) <= 10**5`.
- Strings consist of printable ASCII characters.

Function signature

```
def str_str(haystack: str, needle: str) -> int:
    ...
```

Examples

```
Input:  haystack = "sadbutsad", needle = "sad"
Output: 0
Explanation: "sad" first appears at index 0 (it also appears at index 6).
```

```
Input:  haystack = "leetcode", needle = "leeto"
Output: -1
Explanation: "leeto" is not a substring.
```

```
Input:  haystack = "abc", needle = ""
Output: 0
Explanation: The empty needle matches at index 0.
```

Follow-ups

- The candidate solved it with KMP; achieve `O(len(haystack) + len(needle))` by precomputing the prefix-function of `needle`.

Problem 72. Nearest Favorite City by Travel Time

Source: Elikem, 2025-10-17 — Technical round 1, question 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Graph, Dijkstra, Shortest paths **Difficulty:** Medium

Problem

You are given a network of `num_cities` cities (`0..num_cities-1`) connected by weighted, undirected roads `roads[i] = [u, v, w]` where `w > 0` is the travel time. Given a `start` city and a non-empty list `favorites` of favorite cities, return the favorite city reachable from `start` in the **shortest total travel time**. Break ties by smallest city index. If none of the favorite cities is reachable, return `-1`.

Constraints:

- `1 <= num_cities <= 10**5`, `0 <= len(roads) <= 2*10**5`, `1 <= w <= 10**9`.
- The interviewer required the optimal solution (Dijkstra with a priority queue).

Function signature

```
def nearest_favorite(num_cities: int, roads: list[list[int]],
                     start: int, favorites: list[int]) -> int:
    ...
```

Examples

```
Input:  num_cities = 5, roads = [[0,1,2],[0,2,5],[1,3,1],[2,4,1]], start = 0, favorites
= [3, 4]
Output: 3
Explanation: Shortest times from 0: to 3 = 2+1 = 3, to 4 = 5+1 = 6. City 3 is nearer.
```

```
Input:  num_cities = 3, roads = [[0,1,4]], start = 0, favorites = [2]
Output: -1
Explanation: City 2 is disconnected from start, so no favorite is reachable.
```

Problem 73. Maximize Shelf Space (Unbounded Knapsack)

Source: Elikem, 2025-10-17 — Technical round 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Dynamic programming, Unbounded knapsack **Difficulty:** Medium

Problem

A shelf has a fixed integer capacity `capacity`. You have boxes of various integer sizes given in `sizes`, and you may use **any number of each box, including duplicates** (each size is available infinitely). Choose boxes so their total size does not exceed `capacity`, maximizing the total size used. Return that maximum total.

Constraints:

- `0 <= capacity <= 10**4`.
- `1 <= len(sizes) <= 100`, each size is a positive integer.

Function signature

```
def max_shelf_space(capacity: int, sizes: list[int]) -> int:
    ...
```

Examples

```
Input:  capacity = 10, sizes = [3, 5, 7]
Output: 10
Explanation: 3 + 7 = 10 exactly fills the shelf.
```

```
Input:  capacity = 11, sizes = [5, 3]
Output: 11
Explanation: 5 + 3 + 3 = 11.
```

```
Input:  capacity = 7, sizes = [4, 5]
Output: 5
Explanation: Options within 7 are 4 or 5 (two boxes would be >= 8). The best is 5.
```

Problem 74. Ways to Sum to N Using 1s and 2s

Source: Nahili, 2025-10-19 — Technical round 1 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Dynamic Programming, Recursion, Combinatorics (Fibonacci / Climbing Stairs) **Difficulty:** Easy

Problem

You are given a non-negative integer `n`. Count the number of distinct *ordered* sequences of `1`s and `2`s whose values add up exactly to `n`. Because the sequences are ordered, `[1, 2]` and `[2, 1]` are considered different.

Return the count as an integer.

Constraints:

- `0 <= n <= 10000`.
- For `n == 0` there is exactly one valid sequence: the empty sequence, so the answer is `1`.
- The count grows like the Fibonacci sequence, so use an integer type that can hold large values (Python's `int` is fine).

Function signature

```
def count_sum_ways(n: int) -> int:
    """Return the number of ordered sequences of 1s and 2s summing to n."""
    ...
```

Examples

```
Input:  n = 1
Output: 1
Explanation: Only [1].

Input:  n = 2
Output: 2
Explanation: [1, 1] and [2].

Input:  n = 3
Output: 3
Explanation: [1, 1, 1], [1, 2], [2, 1].

Input:  n = 4
Output: 5
Explanation: [1,1,1,1], [1,1,2], [1,2,1], [2,1,1], [2,2].
```

Follow-ups

- Instead of the count, return every sequence explicitly. Hint: backtracking; the number of sequences is exponential, so only feasible for small `n`.

Problem 75. Minimum Hands to Play a Piano Melody

Source: Nahili, 2025-10-19 — Technical round 1 (Slack report) **Fidelity:** Framed to match the described pattern **Topics:** Greedy, Patience Sorting, Binary Search **Difficulty:** Medium

Problem

A melody is given as a list `notes`, played strictly left to right in time. A single hand plays a subsequence of the notes, but a hand can only sweep in one direction: the sequence of notes assigned to one hand must be **non-decreasing** (each note it plays is greater than or equal to the previous note that same hand played).

You want to assign every note in the melody to some hand so that each hand's assigned notes (in their original left-to-right order) form a non-decreasing sequence. Return the **minimum number of hands** required.

Constraints:

- `0 <= len(notes) <= 10^5`.
- Each note is an integer, `1 <= notes[i] <= 10^9`.
- For an empty melody the answer is `0`.
- Equal consecutive values are allowed on the same hand (non-decreasing, not strictly increasing).

(The minimum number of non-decreasing subsequences that partition the melody equals the length of the longest strictly decreasing subsequence.)

Function signature

```
from typing import List

def min_hands(notes: List[int]) -> int:
    """Return the minimum number of hands to play the melody, where each
    hand's notes form a non-decreasing subsequence."""
    ...
```

Examples

```
Input:  notes = [1, 2, 3, 4, 5, 6, 4, 3, 2, 1]
Output: 5
Explanation: One hand can cover 1,2,3,4,5,6 (non-decreasing). The trailing
4,3,2,1 is strictly decreasing, so each of those needs its own hand.
Longest strictly decreasing subsequence is 6,4,3,2,1 (length 5).

Input:  notes = [1, 2, 3]
Output: 1
Explanation: A single hand sweeps upward through all notes.

Input:  notes = [3, 2, 1]
Output: 3
Explanation: Strictly decreasing, so no two notes can share a hand.

Input:  notes = [1, 1, 1]
Output: 1
Explanation: Non-decreasing allows equal notes on one hand.
```

Follow-ups

- Solve it in O(n log n). Hint: greedily place each note on the hand whose current top note is the largest value still <= the current note (a `bisect`-driven variant of patience sorting).

Problem 76. Count People in a Postal Code Range

Source: Nahili, 2025-10-19 — Technical round 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Sorting, Binary Search (bisect), Preprocessing for range queries **Difficulty:** Easy

Problem

You are given a list of people, each as a tuple `(name, postal_code)`. Build a structure that can answer many queries of the form: "how many people have a postal code within the inclusive range `[low, high]`?"

The interviewer explicitly wanted the *optimal* query implementation — not a linear scan of the people (or a scan over the range and a hash-map lookup for each value). Preprocess once, then answer each query in `O(log n)`.

Implement a class with:

- a constructor that accepts the list of people, and
- `count_in_range(low, high)` returning the number of people whose postal code is in `[low, high]` inclusive.

Constraints:

- `0 <= number of people <= 10^5`; the same postal code may appear for multiple people (duplicates allowed).
- Postal codes are integers, `0 <= postal_code <= 10^9`.
- `0 <= low <= high <= 10^9`.
- Each `count_in_range` call must run in `O(log n)` after an `O(n log n)` preprocessing step.

Function signature

```
from typing import List, Tuple

class PostalRegistry:
    def __init__(self, people: List[Tuple[str, int]]) -> None:
        """Preprocess the (name, postal_code) pairs for fast range counts."""
        ...

    def count_in_range(self, low: int, high: int) -> int:
        """Return how many people have postal_code in [low, high] inclusive."""
        ...
```

Examples

```
Input:  people = [("Alice", 90210), ("Bob", 10001), ("Cara", 90211), ("Dan", 30301)]
        count_in_range(90000, 99999)
Output: 2
Explanation: Alice (90210) and Cara (90211) fall in the range.

Input:  same people; count_in_range(10001, 30301)
Output: 2
Explanation: Bob (10001) and Dan (30301); range is inclusive on both ends.

Input:  same people; count_in_range(50000, 60000)
Output: 0
Explanation: No postal codes in that range.
```

Follow-ups

- Also return the list of matching names. Hint: sort by `(postal_code, name)` and slice between `bisect_left(low)` and `bisect_right(high)`.

Problem 77. K Largest Elements Across Sorted Lists

Source: Kofi Amo-Antwi, 2025-10-31 — Technical round 1 (Slack report) **Fidelity:** Completed from a partial description **Topics:** Heap, K-way merge, Binary Search **Difficulty:** Medium

Problem

You are given a list of inner lists, where **each inner list is sorted in ascending order**. Return the `k` largest elements taken from the union of all the inner lists, and return them **sorted in ascending order**.

Duplicate values are kept (an element counts once per position it occupies). The interviewer first accepted a simple solution, then asked for an optimized one.

Constraints:

- Let `m` be the number of inner lists and `T` the total number of elements across all of them.
- `0 <= m <= 10^4`, `0 <= T <= 10^6`.
- Each element is an integer in `[-10^9, 10^9]`.
- `0 <= k`. If `k == 0` return `[]`. If `k >= T`, return every element sorted ascending.
- Some inner lists may be empty.

Function signature

```
from typing import List

def k_largest_sorted(lists: List[List[int]], k: int) -> List[int]:
    """Return the k largest elements across all (individually ascending) inner
    lists, in ascending sorted order."""
    ...
```

Examples

```
Input:  lists = [[1, 4, 7], [2, 5, 8], [3, 6, 9]], k = 4
Output: [6, 7, 8, 9]
Explanation: The four largest values overall are 9, 8, 7, 6, returned ascending.

Input:  lists = [[10, 20, 30]], k = 2
Output: [20, 30]

Input:  lists = [[5], [1, 3]], k = 5
Output: [1, 3, 5]
Explanation: k exceeds the total count, so all elements are returned sorted.

Input:  lists = [[1, 2], [3, 4]], k = 0
Output: []
```

Follow-ups

- Optimize beyond concatenate-and-sort. Hint: because each list is already sorted, seed a max-heap with the last element of every non-empty list and pop `k` times, pushing the predecessor within a list after each pop; collect the popped values and reverse them.

Problem 78. Count Axis-Aligned Squares Among Grid Points

Source: Margaret Galvez, 2025-11-03 — Technical round 1 (Slack report) **Fidelity:** Framed to match the described pattern **Topics:** Hashing (set lookup), Geometry, Enumeration **Difficulty:** Medium

Problem

The interviewer described a diagram of squares and rectangles and asked which data structure to use and how to count the squares. Framed concretely: you are given a list of distinct integer lattice `points` in the plane. Count the number of **axis-aligned squares** whose four corners all appear in `points`. A square is axis-aligned when its sides are parallel to the x- and y-axes.

Return the count of such squares.

Constraints:

- `0 <= len(points) <= 2000`; all points are distinct.
- Each point is `(x, y)` with `-10^9 <= x, y <= 10^9`.
- A square must have positive side length (four distinct corners); degenerate zero-size "squares" do not count.
- Only axis-aligned squares count (tilted squares are ignored).

Function signature

```
from typing import List, Tuple

def count_axis_aligned_squares(points: List[Tuple[int, int]]) -> int:
    """Return the number of axis-aligned squares whose 4 corners are all in points."""
    ...
```

Examples

```
Input:  points = [(0, 0), (0, 1), (1, 0), (1, 1)]
Output: 1
Explanation: The four points form exactly one unit square.

Input:  points = [(0,0),(0,1),(1,0),(1,1),(0,2),(1,2),(2,0),(2,1),(2,2)]
Output: 5
Explanation: A 3x3 grid of points has four 1x1 squares and one 2x2 square.

Input:  points = [(0, 0), (2, 0), (0, 1), (2, 1)]
Output: 0
Explanation: These form a 2x1 rectangle, which is not a square.
```

Follow-ups

- A cleaner O(n^2) approach: put all points in a set, then for every pair that could be two opposite corners of a square (equal absolute x- and y-difference), check the other two corners; each square is found via both its diagonals, so divide the total by 2.

Problem 79. Next Birthday (BST In-Order Successor with Wraparound)

Source: Omotayo Oludemi / Victor Olivo, 2025-10-17 — Interview problem discussion (Slack report)
Fidelity: Framed to match the described pattern **Topics:** Binary Search Tree, In-order traversal, Successor
Difficulty: Easy

Problem

This is the "alarm / next-birthday" problem: from a set of birthdays stored in a binary search tree, find the next one that will occur.

Each birthday is stored as an integer day-of-year in `[1, 365]`, and the tree is a valid BST keyed on that value. Given the `root` of the BST and an integer `today`, return the **next upcoming birthday**: the smallest key that is strictly greater than `today`. If no key is strictly greater than `today`, the calendar wraps around, so return the **smallest key in the whole tree**. If the tree is empty, return `None`.

Constraints:

- `0 <= number of nodes <= 10^5`; all node values are distinct and in `[1, 365]`.
- `1 <= today <= 365`.
- Aim for `O(h)` time where `h` is the tree height (do not sort all nodes if you can avoid it).

Function signature

```
from typing import Optional

class TreeNode:
    def __init__(self, val: int, left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right

def next_birthday(root: Optional[TreeNode], today: int) -> Optional[int]:
    """Return the smallest key strictly greater than `today`, wrapping around
    to the smallest key if none is greater. Return None if the tree is empty."""
    ...
```

Examples

```
Tree (BST):        20
                  /  \
                10    30
                        \
                        40

Input:  root = above, today = 25
Output: 30
Explanation: Smallest key strictly greater than 25 is 30.

Input:  root = above, today = 40
Output: 10
Explanation: No key > 40, so wrap around to the smallest key, 10.

Input:  root = above, today = 5
Output: 10
Explanation: Smallest key greater than 5 is 10.

Input:  root = None, today = 100
Output: None
```

Follow-ups

- If the input were an unsorted binary tree (not a BST), extract all values, sort them, and binary-search for the successor. Hint: `bisect_right`.

Problem 80. Count Passcode Patterns on a Keypad

Source: Kayleen Ramírez, 2025-11-07 — Technical round 1 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Graph, DFS, Backtracking **Difficulty:** Medium

Problem

A keypad is a `rows x cols` grid of keys (the classic phone dial is 3x3):

```
1 2 3
4 5 6
7 8 9
```

A **passcode pattern** is a path across keys where each move goes only to an orthogonally adjacent key (horizontal or vertical — no diagonals), and no key is visited more than once. The order matters: `1 -> 2 -> 3 -> 6` and `6 -> 3 -> 2 -> 1` are different patterns.

Given the grid dimensions and an integer `k`, return the number of distinct passcode patterns that use **exactly `k` keys**.

Constraints:

- `1 <= rows, cols <= 4`.
- `1 <= k <= rows * cols`.
- A single key (k = 1) is a valid pattern of length 1.
- Count directed patterns (a path and its reverse are counted separately).

Function signature

```
def count_patterns(rows: int, cols: int, k: int) -> int:
    """Return the number of distinct passcode patterns using exactly k keys,
    moving only to orthogonally adjacent, unvisited keys on a rows x cols grid."""
    ...
```

Examples

```
Input:  rows = 3, cols = 3, k = 1
Output: 9
Explanation: Each of the 9 keys alone is one pattern.

Input:  rows = 3, cols = 3, k = 2
Output: 24
Explanation: There are 12 adjacent key pairs; each can be traversed in 2
directions, giving 24 patterns.

Input:  rows = 3, cols = 3, k = 3
Output: 44
Explanation: Summing over each possible middle key b, deg(b)*(deg(b)-1)
directed 3-key paths: corners give 4*2=8, edge-mids 4*6=24, center 12; total 44.
```

Follow-ups

- Count patterns of length **at least** `k` instead of exactly `k`. Hint: sum the DFS counts over all lengths from `k` to `rows*cols`.

Problem 81. Root a Tree So It Becomes a Binary Tree

Source: Kayleen Ramírez, 2025-11-07 — Technical round 2 (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Graph, Trees, Degree analysis **Difficulty:** Medium

Problem

You are given a connected, undirected, acyclic graph (i.e. a tree) with `n` nodes labeled `0 .. n-1`, described by an edge list. It is guaranteed that **every node has at most 3 neighbors**.

Choose a node to be the root. When rooted at node `r`, one neighbor of each non-root node becomes its parent and the rest become its children; the root has no parent, so all of its neighbors become children. The rooted tree is a **binary tree** if every node has at most 2 children.

Return the smallest label of a node that can serve as such a root. If no node works, return `-1`.

Key observation (the trick): a non-root node with degree `d` has `d - 1` children, so it needs `d <= 3`; the root has `degree` children, so the root needs `degree <= 2`. Since all nodes already have degree `<= 3`, **any node with degree `<= 2` is a valid root** — and leaves (degree 1) always exist in a nonempty tree, so a valid root always exists for `n >= 1`.

Constraints:

- `1 <= n <= 10^5`; the input is a valid tree with `n - 1` edges (for `n == 1`, no edges).
- Every node has at most 3 neighbors.
- Return the smallest valid root label.

Function signature

```
from typing import List, Tuple

def find_binary_tree_root(n: int, edges: List[Tuple[int, int]]) -> int:
    """Return the smallest node label that can root the tree as a binary tree
    (every node has at most 2 children), or -1 if none exists."""
    ...
```

Examples

```
Input:  n = 4, edges = [(0, 1), (0, 2), (0, 3)]
Output: 1
Explanation: Node 0 has degree 3, so as root it would have 3 children (invalid).
Nodes 1, 2, 3 have degree 1; the smallest valid root is 1.

Input:  n = 3, edges = [(0, 1), (1, 2)]
Output: 0
Explanation: A path; every node has degree <= 2, so the smallest label 0 works.

Input:  n = 1, edges = []
Output: 0
Explanation: A single node is trivially a binary tree.
```

Follow-ups

- Now nodes must be 2-colored (black/white) so that every parent differs in color from its children and all nodes at the same depth share a color. Determine whether a valid root plus coloring exists and return one. Hint: coloring each node by `depth % 2` from any chosen root always satisfies both rules in a tree, so a valid coloring exists whenever a valid root does.

Problem 82. Group Shifted Strings

Source: Elyon, 2026-01-20 — Technical round (coding interview 1) (Slack report) **Fidelity:** Reconstructed faithfully from a detailed report **Topics:** Hash Map, String, Grouping **Difficulty:** Medium

Problem

Two strings are "shift buddies" if one can be turned into the other by shifting every character forward by the same amount, with wraparound modulo 26. For example `"abc" -> "bcd" -> "cde" -> ... -> "xyz" -> "yza"` are all shifts of one another, and `"az" -> "ba"` is a valid shift (`a` +1 = `b` , `z` +1 wraps to `a`).

Given a list of lowercase strings, group together all strings that are shift buddies of each other. Return the groups as a list of lists. Strings in a group may be returned in any order, and the groups themselves may be returned in any order.

Constraints:

- `1 <= len(strings) <= 2 * 10^4`
- `0 <= len(strings[i]) <= 50`
- Each `strings[i]` consists only of lowercase English letters (it may be the empty string `""`).
- Two strings can only be shift buddies if they have the same length.
- Duplicate strings are allowed and must each appear in the output (they land in the same group).

Function signature

```
from typing import List

def group_shifted_strings(strings: List[str]) -> List[List[str]]:
    ...
```

Examples

```
Input:  ["abc", "bcd", "acef", "xyz", "az", "ba", "a", "z"]
Output: [["abc", "bcd", "xyz"], ["acef"], ["az", "ba"], ["a", "z"]]
Explanation: "abc","bcd","xyz" share the consecutive-gap pattern (1,1).
             "az" has gap (25); "ba" has gap (b->a = -1 mod 26 = 25) — same group.
             "a" and "z" are single characters, so their (empty) gap pattern
             matches — every length-1 string is a shift buddy of every other.
             "acef" has gaps (2,2,1) and is alone.
```

```
Input:  ["abc", "abc"]
Output: [["abc", "abc"]]
Explanation: Duplicates land in the same group and both are kept.
```

```
Input:  [""]
Output: [[""]]
Explanation: A single empty string forms its own group.
```

Follow-ups

- Start with the brute-force pairwise comparison, then optimize to O(N·K) using a canonical key: the tuple of `(ord(c) - ord(prev)) % 26` gaps between consecutive characters. Discuss time/space complexity of the canonical-form approach.

Problem 83. Range Module (Insert and Point Query)

Source: Elyon, 2026-01-20 — Technical round (coding interview 2) (Slack report) **Fidelity:** Completed from a partial description **Topics:** OOP design, Intervals, Binary Search **Difficulty:** Medium

Problem

Design a data structure that tracks a set of half-open ranges `[left, right)` over the number line and supports two operations:

- `add_range(left, right)`: add the half-open interval `[left, right)` to the tracked set. Overlapping or touching intervals should be merged so the structure always stores a minimal set of disjoint, non-adjacent intervals.
- `query_point(x)`: return `True` if the point `x` is contained in any currently tracked interval, otherwise `False`. Because intervals are half-open, a point equal to a right endpoint is NOT contained.

Constraints:

- Endpoints and query points are integers or floats with `-10^9 <= value <= 10^9`.
- For every `add_range` call, `left < right`.
- Up to `10^5` total operations.
- Touching intervals merge: `add_range(1, 5)` followed by `add_range(5, 10)` yields one interval `[1, 10)`.

Function signature

```
class RangeModule:
    def __init__(self) -> None:
        """Initialize an empty set of tracked ranges."""
        ...

    def add_range(self, left: float, right: float) -> None:
        """Add the half-open interval [left, right), merging with any that overlap or
touch."""
        ...

    def query_point(self, x: float) -> bool:
        """Return True iff x lies inside some tracked half-open interval [left,
right)."""
        ...
```

Examples

```
Input:  rm = RangeModule()
        rm.add_range(10, 20)
        rm.query_point(15)   -> True
        rm.query_point(20)   -> False
        rm.query_point(10)   -> True
Explanation: [10, 20) contains 10 and 15 but not 20 (right endpoint excluded).
```

```
Input:  rm = RangeModule()
        rm.add_range(10, 20)
        rm.add_range(20, 30)
        rm.query_point(20)   -> True
        rm.query_point(25)   -> True
        rm.query_point(30)   -> False
Explanation: The touching ranges merge into [10, 30), so 20 is now covered.
```

```
Input:  rm = RangeModule()
        rm.query_point(5)    -> False
Explanation: No ranges added yet.
```

Follow-ups

- First implement with a plain list of intervals scanned linearly. Then keep the merged intervals sorted and use binary search (e.g. `bisect`) on the left endpoints so `query_point` runs in O(log N). Discuss inclusive vs exclusive endpoint conventions and float precision.

Problem 84. First Out-of-Order Character

Source: Elyon, 2026-01-30 — Technical round (tiebreaker, problem 1) (Slack report) **Fidelity:** Completed from a partial description **Topics:** String, Single Pass **Difficulty:** Easy

Problem

You are given a string of lowercase English letters. Scanning left to right, the string is considered "in order" as long as each character is greater than or equal to the character before it (a non-decreasing sequence; equal adjacent characters are allowed).

Return the index of the FIRST character that breaks this rule — that is, the first index `i >= 1` where `s[i] < s[i-1]`. If the entire string is non-decreasing, return `-1`.

Constraints:

- `0 <= len(s) <= 10^5`
- `s` consists only of lowercase English letters.
- An empty string or a single-character string is trivially in order → return `-1`.

Function signature

```
def first_out_of_order(s: str) -> int:
    ...
```

Examples

```
Input:  "abcda"
Output: 4
Explanation: a<=b<=c<=d holds, but at index 4 'a' < 'd' (s[3]), so 4 is returned.
```

```
Input:  "aabbz"
Output: -1
Explanation: Every character is >= the previous one; fully non-decreasing.
```

```
Input:  "cba"
Output: 1
Explanation: At index 1, 'b' < 'c', the first violation.
```

```
Input:  ""
Output: -1
Explanation: An empty string is trivially in order.
```

Follow-ups

- Solve in a single O(N) pass with O(1) extra space; state the time/space complexity as the interviewer expected.

Problem 85. Radio Broadcast Reachability

Source: Elyon, 2026-01-30 — Technical round (tiebreaker, problem 2) (Slack report) **Fidelity:** Completed from a partial description **Topics:** Graph, BFS, Geometry **Difficulty:** Medium

Problem

You are given a list of radios placed on a 2D plane, each described by its `(x, y)` coordinates. Every radio can broadcast to any other radio within a fixed Euclidean distance `radius` (inclusive). A message can travel from one radio to another either directly (within `radius`) or through a chain of intermediate radios that each relay it to a next radio within `radius`.

Given the radios, the broadcast `radius`, a `source` index, and a `target` index, determine whether a message originating at `source` can reach `target` through zero or more hops. Return `True` or `False`.

Constraints:

- `1 <= len(radios) <= 2000`
- Coordinates are integers or floats with `-10^4 <= x, y <= 10^4`.
- `radius >= 0`.
- `0 <= source, target < len(radios)`.
- Two radios `a` and `b` are directly connected when `sqrt((ax-bx)^2 + (ay-by)^2) <= radius`.
- If `source == target`, the answer is `True`.

Function signature

```
from typing import List, Tuple

def can_reach(radios: List[Tuple[float, float]],
              radius: float,
              source: int,
              target: int) -> bool:
    ...
```

Examples

```
Input:  radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5,
        source = 0, target = 2
Output: True
Explanation: 0->1 (dist 1) and 1->2 (dist 1) are each within 1.5, so 2 is reachable.
```

```
Input:  radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5,
        source = 0, target = 3
Output: False
Explanation: Radio 3 is 3 units from radio 2 (> 1.5) and farther from the rest;
             no chain of hops reaches it.
```

```
Input:  radios = [(0, 0), (3, 4)], radius = 5.0, source = 0, target = 1
Output: True
Explanation: The Euclidean distance is exactly 5.0, which is within radius.
```

Follow-ups

- Model the radios as nodes and connect any two within `radius`, then run BFS (or DFS / union-find) from `source`. To avoid floating-point issues, compare squared distances against `radius * radius`.

Problem 86. Flood Fill

Source: Braden Jones, 2026-01-28 — Technical round (coding interview 1) (Slack report) **Fidelity:** Framed to match the described pattern **Topics:** Graph, DFS, Matrix **Difficulty:** Easy

Problem

You are given an `m x n` grid `image` where `image[r][c]` is the integer color of the pixel at row `r`, column `c`. You are also given a starting pixel `(sr, sc)` and a `new_color`.

Perform a flood fill: starting from `(sr, sc)`, repaint that pixel and every pixel connected to it 4-directionally (up, down, left, right) that shares the SAME original color as the starting pixel, changing them all to `new_color`. Pixels connected only diagonally are not filled. Return the modified grid.

Constraints:

- `1 <= m, n <= 50`
- `0 <= image[r][c], new_color < 2^16`
- `0 <= sr < m`, `0 <= sc < n`
- If the starting pixel already has `new_color`, the grid is returned unchanged (guard against infinite recursion).

Function signature

```
from typing import List

def flood_fill(image: List[List[int]], sr: int, sc: int, new_color: int) ->
List[List[int]]:
    ...
```

Examples

```
Input:  image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, new_color = 2
Output: [[2,2,2],[2,2,0],[2,0,1]]
Explanation: Starting color is 1. All 1-pixels connected 4-directionally to (1,1)
             become 2. The bottom-right 1 is only diagonally reachable, so it stays.
```

```
Input:  image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, new_color = 5
Output: [[5,5,5],[5,5,5]]
Explanation: The whole grid is one connected region of color 0.
```

```
Input:  image = [[0,0,0],[0,1,0]], sr = 1, sc = 1, new_color = 1
Output: [[0,0,0],[0,1,0]]
Explanation: The start pixel already equals new_color, so nothing changes.
```

Follow-ups

- Implement with DFS (recursive or explicit stack) or BFS; discuss the base case that prevents re-visiting when `new_color` equals the original color.

Problem 87. Word Break: All Sentences

Source: Braden Jones, 2026-01-28 — Technical round (coding interview 2) (Slack report) **Fidelity:** Completed from a partial description **Topics:** Backtracking, Recursion, Memoization, String **Difficulty:** Hard

Problem

You are given a string `s` and a dictionary `word_dict` of distinct words. Return ALL possible ways to segment `s` into a sequence of space-separated words such that every word in the sequence appears in `word_dict`. A dictionary word may be reused any number of times. Each returned sentence is a single string of the chosen words joined by single spaces. The sentences may be returned in any order.

Constraints:

- `1 <= len(s) <= 20`
- `1 <= len(word_dict) <= 1000`
- `1 <= len(word) <= 10` for each word; words consist of lowercase letters and are distinct.
- If no valid segmentation exists, return an empty list `[]`.

Function signature

```
from typing import List

def word_break_all(s: str, word_dict: List[str]) -> List[str]:
    ...
```

Examples

```
Input:  s = "catsanddog", word_dict = ["cat","cats","and","sand","dog"]
Output: ["cats and dog", "cat sand dog"]
Explanation: Two segmentations use only dictionary words.
```

```
Input:  s = "pineapplepenapple",
        word_dict = ["apple","pen","applepen","pine","pineapple"]
Output: ["pine apple pen apple", "pineapple pen apple", "pine applepen apple"]
Explanation: All three segmentations consist entirely of dictionary words.
```

```
Input:  s = "catsandog", word_dict = ["cats","dog","sand","and","cat"]
Output: []
Explanation: No segmentation covers the whole string with dictionary words.
```

Follow-ups

- Solve with backtracking: at each position try every dictionary word that is a prefix of the remaining string, then recurse. Add memoization keyed on the suffix (or start index) to reuse partial results and avoid

exponential re-work.

Problem 88. Caesar Cipher: Recover Key, Encrypt, and Decrypt

Source: Moyo, 2026-02-08 — Technical round (3rd interview) (Slack report) **Fidelity:** Framed to match the described pattern **Topics:** String, Modular Arithmetic, Reverse Engineering **Difficulty:** Easy

Problem

A message was scrambled with a Caesar cipher: every lowercase letter was shifted forward by the same secret amount `k` (0 <= k < 26) with wraparound (`z` +1 -> `a`); all non-lowercase characters were left unchanged.

You are given one example pair — a known `plaintext` and its corresponding `ciphertext` — from which you must recover the secret shift, and then implement the cipher in both directions. Specifically implement:

- `find_shift(plaintext, ciphertext) -> int` : return the shift `k` (0..25) that maps `plaintext` to `ciphertext` . The two strings have equal length and contain at least one lowercase letter that pins down `k` .
- `encrypt(text, k) -> str` : shift each lowercase letter forward by `k` with wraparound; leave other characters unchanged.
- `decrypt(text, k) -> str` : the inverse of `encrypt` — shift each lowercase letter backward by `k` with wraparound.

Constraints:

- `1 <= len(text) <= 10^5` .
- `text` may contain lowercase letters, spaces, digits, and punctuation; only lowercase letters `a` – `z` are shifted.
- For `find_shift` , `len(plaintext) == len(ciphertext)` and they align character by character.

Function signature

```
def find_shift(plaintext: str, ciphertext: str) -> int:
    ...

def encrypt(text: str, k: int) -> str:
    ...

def decrypt(text: str, k: int) -> str:
    ...
```

Examples

```
Input:  find_shift("abc", "def")
Output: 3
Explanation: 'a'->'d', 'b'->'e', 'c'->'f' is a uniform forward shift of 3.
```

```
Input:  encrypt("hello world", 3)
Output: "khoor zruog"
Explanation: Each letter moves forward 3 ('h'->'k', 'o'->'r', 'w'->'z'); the space
             is unchanged.
```

```
Input:  decrypt("khoor zruog", 3)
Output: "hello world"
Explanation: decrypt inverts encrypt, so the original message is recovered.
```

```
Input:  encrypt("xyz", 3)
Output: "abc"
Explanation: Wraparound: 'x'->'a', 'y'->'b', 'z'->'c'.
```

Follow-ups

- Given only a ciphertext (no key and no plaintext), how would you break the cipher? Discuss trying all 26 shifts (brute force) or English letter-frequency analysis.